MesoConnect Atlas Tutorial

Supplementary Methods

Corridor-constrained tractography and along-tract microstructure for mesolimbic pathways

November 2026

https://diffusiontensorimaging-repos.github.io/MesoConnect-Tutorial/

Introduction

The MesoConnect Atlas is a probabilistic atlas of mesolimbic white-matter pathways derived from 7 T diffusion magnetic resonance imaging data of the Human Connectome Project (HCP; Vu et al., 2015). It comprises seven bilateral pathways connecting the ventral tegmental area (VTA), hippocampus, nucleus accumbens, ventral pallidum and amygdala, distributed as a single package (Downloads). This tutorial describes how to reconstruct those pathways in new participants using the atlas as an anatomical constraint on participant-level tractography, and how to quantify microstructure along the reconstructed bundles.

Scope

The procedure comprises nine steps. The atlas map for a tract is registered to each participant and dilated into a corridor; probabilistic tractography is run through the corridor from the pathway's seed region to its target; the resulting bundle is cleaned of outlying streamlines; and scalar maps, fractional anisotropy (FA) and the indices of neurite orientation dispersion and density imaging (NODDI; Zhang et al., 2012), are sampled at 100 equidistant points along the bundle. Group-level inference may be performed at three resolutions, the whole tract, quartiles of the tract, or individual nodes, and the tutorial describes the model and correction appropriate to each. Node-wise results may be inspected in the Node-wise Tract Explorer, a browser-based viewer distributed with this site that renders *t*-value profiles, clusters and hemispheric comparisons from a results file.

Three alternative uses of the atlas, averaging a scalar map within the warped atlas mask, warping the group-average streamlines distributed with the atlas into each participant, and sampling along an atlas-derived centerline, are described in the appendix. Corridor-constrained tractography is presented as the primary method because the streamlines are estimated from each participant's own data and support along-tract analysis.

Prerequisites

The tutorial assumes preprocessed diffusion data (denoised, corrected for susceptibility and eddy-current distortion, and brain-masked), a T1-weighted anatomical image, and a Unix environment with FSL (Jenkinson et al., 2012), ANTs (Avants et al., 2008), MRtrix3 (Tournier et al., 2019) and Python with DIPY (Garyfallidis et al., 2014) and pyAFQ (Kruper et al., 2021). Preprocessing is documented in the Diffusion MRI Preprocessing tutorial, which ends with the exact set of files this workflow expects; its list of required outputs lists them and names the step that produces each. QSIPrep, a containerized preprocessing pipeline for diffusion MRI, produces equivalent inputs.

Organization

Table S1 summarizes the sections of the site.

Table S1

Sections of the Tutorial

Section	Contents
Atlas	The seven pathways, how the atlas was constructed, sources of the seed and target regions, the complete package for download
Workflow	The nine steps, each with parameters, verification criteria, the full script, and illustrative output from an example dataset
Explorer	The results viewer and its input format
Alternative approaches	Warping the atlas streamlines; whole-tract extraction; atlas-guided synthetic streamlines
Reference	Consolidated parameters, troubleshooting, software, scripts, suggested methods text, abbreviations, citation, references

Example dataset

Figures and illustrative values on the workflow pages are drawn from a single dataset of 57 adults scanned at 3 T with a multi-shell protocol (b = 1000, 2000, 3250 and 5000 s/mm²) and processed for the posterior and anterior VTA → hippocampus pathways. The atlas itself was constructed at 7 T. The example dataset is included to illustrate expected output at a field strength typical of most studies.

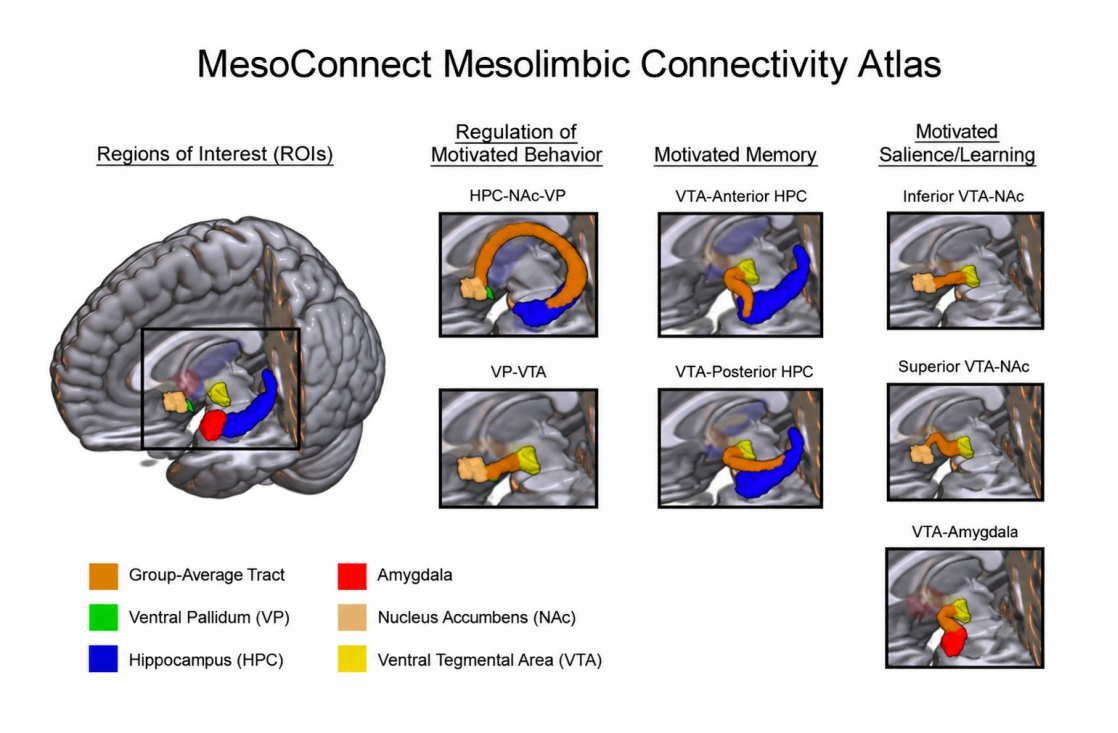
Atlas overview

The MesoConnect Atlas comprises group probabilistic maps of seven bilateral mesolimbic pathways connecting the ventral tegmental area (VTA), hippocampus, nucleus accumbens (NAc), ventral pallidum (VP) and amygdala. The maps were generated from Human Connectome Project (HCP) 7 T diffusion data (Vu et al., 2015) using probabilistic tractography (Tournier et al., 2010) with pathway-specific anatomical constraints, streamline cleaning and group-level probabilistic mapping (see Atlas construction), and are distributed in Montreal Neurological Institute (MNI) space, as maps on the FSL MNI152 1 mm grid (182 × 218 × 182 voxels) and as group-average streamlines in the same space.

The seven pathways are organized into three groupings (Figure S1): regulation of motivated behaviour (hippocampus → NAc → VP; VP → VTA), motivated memory (VTA → anterior hippocampus; VTA → posterior hippocampus), and motivated salience and learning (inferior VTA → NAc; superior VTA → NAc; VTA → amygdala). The groupings organize the atlas for navigation; they do not imply a fixed one-pathway, one-function mapping, and the pathways are overlapping components of a broader circuit.

Figure S1

The Seven Pathways of the MesoConnect Atlas



Note. Orange indicates each group-average pathway at the 50% threshold; region colours are given in the figure. Figure from the MesoConnect repository (CC BY 4.0).

The atlas represents plausible streamline trajectories. It does not establish monosynaptic connectivity, fibre directionality or neurotransmitter identity, and several components of the hippocampal–VTA circuit are polysynaptic.

File types

Each pathway is distributed as two maps and one streamline file per hemisphere (Table S2); the downloads page gives the package layout and file names.

Table S2

Atlas File Types

Suffix	Contents	Typical use
`*_overlap_prop.nii.gz`	Probabilistic map. Each voxel holds the proportion of contributing participants whose binarized tract included it (range 0 to 1).	Visualization, threshold selection, probability-weighted extraction
`*_thr50.nii.gz`	Binary map of voxels present in at least 50% of contributing participants.	Corridor construction, whole-tract extraction
`*.tck`	Group-average streamlines of the pathway, as an MRtrix tractogram of vertex coordinates in MNI millimetres.	Warping into a participant for along-tract sampling

The 50% threshold is the default for corridor construction. A stricter threshold (75%) reduces overreach but may omit legitimate between-participant variation. The 25% threshold retains greater extent and is appropriate when a warped 50% core is short or discontinuous; maps at other thresholds are made from the overlap-proportion map (downloads page).

Modes of use

1. *Corridor-constrained tractography.* The warped and dilated atlas defines the region within which participant-level tractography may travel. Streamlines are estimated from the participant's own data. This mode supports along-tract analysis and is the subject of the workflow section.
2. *Whole-tract extraction.* A scalar map is averaged within the warped atlas mask. This mode is suitable for descriptive summaries; spatial variation along the tract is not retained. See the appendix.
3. *Warped atlas streamlines.* The group-average streamlines distributed with the atlas are carried into a participant by a registration and sampled at their 100 nodes. This mode is suitable when the outcome is microstructure along a pathway and participant-level

tractography is not available. The trajectory is the group's rather than the participant's, and its accuracy depends on the registration. See the appendix.

4. *Atlas-guided synthetic streamlines.* A centerline scaffold constructed inside the warped atlas is sampled at 100 nodes. This mode is suitable when participant-level tractography fails in a subset of the sample and the distributed streamlines are not used. The scaffold is a sampling coordinate system rather than a reconstruction. See the appendix.

Coordinate spaces and interpolation

Atlas space is FSL MNI152 1 mm space unless stated otherwise. Native space refers to a participant's T1, diffusion or NODDI grid (NODDI: neurite orientation dispersion and density imaging; Zhang et al., 2012).

Probabilistic maps are warped with linear interpolation and thresholded in native space. Binary maps are warped with nearest-neighbour interpolation. Spline interpolation is not appropriate for label images. In registration software the moving image is the image being transformed (here the atlas) and the fixed image defines the target grid (the participant); flag names differ between packages. Warped masks should be overlaid on the participant's image and inspected before use.

Interpretive limits

A group atlas warped to native space does not substitute for participant-level tractography when the research question depends on individual anatomy. Dilation increases sensitivity at the cost of specificity, and the smallest dilation that supports reliable tracking should be used. Whole-tract averages are summary descriptors. Warped atlas streamlines carry the group trajectory rather than the participant's, so registration quality bounds what they can show. Synthetic streamlines are sampling scaffolds and should be described as such in reports.

Tract families and regions of interest

The atlas contains seven bilateral pathways (Table S3). Each is processed with the same nine-step workflow; only the seed, target and atlas file change. The exclusion masks used during atlas construction are already reflected in each atlas map. They are listed here so that users can anticipate where a reconstruction may leak when the corridor is dilated too far. Throughout, the ventral tegmental area is abbreviated VTA, the nucleus accumbens NAc, the ventral pallidum VP, and a region of interest ROI.

Table S3

Pathways in the MesoConnect Atlas

Pathway	Grouping	File stem (left / right)	Seed	Target	Notes
VTA → posterior hippocampus	Motivated memory	`left_vta_posterior_hpc`, `right_vta_posterior_hpc`	VTA	posterior hippocampus	The example dataset's primary tract. A ventral secondary bundle occurs in some participants (see Step 6).
VTA → anterior hippocampus	Motivated memory	`left_vta_anterior_hpc`, `right_vta_anterior_hpc`	VTA	anterior hippocampus	Shares its course with the posterior pathway for roughly the first 60 to 80 of 100 nodes.
VTA → amygdala	Motivated salience and learning	`left_vta_amygdala`, `right_vta_amygdala`	VTA	amygdala	Fornix and hippocampus exclusions.
Inferior VTA → NAc	Motivated salience and learning	`left_inferior_vta_nac`, `right_inferior_vta_nac`	VTA	NAc	Inferior and superior divisions are separated by different anterior-commissure exclusions.
Superior VTA → NAc	Motivated salience	`left_superior_vta_nac`, `right_superior_vta_nac`	VTA	NAc	As above.

Pathway	Grouping	File stem (left / right)	Seed	Target	Notes
	and learning				
Hippocampus → NAc → VP	Regulation of motivated behaviour	`left_hpc_nac_vp`, `right_hpc_nac_vp`	hippocampus	VP, with the NAc as a required waypoint	Frequently reconstructs as two bundles; see Step 6.
VP → VTA	Regulation of motivated behaviour	`left_vp_vta`, `right_vp_vta`	VP	VTA	

Note. File stems are those of the atlas package (`<stem>_mni152_1mm_thr50.nii.gz`, `<stem>_mni152_1mm_overlap_prop.nii.gz` and `<stem>_mni152_1mm.tck`). The atlas-generation settings of each pathway are on the construction page; the number of contributing participants is listed there as well.

Regions of interest

Seeds and targets are derived from published atlases, thresholded and binarized in Montreal Neurological Institute (MNI) 1 mm space (Table S4). The atlas package does not redistribute them; the downloads page gives their sources and provides the VTA and hippocampus regions used in the example dataset.

Table S4

Sources and Preparation of the Regions of Interest

Region	Source	Preparation
VTA	7 T probabilistic VTA atlas (Trutti et al., 2021), 25% threshold	Used unmodified; the VTA is subtracted from the red-nucleus, lateral-hypothalamus and mammillary-body exclusion masks.
Hippocampus	Harvard–Oxford subcortical atlas (Frazier et al., 2005; Makris et al., 2006), 50% threshold	Anterior and posterior divisions for the two VTA → hippocampus pathways.
Amygdala	Harvard–Oxford subcortical atlas	Hippocampus subtracted; divided by hemisphere masks.
Nucleus	Harvard–Oxford subcortical	Ventral pallidum subtracted.

Region	Source	Preparation
accumbens	atlas	
Ventral pallidum	Subcortical atlas (Pauli et al., 2018)	Divided by hemisphere masks.
SN/VTA (alternative)	Functional–anatomical SN/VTA masks (Murty et al., 2014)	Comparison resource for the dopaminergic midbrain; not the VTA region used for the atlas.

Note. SN = substantia nigra.

Exclusion masks used during atlas construction

The corridor workflow does not require these masks. They are listed so that users know what the atlas already excludes and can reintroduce any of them as an additional `-exclude` argument if a reconstruction enters a specific structure.

- Thalamus; cortical gray matter with cerebellum (single mask); inferior brainstem.
- Dorsal striatum: caudate and putamen, with accumbens and ventral pallidum subtracted.
- Lateral hypothalamus and mammillary bodies, with the VTA subtracted.
- Red nucleus, with the VTA subtracted.
- Fornix body, with hippocampus and amygdala subtracted (VTA → hippocampus and VTA → amygdala).
- Ipsilateral off-target regions of the circuit, per pathway (for example the ventral pallidum and striatum for VTA → hippocampus, the hippocampus for VTA → NAc).
- Contralateral hemisphere and contralateral target, enforcing ipsilateral tracking.
- Two anterior-commissure exclusion masks, one per division, defining the superior and inferior VTA → NAc pathways.

The ROI preparation script (`prepare_rois.sh`) and the tractography commands with these masks are in the atlas repository's `analysis-code` folder.

Downloads

Complete atlas

The MesoConnect Atlas is distributed as one package, MesoConnect_Atlas.zip (46 MB). The canonical copy and the individual files are in the atlas GitHub repository; the copy served here is identical (SHA-256 checksum file).

The package contains the 28 atlas maps and the 14 pathway tractograms (seven pathways, two hemispheres, two map types and one streamline file each), `mesoconnect_subject_counts.csv`, the FSL MNI152 1 mm brain template (Montreal Neurological Institute, MNI) used as the spatial reference, a README and the licence notice:

```
MesoConnect_Atlas/
  README.md   LICENSE   mesoconnect_subject_counts.csv
  templates/MNI152_T1_1mm_brain.nii.gz
  inferior_vta_nac/   superior_vta_nac/   vta_amygdala/   vta_anterior_hpc/
  vta_posterior_hpc/  hpc_nac_vp/         vp_vta/
```

Each pathway folder holds six files: four maps named `{hemisphere}_{pathway}_mni152_1mm_{map_type}.nii.gz`, for example `vta_posterior_hpc/left_vta_posterior_hpc_mni152_1mm_thr50.nii.gz`, and two streamline files named `{hemisphere}_{pathway}_mni152_1mm.tck`, which carry no map-type field. Table S5 gives the three file types. The maps are on the FSL MNI152 1 mm grid (182 × 218 × 182 voxels, 1 mm isotropic) as compressed NIfTI-1; the streamline files hold vertex coordinates in millimetres in the same space and are not on a voxel grid.

Table S5

File Types in the Atlas Package

File suffix	Contents	Use
`_overlap_prop.nii.gz`	Probabilistic map. Each voxel holds the proportion of contributing participants whose binarized tract included it (0 to 1).	Visualization, custom thresholds, probability-weighted extraction
`_thr50.nii.gz`	Binary map. A voxel is 1 when the tract was present in at least half of the contributing participants.	Corridor construction (this tutorial), whole-tract extraction
`.tck`	Group-average streamlines of the pathway as an MRtrix tractogram. Each streamline carries 100 vertices.	Warping into a participant for along-tract sampling (appendix)

Note. `mesoconnect_subject_counts.csv` gives the number of contributing participants for

every pathway and hemisphere, the denominator of the corresponding overlap-proportion map. Binary maps are warped with nearest-neighbour interpolation. A map at another threshold is made from the overlap-proportion map and kept outside the package folders, for example `fslmaths left_vta_posterior_hpc_mni152_1mm_overlap_prop.nii.gz -thr 0.25 -bin derived/left_vta_posterior_hpc_thr25.nii.gz`.

The package's terms are scoped by material: the Human Connectome Project (HCP)-derived maps and participant counts fall under the WU-Minn HCP Consortium Open Access Data Use Terms, the documentation under CC BY 4.0, and the MNI152 template under the FSL licence; the `LICENSE` file in the package gives the full notice.

Seed and target regions

The seven pathways were built from five regions, all of them published elsewhere. The atlas package does not redistribute them, so each is obtained from its own source (Table S6). The atlas repository's region-of-interest resources page gives the same locations alongside the citations and a checklist for preparing them.

Table S6

Sources of the Seed and Target Regions

Region	Pathways using it	Source	Obtained from
Hippocampus	VTA → anterior hippocampus; VTA → posterior hippocampus; hippocampus → NAc → VP	Harvard–Oxford subcortical atlas (Frazier et al., 2005; Makris et al., 2006)	Distributed with FSL; `atlasq list --extended` locates it after installation
Amygdala	VTA → amygdala	Harvard–Oxford subcortical atlas	As above
Nucleus accumbens (NAc)	Inferior VTA → NAc; superior VTA → NAc; hippocampus → NAc → VP	Harvard–Oxford subcortical atlas	As above
Ventral tegmental area (VTA)	All pathways except hippocampus → NAc → VP	Trutti et al. (2021) 7 T probabilistic VTA	GitHub or OSF; thresholded at 25%
Ventral pallidum (VP)	VP → VTA; hippocampus → NAc → VP	CIT168 subcortical atlas (Pauli et al., 2018)	GitHub or OSF

Note. NAc = nucleus accumbens; VP = ventral pallidum; VTA = ventral tegmental area. Binary masks are warped with nearest-neighbour interpolation, and the MNI grid should be confirmed before transforming a region to participant space.

The two regions used by the VTA → hippocampus example on this site are mirrored here, already thresholded and binarized on the FSL MNI152 1 mm grid, so that the example runs without further preparation (Table S7).

Table S7

Regions Mirrored for the Worked Example

File	Hemisphere	Contents
left_VTA_0.25_bin.nii.gz	Left	VTA seed (Trutti et al., 2021; 25% threshold)
right_VTA_0.25_bin.nii.gz	Right	VTA seed (Trutti et al., 2021; 25% threshold)
HPC_L_0.5_bin.nii.gz	Left	Hippocampus target (Harvard–Oxford; 50% threshold)
HPC_R_0.5_bin.nii.gz	Right	Hippocampus target (Harvard–Oxford; 50% threshold)

Setting up the files for the scripts

The scripts expect the unzipped package as `ATLAS_DIR`, with the seed and target regions in a `roi_maps` folder inside it. The following commands produce that layout for the VTA → hippocampus example.

```
curl -sSLO "https://diffusiontensorimaging-repos.github.io/MesoConnect-Tutorial/downloads/
MesoConnect_Atlas.zip"
unzip -q MesoConnect_Atlas.zip
mkdir -p MesoConnect_Atlas/roi_maps
base="https://diffusiontensorimaging-repos.github.io/MesoConnect-Tutorial/atlas"
for f in left_VTA_0.25_bin right_VTA_0.25_bin HPC_L_0.5_bin HPC_R_0.5_bin; do
  curl -sSL "$base/$f.nii.gz" -o "MesoConnect_Atlas/roi_maps/$f.nii.gz"
done
```

In `00_config.sh`, `ATLAS_DIR` then points at `MesoConnect_Atlas`, and the atlas map of a tract is named by its package path, for example `$ATLAS_DIR/vta_posterior_hpc/left_vta_posterior_hpc_mni152_1mm_thr50.nii.gz`. `ATLAS_STREAMLINES_MNI` names the matching `.tck` file and is read only by the atlas-streamline route.

Source atlases

- Trutti et al. (2021): probabilistic VTA in MNI space; threshold at 25% for a seed region.

- Pauli et al. (2018): subcortical atlas including the ventral pallidum, VTA and substantia nigra.
- Harvard–Oxford subcortical atlas (Frazier et al., 2005; Makris et al., 2006), distributed with FSL: hippocampus, amygdala, accumbens, caudate, putamen, thalamus.
- Murty et al. (2014): functional–anatomical substantia nigra/VTA masks, a comparison resource for the dopaminergic midbrain.
- MNI152 templates (FSL, `$FSLDIR/data/standard`): the 1 mm brain image and brain mask used for registration.

Workflow overview

The workflow comprises nine steps applied to one tract at a time (Figure S2). Step 1 is performed once per participant and reused across tracts. Steps 2 through 9 are repeated for each tract by changing the tract name and the seed, target and atlas files in the configuration.

Figure S2

Sequence of the Corridor Workflow

1. Step 1. Registration of the MNI template to native T1 space (ANTs SyN).
2. Step 2. Warping of the seed, target and atlas into diffusion space.
3. Step 3. Corridor construction (dilate, add seed and target, invert).
4. Step 4. Cutoff selection by pilot sweep and side-by-side comparison.
5. Step 5. Corridor-constrained tractography (tckgen).
6. Step 6. Bundle cleaning (Mahalanobis outlier removal).
7. Step 7. Quality control (tract-density overlays).
8. Step 8. Along-tract profiles (100 nodes; FA and NODDI).
9. Step 9. Group-level inference (whole tract, quartiles, nodes), with node-wise results viewed in the Node-wise Tract Explorer.

Note. MNI = Montreal Neurological Institute; FA = fractional anisotropy; NODDI = neurite orientation dispersion and density imaging. The Node-wise Tract Explorer is the browser-based results viewer described in the Explorer section.

Inputs

Every input below comes from preprocessing; the Diffusion MRI Preprocessing tutorial produces them, and its required outputs page names the step that writes each one. Table S8 lists them per participant. When the T1 image is already aligned to the diffusion image, as in Human Connectome Project data, no affine matrix is needed: `T1_TO_DWI` is set to `header` in the configuration and the warp script resamples by image header. With the default, `matrix`, a participant without a matrix is reported as missing an input.

Table S8

Per-Participant Inputs

Input	Origin	Path used by the scripts
Skull-stripped T1-weighted image	preprocessing	`$PROJECT/anat/<subj>/ <subj>_T1w_brain.nii.gz`
Normalized white-matter fibre	MRtrix3 multi-shell	`$PROJECT/dwi/<subj>/`

Input	Origin	Path used by the scripts
orientation distribution (FOD; `.mif`)	multi-tissue CSD and `mtnormalise`	`wm_fod_norm.mif`
Diffusion-space brain mask	preprocessing	`$PROJECT/dwi/<subj>/nodif_brain_mask.nii.gz`
Preprocessed diffusion data with *b*-values and gradient directions (needed only to estimate FODs or fit NODDI)	preprocessing	`$PROJECT/dwi/<subj>/data.nii.gz`, `bvals`, `bvecs`
Mean *b* = 0 image (optional; background of the quality-control images)	preprocessing	`$PROJECT/dwi/<subj>/mean_b0.nii.gz`
T1 → diffusion affine matrix (when `T1_TO_DWI=matrix`)	FLIRT	`$PROJECT/xfm/<subj>/str2diff.mat`
Scalar maps for profiling	DTIFIT; AMICO NODDI	`$PROJECT/dwi/<subj>/fa.nii.gz`; `$PROJECT/noddi/<subj>/fit_*.nii.gz`

Note. CSD = constrained spherical deconvolution.

Outputs

```
$OUT/<subj>/
  reg/          mni2t1_0GenericAffine.mat, mni2t1_1Warp.nii.gz, mni2t1_1InverseWarp.nii.gz
  rois/         <tract>_{seed,target,atlas}_{t1,diff}.nii.gz
                <tract>_atlas_dilated.nii.gz  <tract>_inclusion_zone.nii.gz
<tract>_exclusion_mask.nii.gz
  tckgen/<tract>/
                <tract>_pilot_<cutoff>.tck          (Step 4)
                <tract>_<cutoff>.tck  <tract>_<cutoff>_cleaned.tck
$OUT/qc/<tract>_cutoff_pilot/  cutoff comparison panels, cutoff_summary.csv         (Step 4b)
$OUT/qc/<tract>_tckgen_summary.csv   streamlines selected and generated            (Step 5)
$OUT/qc/<tract>/               per-participant overlay images, qc_flags.csv        (Step 7)
$OUT/nodewise/                 <tract>_tract_stats.csv     cleaned-bundle Streamline_count,
                                                           Mean_length_mm          (Step 6)
                               <tract>_nodewise_all_subjects.csv
                                   Subject, Tract, Node, FA, NDI, ODI, FWF         (Step 8)
$OUT/analysis/                 <tract>__<metric>__analysis.csv   one row per participant (Step
8b)
                               <tract>__<metric>__tract_models.csv                 (Step 9a)
$OUT/permutation/              <tract>__<metric>__<outcome>_{nodewise,clusters,summary}.csv
                               results_long.csv            Explorer input     (Steps 9b, 9c)
$OUT/logs/                     one log per shell step script
```

A manifest should be kept for each analysis recording the atlas version and threshold, dilation, interpolation, transform files, cutoff, scalar maps, covariates and software versions.

Scripts

All scripts read a single configuration file, `00_config.sh`, which specifies project paths, the participant list, the tract definition and the tractography parameters. The configuration is edited once per project; the tract fields (`TRACT`, `SEED_MNI`, `TARGET_MNI`, `ATLAS_MNI`) are changed for each tract. The shell scripts write a log to `$OUT/logs/`. Steps 0b, 1, 2, 3, 5, 6, 7 and 8a skip participants whose output already exists, so an interrupted run can be restarted with the same command; tractograms and FOD images are written under a temporary name and renamed on completion, so a file left by an interrupted run is never taken for a finished one. Setting `FORCE=1` recomputes existing outputs. Steps 4, 8, 8b and 9 recompute on every run. The Python and R scripts read the settings exported by `source 00_config.sh`; the file must be sourced again after every edit, otherwise those scripts continue to use the previous values. Steps 0b, 1 and 5 require hours on a full sample and should be run under `tmux` or a job scheduler. The concurrency settings in the configuration (`NTHREADS`, `MAXJOBS`, `ANTS_THREADS`, `ANTS_JOBS`, `FOD_JOBS`) were chosen for a 48-core node and should be reduced on a workstation or a shared machine.

```
cd scripts
nano 00_config.sh                 # project paths, tract definition, parameters
source 00_config.sh               # the Python and R scripts read these settings

bash 00b_fod_estimation.sh        # only if wm_fod_norm.mif does not yet exist
bash 01_register_mni_to_t1.sh
bash 02_warp_rois.sh
bash 03_build_corridor_mask.sh
bash 04_tune_cutoff.sh "sub-01 sub-02 sub-03 sub-04 sub-05" "0.1 0.08 0.06 0.01"
python 04b_compare_cutoffs.py "sub-01 sub-02 sub-03 sub-04 sub-05" "0.1 0.08 0.06 0.01"
bash 05_tractography.sh
python 06_clean_bundles.py
python 07_visual_qc.py
python 08a_noddi_fit.py           # omit to profile FA only; Step 8 drops metrics with no maps
python 08_node_profiles.py
python 08b_build_analysis_csv.py  # analysis files for Step 9

python 09a_tract_models.py --metric NDI --outcomes memory_accuracy
mkdir -p "$OUT/permutation"
Rscript 09b_nodewise_permutation.R "$OUT/analysis/${TRACT}__NDI__analysis.csv" \
    memory_accuracy NDI_ "$OUT/permutation" "${TRACT}__NDI__memory_accuracy"
python 09c_stack_for_explorer.py "$OUT/permutation"
```

The configuration file follows.

The script `00_config.sh` is reproduced in the Supplementary Scripts appendix.

Step 1. Registration of the MNI template to native T1 space

The atlas and region files are defined in Montreal Neurological Institute (MNI) space, whereas tractography is performed in each participant's diffusion space. Moving an MNI-space image into a participant therefore requires two transforms: a nonlinear warp from MNI to the participant's T1 image, computed in this step, and a linear transform from T1 to diffusion space, obtained during preprocessing and applied in Step 2.

A nonlinear registration is necessary because affine transforms account for global rotation, scaling and shear but not for individual differences in the size and position of subcortical structures. Positional errors of a few millimetres in the midbrain displace the ventral tegmental area (VTA) region outside the VTA.

Procedure

Registration is performed with `antsRegistrationSyNQuick.sh` (Avants et al., 2008), with the participant's skull-stripped T1 image as the fixed image and the skull-stripped MNI152 1 mm template as the moving image. Image coverage should match: registering a skull-stripped participant image to a whole-head template biases the solution.

```
antsRegistrationSyNQuick.sh -d 3 \
  -f "$PROJECT/anat/$s/${s}_T1w_brain.nii.gz" \
  -m "$FSLDIR/data/standard/MNI152_T1_1mm_brain.nii.gz" \
  -o "$OUT/$s/reg/mni2t1_" -n 4
```

The outputs are the affine component (`mni2t1_0GenericAffine.mat`), the forward warp from MNI to T1 (`mni2t1_1Warp.nii.gz`, used in Step 2), the inverse warp from T1 to MNI (`mni2t1_1InverseWarp.nii.gz`, used to return participant-level results to atlas space) and the template resampled into participant space (`mni2t1_Warped.nii.gz`).

If the T1 image has not been skull-stripped, this should be done first (Diffusion MRI Preprocessing); atlas construction used SynthStrip (Hoopes et al., 2022) and the example dataset used ANTs brain extraction. If the T1 image is already aligned to the diffusion image, this warp is the only transform required (`T1_TO_DWI=header` in the configuration).

The full script follows. Each participant requires approximately 10 to 15 min with four threads, and the script runs four participants concurrently.

The script `01_register_mni_to_t1.sh` is reproduced in the Supplementary Scripts appendix.

Verification

The script concludes with a per-participant table of the three transform files. Registration quality is assessed by overlaying `mni2t1_Warped.nii.gz` on the participant's T1 image; the ventricles,

corpus callosum and brainstem outline should coincide. A registration that is globally acceptable but locally displaced in the midbrain will manifest in Step 2 as a misplaced VTA region, so this inspection supplements rather than replaces the region-level inspection described there.

Step 2. Warping the seed, target and atlas into diffusion space

Three Montreal Neurological Institute (MNI) space images are transformed for each tract: the seed region, the target region and the atlas's 50% binary map. Each passes through the nonlinear warp from Step 1 and then the linear T1 → diffusion transform. Nearest-neighbour interpolation is used at both stages because the images are labels, and the result is re-binarized.

Procedure

```
# MNI -> T1 (ANTs, nonlinear)
antsApplyTransforms -d 3 -i "$ATLAS_MNI" -r "$T1" -o "$d/${TRACT}_atlas_t1.nii.gz" \
  -t "$OUT/$s/reg/mni2t1_1Warp.nii.gz" -t "$OUT/$s/reg/mni2t1_0GenericAffine.mat" -n
NearestNeighbor

# T1 -> diffusion (FLIRT, linear); with T1_TO_DWI=header, -usesqform replaces -init <matrix>
flirt -in "$d/${TRACT}_atlas_t1.nii.gz" -ref "$PROJECT/dwi/$s/nodif_brain_mask.nii.gz" \
  -applyxfm -init "$PROJECT/xfm/$s/str2diff.mat" -out "$d/${TRACT}_atlas_diff.nii.gz" -interp
nearestneighbour
fslmaths "$d/${TRACT}_atlas_diff.nii.gz" -thr 0.5 -bin "$d/${TRACT}_atlas_diff.nii.gz"
```

Two successive nearest-neighbour resamplings of a binary mask have negligible effect on its extent. If the probabilistic map is required in native space, it should be warped with linear interpolation and thresholded afterwards. The full script follows; it completes in seconds per participant.

The script `02_warp_rois.sh` is reproduced in the Supplementary Scripts appendix.

Verification

The script ends with a per-participant table of the voxel counts of the warped seed, target and atlas and of the seed–target overlap, which must be zero; participants with missing outputs are listed as such. Voxel counts far from the sample mean (more than two standard deviations is a useful criterion) indicate a failed registration and call for inspection of that participant. Table S9 gives the counts obtained in the example dataset, as a reference for the sizes to expect at 2 mm resolution.

Table S9

Voxel Counts of Warped Regions in the Example Dataset (N = 57), VTA → Hippocampus

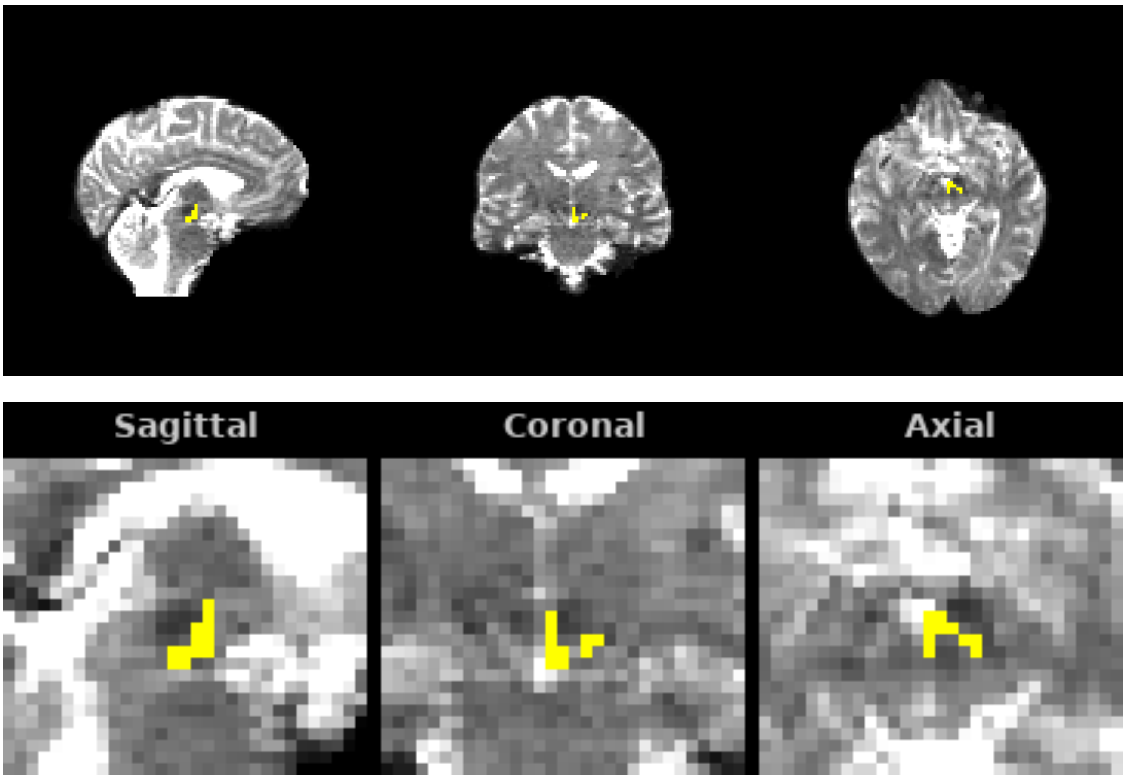
Region	Minimum	*M*	Maximum	*SD*
VTA (L / R)	32 / 34	44 / 44	56 / 56	5.5 / 5.2
Hippocampus (L / R)	308 / 332	395 / 407	497 / 493	33 / 34
Atlas, 50% (L / R)	107 / 99	126 / 123	146 / 149	9.9 / 10.6

Note. VTA = ventral tegmental area; L = left; R = right.

Visual inspection is performed for every participant by overlaying each warped region on the mean $b = 0$ image. Three views are informative: a whole-brain view for gross placement, a magnified view (5×) around the region for voxel-level placement, and an orthogonal render for the record (Figures S3 to S5). The VTA should lie in the ventral midbrain anterior to the red nucleus near the midline and occupy a small number of voxels; placement in the cerebral peduncle, pons or outside the brainstem indicates a failed registration. The hippocampus should follow the medial temporal lobe along the floor of the lateral ventricle; overlap with the ventricle or placement in white matter indicates an error. The atlas should form a narrow corridor from the VTA through the midbrain to the target. A region located in ventricle, white matter or cortex, a region with zero voxels, or a left-hemisphere region on the right returns the participant to Step 1.

Figure S3

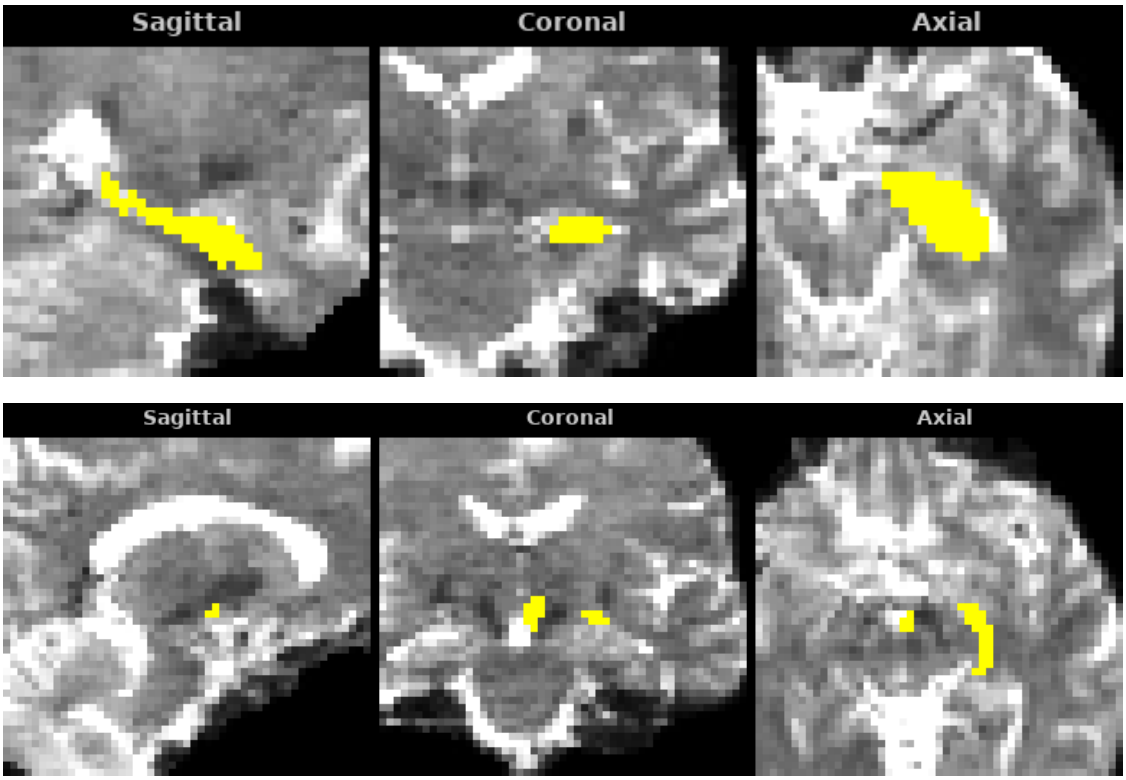
Warped Left VTA Seed Region Over the Mean b = 0 Image



Note. Whole-brain view (top) and magnified view (bottom), example dataset.

Figure S4

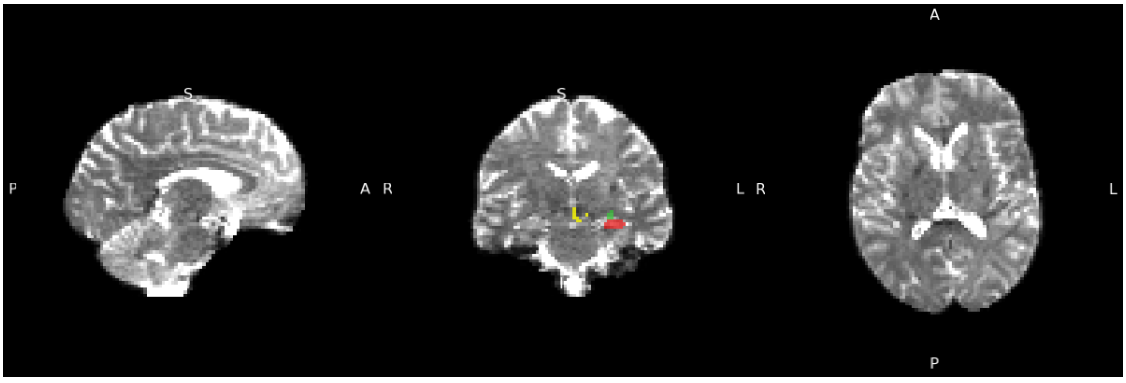
Warped Left Hippocampus Target and Left VTA → Hippocampus Atlas



Note. Hippocampus (top) and atlas at the 50% threshold (bottom), magnified views.

Figure S5

Orthogonal Render of the Warped Regions

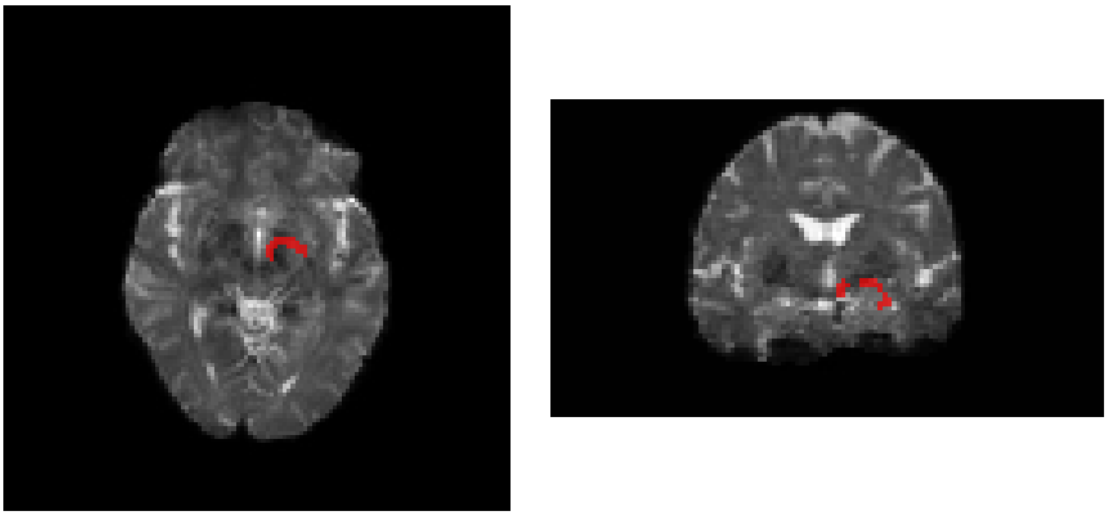


Note. VTA in yellow, hippocampus in red, atlas in green.

For the anterior hippocampus tract the same procedure is repeated with the anterior atlas file; the seed and target regions are shared (Figure S6).

Figure S6

Warped Anterior VTA → Hippocampus Atlas in Diffusion Space



Note. Atlas in red over the mean $b = 0$ image. Left: axial view. Right: coronal view.

Step 3. Corridor construction

Atlas construction used approximately a dozen individually prepared exclusion regions per tract. The corridor workflow replaces them with a single mask derived from the warped atlas. The atlas is dilated, the seed and target regions are added, the union is binarized to form an inclusion zone, and the inclusion zone is inverted. The inverted image serves as the sole exclusion mask for tractography: any streamline leaving the corridor is discarded.

Procedure

```
fslmaths "$d/${TRACT}_atlas_diff.nii.gz" -dilM -dilM "$d/${TRACT}_atlas_dilated.nii.gz"
fslmaths "$d/${TRACT}_atlas_dilated.nii.gz" -add "$d/${TRACT}_seed_diff.nii.gz" \
         -add "$d/${TRACT}_target_diff.nii.gz" -bin "$d/${TRACT}_inclusion_zone.nii.gz"
fslmaths "$d/${TRACT}_inclusion_zone.nii.gz" -binv "$d/${TRACT}_exclusion_mask.nii.gz"
```

The seed and target must be added before inversion; otherwise the corridor terminates short of them and tracking stops at the boundary. The full script follows; the `DILATE_VOX` setting in the configuration controls the number of `-dilM` passes.

The script `03_build_corridor_mask.sh` is reproduced in the Supplementary Scripts appendix.

Dilation

A dilation of two voxels was used in the example dataset at 2 mm isotropic resolution. One voxel is appropriate when registration is accurate and pilot reconstructions appear loose; four voxels serves as a sensitivity setting when registration is uncertain. The 50% atlas is already somewhat wider than the median participant's core, which is why two voxels is generally sufficient. The dilation used should be reported.

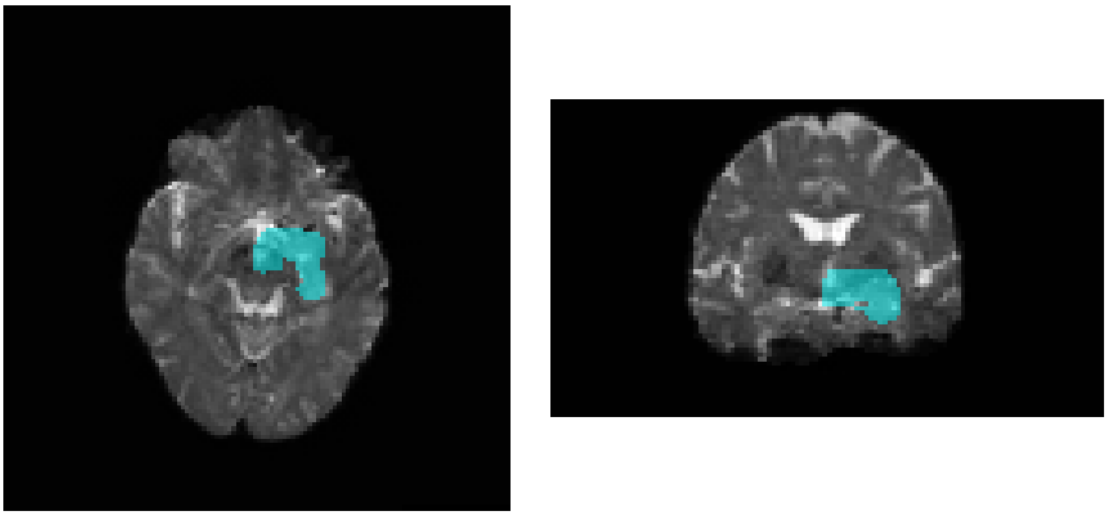
Verification

The script confirms that the seed and target voxels have value 0 in the exclusion mask (that is, are not excluded) and reports the corridor size, which should be similar across participants (roughly 1,500 to 1,900 voxels at 2 mm for VTA → hippocampus in the example dataset). A corridor much larger or smaller than the others indicates a registration problem.

Visual inspection confirms that the corridor follows a plausible path from seed to target and contains both regions (Figure S7). Contact with the ventricle or extension into cortex indicates excessive dilation or a registration error.

Figure S7

Inclusion Zone for the Anterior VTA → Hippocampus Tract



Note. Inclusion zone in cyan over the mean $b = 0$ image, with the seed and target contained within it. Left: axial view. Right: coronal view. VTA = ventral tegmental area.

Step 4. Selection of the FOD amplitude cutoff

The fibre orientation distribution (FOD) amplitude cutoff determines the minimum FOD amplitude at which tracking continues. A high cutoff terminates streamlines before they reach the target; a low cutoff, in unconstrained tracking, produces spurious streamlines. The corridor alters this trade-off. With tracking confined to the corridor, a permissive cutoff produces well-formed bundles while consuming far fewer seeds than a conservative cutoff. This behaviour should be verified for each dataset and tract family by a pilot sweep on a small number of participants, followed by tabulation and side-by-side inspection of the reconstructions.

Cutoff in routine use

The value in routine use with the corridor mask is 0.01, and it is the default in `00_config.sh`. It is the recommended starting value for any tract reconstructed with this workflow; the pilot sweep described below confirms it for a new dataset or tract family and takes under an hour. Table S10 lists the cutoffs that have been used with the atlas.

Table S10

FOD Amplitude Cutoffs in Use

Setting	Cutoff	Outcome
Corridor workflow, 3 T multi-shell data (example dataset): posterior and anterior ventral tegmental area (VTA) → hippocampus	0.01	Value used for the example dataset
Corridor workflow, VTA → accumbens (other users of the atlas)	0.01	Reported to reconstruct the tract
Atlas construction, 7 T: VTA → hippocampus (both pathways)	0.06	Value used to build the atlas
Atlas construction, 7 T: VTA → amygdala	0.08	Value used to build the atlas
Atlas construction, 7 T: hippocampus → accumbens → ventral pallidum	0.04	Value used to build the atlas
Atlas construction, 7 T: VTA → accumbens (both divisions) and ventral pallidum → VTA	0.03	Value used to build the atlas
Corridor workflow, the atlas repository's worked example (inferior VTA → accumbens, 2 mm-dilated 50% corridor, 1,000 streamlines)	0.05	Value used in the atlas repository's example

Setting	Cutoff	Outcome
MRtrix3 default for FOD-based tracking	0.05	For reference

Note. A cutoff of 0.01 is usable only with the corridor's exclusion mask. Without the mask it produces streamlines throughout the brain.

Because lower field strength yields noisier FOD estimates, a cutoff higher than the 7 T values (approximately 0.08) was anticipated for 3 T data. The pilot sweep did not support that expectation: the conservative cutoffs failed to reach the target in several participants, and 0.01 reached it in all of them with about one fifth of the seeds.

Pilot sweep

The sweep uses five participants, four cutoffs and reduced budgets (`-select 1000`, `-seeds 5000000`); all other parameters match the production run.

```
bash 04_tune_cutoff.sh "sub-01 sub-02 sub-03 sub-04 sub-05" "0.1 0.08 0.06 0.01"
```

The script reports streamlines selected, streamlines generated and mean length for each participant and cutoff. `tckgen` generates one streamline per seed, so the number generated is also the number of seeds consumed. Table S11 gives the streamline counts obtained in the example dataset. At 0.1 most runs exhausted the seed budget before reaching the target. At 0.08 some participants reached the target and one stopped at 309. At 0.06 and at 0.01 all runs reached the target; the 0.01 runs consumed approximately one fifth of the seeds (about 415,000 versus 2.1 million or more at 0.06). Reaching the target in every pilot participant is a necessary condition for a cutoff; the reconstructions must also correspond to the tract, which is assessed next.

Table S11

Streamlines Reached by Cutoff in the Pilot Sweep, Posterior VTA → Hippocampus

Participant	Hemisphere	0.1	0.08	0.06	0.01
P1	L	118	556	1000	1000
P1	R	30	434	1000	1000
P2	L	274	1000	1000	1000
P2	R	165	906	1000	1000
P3	L	1000	1000	1000	1000
P3	R	1000	1000	1000	1000
P4	L	88	309	1000	1000
P4	R	731	1000	1000	1000

Note. Target 1,000 streamlines; seed limit 5 million. P1 to P4 = pilot participants; L = left; R = right.

Side-by-side comparison

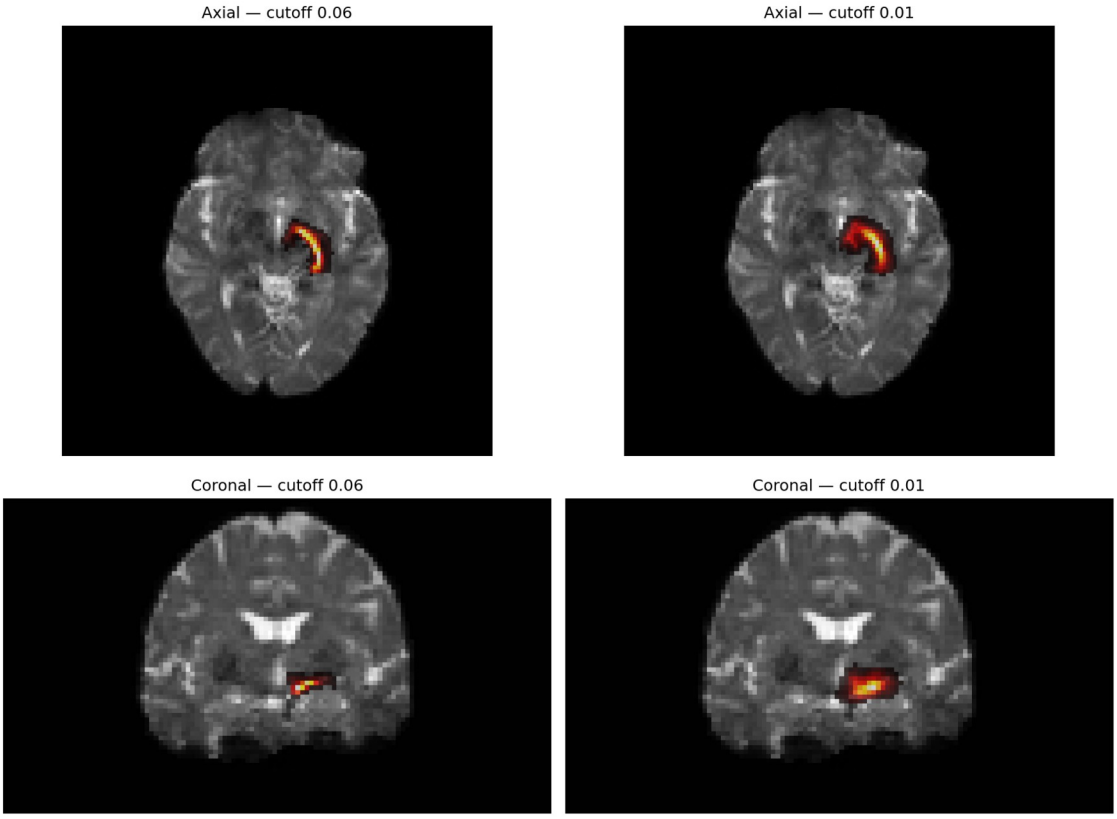
For each participant, the tract-density image (TDI) at each cutoff is rendered on the same axial and coronal slices, one column per cutoff. Dice overlap of each cutoff against the most conservative one, the number of TDI voxels, and the mean and standard deviation of streamline length are computed and summarized.

```
python 04b_compare_cutoffs.py "sub-01 sub-02 sub-03 sub-04 sub-05" "0.1 0.08 0.06 0.01"
```

The script writes one panel per participant, `cutoff_summary.csv`, and a summary chart (Figures S8 and S9). In the example dataset the two cutoffs traced the same trajectory in both views. The 0.01 reconstruction was somewhat thicker (mean Dice = .66 against 0.06), mean lengths were similar (approximately 44 versus 47 mm), and the length standard deviation was slightly greater at 0.01 (7 versus 6 mm). Bundle cleaning (Step 6) removes the outlying streamlines responsible for the additional thickness; the cleaned 0.01 bundle is more compact than the uncleaned 0.06 bundle, as shown on that page.

Figure S8

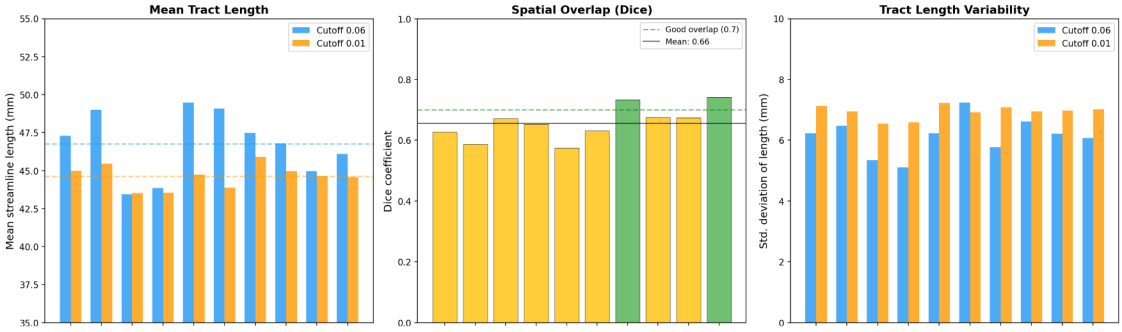
Tract-Density Images at Two Cutoffs, Uncleaned



Note. Cutoff 0.06 (left) and 0.01 (right) for one participant, left posterior VTA → hippocampus.

Figure S9

Summary Statistics by Cutoff Across the Pilot Participants



Note. Bars are grouped by pilot participant and hemisphere. Left: mean streamline length. Middle: Dice overlap between cutoffs. Right: standard deviation of streamline length.

Selection criterion

The cutoff selected is the most permissive value that reaches the streamline target in every pilot participant and whose reconstruction matches the conservative reconstruction. In the example dataset this was 0.01, the value in routine use (Table S10). A higher value is warranted only when the 0.01 reconstruction departs from the conservative reconstruction in the side-by-side images, for example by filling the corridor rather than following the tract; the next value in Table S10 (0.06) is then tried. The sweep should be repeated for each tract family, since a cutoff confirmed for VTA → hippocampus is a starting value for other pathways.

Options not used

Anatomically constrained tractography (ACT) was evaluated during atlas construction and abandoned because the tissue segmentation's white-matter mask was too restrictive relative to the gray-matter mask and tracts did not reconstruct. The `-backtrack` and `-crop_at_gmwmi` options and custom `-angle` and `-step_size` values were left at MRtrix3 defaults. These options should be introduced only in response to a specific, observed failure.

Scripts

The script `04_tune_cutoff.sh` is reproduced in the Supplementary Scripts appendix.

The script `04b_compare_cutoffs.py` is reproduced in the Supplementary Scripts appendix.

Step 5. Corridor-constrained tractography

Tractography is performed with `tckgen` using second-order integration over fibre orientation distributions (iFOD2; Tournier et al., 2010). The FOD image is the last output of preprocessing (Diffusion MRI Preprocessing, Step 12); `00b_fod_estimation.sh` here is a fallback for data that stopped at the tensor. Seeding is unidirectional from the seed region, the target region is required, and the inverted corridor from Step 3 is supplied as the exclusion mask.

Procedure

```
tckgen "$PROJECT/dwi/$s/wm_fod_norm.mif" "$tck" \
  -algorithm iFOD2 \
  -seed_image "$rois/${TRACT}_seed_diff.nii.gz" -seed_unidirectional \
  -include "$rois/${TRACT}_target_diff.nii.gz" \
  -exclude "$rois/${TRACT}_exclusion_mask.nii.gz" \
  -select 2500 -seeds 25000000 -cutoff 0.01 \
  -minlength 35 -maxlength 65 -stop \
  -nthreads 8 -force
```

Table S12 gives the rationale for each option, and Table S13 the values used per tract family. The script records the number of streamlines selected and the number generated (equal to the number of seeds consumed) per participant in `$OUT/qc/<tract>_tckgen_summary.csv`. The streamline count and mean length that enter the group-level models as covariates are taken from the cleaned bundle in Step 6, because every uncleaned tractogram that reaches the target contains the same number of streamlines. The full script follows. At 3 T with eight threads each participant and hemisphere requires approximately 2 min; 57 participants with both hemispheres required about 4 hr run serially.

Table S12

Tractography Parameters

Option	Value	Rationale
`-cutoff`	from Step 4 (0.01 in the example dataset)	The corridor permits a low cutoff.
`-select`	2500	Sufficient for stable profiles and endpoint maps. Streamline count is not a measure of connectivity; participants who stop short are retained.
`-seeds`	25,000,000	Ceiling on seeding attempts. The default (1,000 × `select`) is sometimes insufficient to reach 2,500.

Option	Value	Rationale
`-minlength`, `-maxlength`	35, 65 mm	Set for VTA → hippocampus at 7 T. The example dataset retained them at 3 T, with mean lengths of 44 to 47 mm. Set per tract family from pilot lengths.
`-seed_unidirectional`	on	Tracking proceeds away from the seed only.
`-stop`	on	Tracking terminates on entering the target.
gradient table	embedded in the `.mif`	`-fslgrad` is omitted when the FOD image was built from a `.mif` containing gradients.

Note. VTA = ventral tegmental area; FOD = fibre orientation distribution.

Table S13

Parameters by Tract Family

Family	Cutoff with the corridor	Cutoff at 7 T (atlas construction)	Length bounds	Notes
VTA → hippocampus, posterior and anterior	0.01	0.06	35–65 mm	Used for the example dataset at 3 T.
VTA → amygdala	0.01, confirm by pilot	0.08	27–40 mm at 7 T; confirm by pilot	Shorter tract.
VTA → nucleus accumbens, inferior and superior	0.01	0.03	8–35 mm at 7 T; confirm by pilot	0.01 has been used with the corridor by other users of the atlas; the atlas repository's worked example uses 0.05. The inferior pathway was generated with a 7° angle and 0.25 mm step.
Ventral pallidum → VTA	0.01, confirm by pilot	0.03	15–25 mm at 7 T; confirm by pilot	Generated with a 15° angle.

Family	Cutoff with the corridor	Cutoff at 7 T (atlas construction)	Length bounds	Notes
Hippocampus → nucleus accumbens → ventral pallidum	0.01, confirm by pilot	0.04	minimum 5 mm at 7 T; confirm by pilot	The accumbens is a required waypoint. Frequently reconstructs as two clusters; see Step 6.

Note. The 7 T values are the atlas-generation settings (Atlas construction, Table S12); the corridor cutoff is the value in routine use with the corridor mask (Step 4).

Note. Where a pilot is indicated, the length bounds are taken from the mean lengths reported in the Step 4 summary.

The script `05_tractography.sh` *is reproduced in the Supplementary Scripts appendix.*

Verification

The script reports the number of participants reaching the streamline target and, for each participant, the number of streamlines generated. Reaching the target in every participant with a small fraction of the seed ceiling is the expected outcome. A participant reaching a lower count, for example 1,800, is retained; a participant reaching a very low count, for example 50, is reviewed in Step 7.

Step 6. Bundle cleaning

Some streamlines within the corridor follow atypical trajectories. Cleaning is performed with `clean_bundle` from pyAFQ (Kruper et al., 2021), which implements the procedure of Yeatman et al. (2012): each streamline is resampled to 100 points, its Mahalanobis distance from the bundle's mean trajectory is computed, and streamlines exceeding a distance threshold or a length threshold are removed iteratively (Table S14).

Procedure

Table S14

Cleaning Parameters

Parameter	Value	Meaning
`n_points`	100	Resampling prior to comparison.
`clean_rounds`	5	Maximum iterations; stops earlier when no streamlines are removed.
`distance_threshold`	3	Streamlines more than 3 *SD* (Mahalanobis) from the bundle centroid are removed.
`length_threshold`	2	Streamlines longer than the mean length by more than 2 *SD* are removed.
`stat`	`mean`	Centroid statistic.
`core_only`	0	Distance is computed over the full length of each streamline. pyAFQ 3 and later default to the middle 60%; the script sets 0 so that results do not depend on the installed version.

```
from AFQ.recognition.cleaning import clean_bundle
cleaned, keep = clean_bundle(sft, n_points=100, clean_rounds=5,
                             distance_threshold=3, length_threshold=2,
                             stat="mean", core_only=0, return_idx=True)
```

The `core_only` argument exists in pyAFQ 3 and later; the script omits it under earlier versions, whose behaviour it reproduces. In pyAFQ 1.3.2 and earlier the function is imported from `AFQ.segmentation`; the script tries both locations.

The full script follows. It requires pyAFQ and DIPY; on older systems `pip` may require the `zipp` package to be upgraded first.

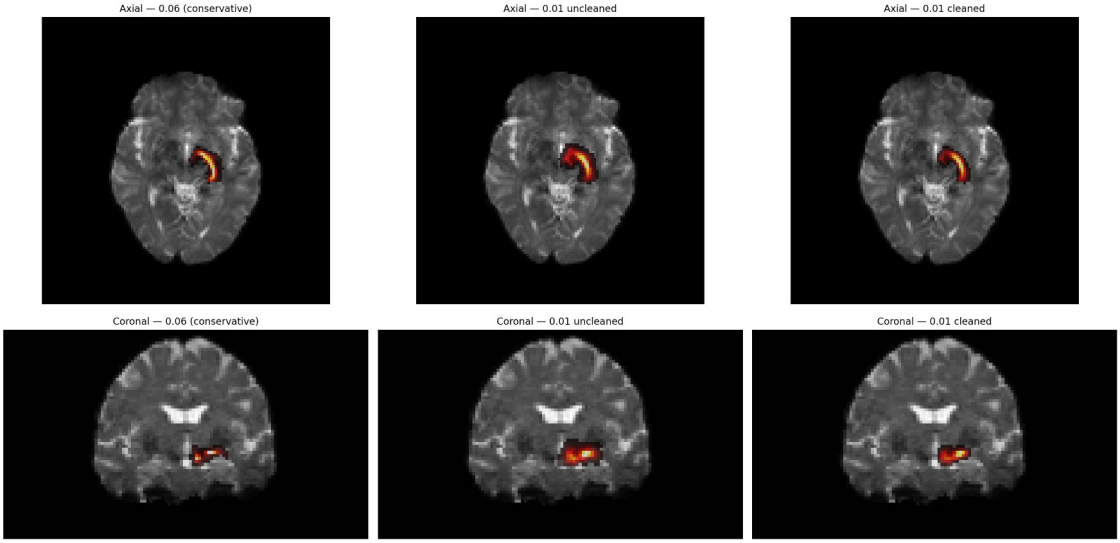
The script `06_clean_bundles.py` is reproduced in the Supplementary Scripts appendix.

Retention

With the thresholds in Table S14, cleaning typically retains 25% to 60% of streamlines, several hundred per participant. Figures S10 and S11 present the comparison referred to in Step 4: the cleaned 0.01 bundle is more compact than the uncleaned 0.06 bundle, with length standard deviations of approximately 3.5 to 4.5 mm for cleaned bundles against 5 to 7 mm for either uncleaned condition.

Figure S10

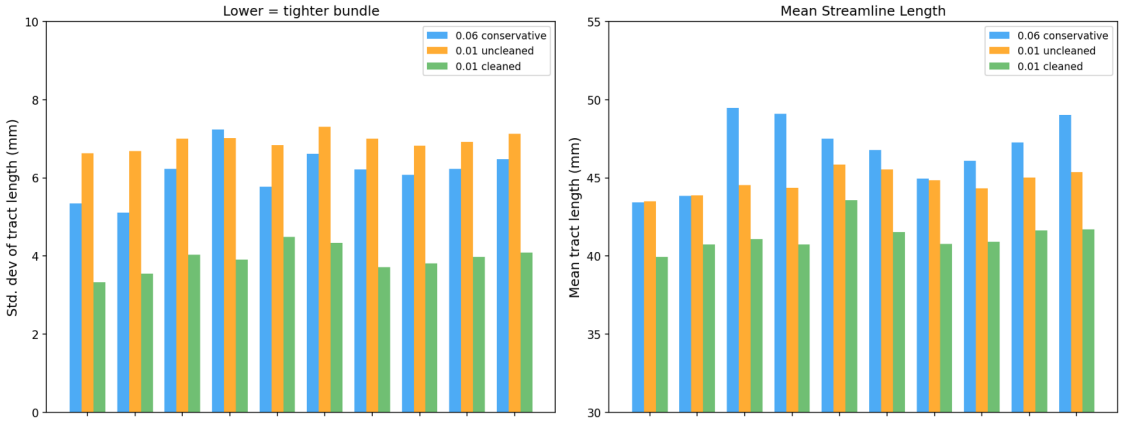
Effect of Cutoff and Cleaning on One Bundle



Note. Left: cutoff 0.06, uncleaned (1,000 streamlines). Middle: cutoff 0.01, uncleaned (2,500). Right: cutoff 0.01, cleaned (824). One participant, left posterior VTA → hippocampus. VTA = ventral tegmental area.

Figure S11

Standard Deviation of Streamline Length by Condition Across the Five Pilot Participants



Note. Bars are grouped by pilot participant and hemisphere. Cleaned 0.01 in green.

Tracts with two bundles

Some tracts reconstruct as two distinct bundles rather than a single bundle with outliers. VTA → hippocampus has a ventral secondary bundle in a subset of participants, and hippocampus → nucleus accumbens → ventral pallidum frequently produces two bundles. Mahalanobis cleaning applied to such a mixture retains the wrong bundle or a blend. The procedure used during atlas construction clusters the streamlines with QuickBundles (Garyfallidis et al., 2012; threshold 5 mm, streamlines resampled to 100 points), cleans the two largest clusters separately, and retains the anatomically correct one after inspection. Setting `QB_SPLIT = True` in the script produces `_cluster1_cleaned.tck` and `_cluster2_cleaned.tck` per participant. The correct cluster is copied to `<tract>_<cutoff>_cleaned.tck` and the script is run again to rebuild the covariate table; the selection should be recorded per participant in the manifest. In the example dataset the corridor and cleaning were sufficient for VTA → hippocampus and the division was not required.

Verification

The cleaned file must exist and contain more than zero streamlines for every participant. Counts before and after cleaning and the retention percentage are reported; retention below 15% or above 80% warrants inspection. The script also writes `$OUT/nodewise/<tract>_tract_stats.csv`, which holds the streamline count and mean length of each cleaned bundle (`Streamline_count`, `Mean_length_mm`). Both index reconstruction quality, vary across participants, and are entered as covariates in Step 9.

Step 7. Quality control of cleaned bundles

Every participant and hemisphere is inspected after cleaning. The script converts each cleaned bundle to a tract-density image (TDI) with `tckmap`, selects the axial and coronal slices of greatest density, overlays the image on the mean $b = 0$ image, and writes one image per participant together with automatic flags (Table S15). Flags appear in the image title and are collected in `qc_flags.csv`.

Table S15

Automatic Flags

Flag	Criterion
`LOW_COVERAGE`	fewer than 50 TDI voxels
`HIGH_COVERAGE`	more than 5,000 TDI voxels
`LOW_DENSITY`	maximum density below 5 streamlines per voxel

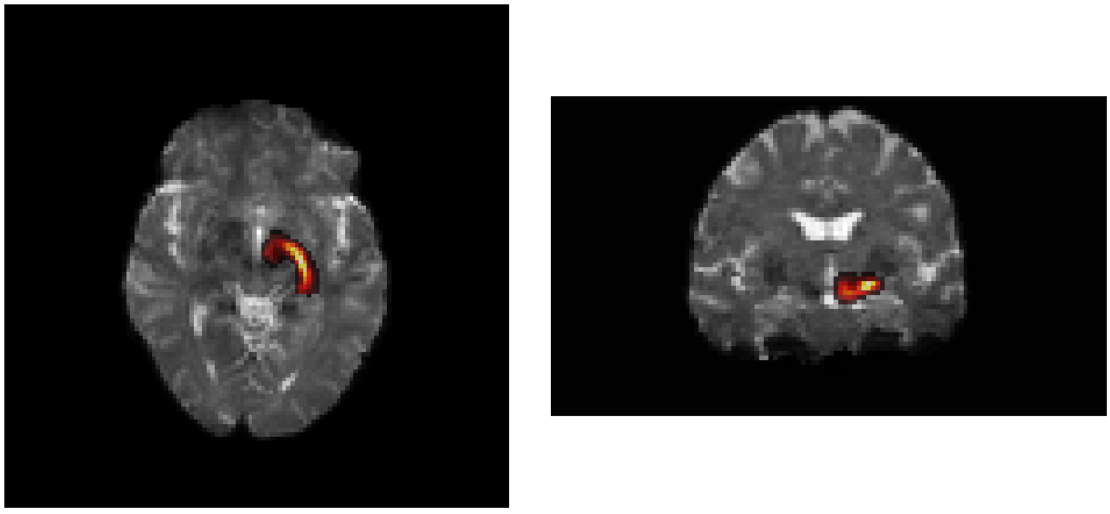
The script `07_visual_qc.py` is reproduced in the Supplementary Scripts appendix.

Criteria

A correct reconstruction shows the expected shape for the tract (for ventral tegmental area [VTA] → hippocampus, an arc from the ventral midbrain laterally into the medial temporal lobe), consistent shape across participants with variation in size, and no isolated voxel clusters, no density in the contralateral hemisphere or frontal lobe, and no empty tracts (Figures S12 to S14).

Figure S12

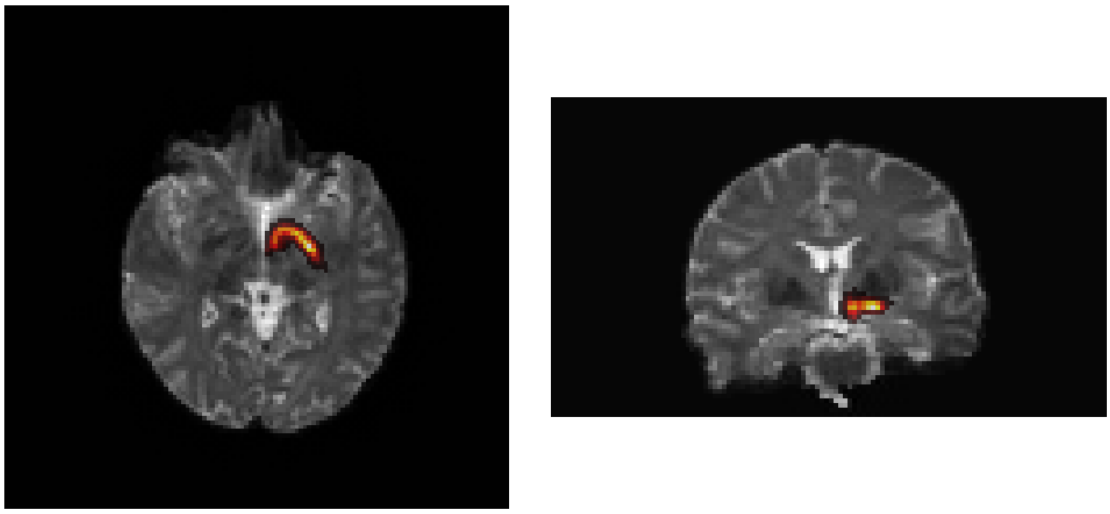
Cleaned Left Posterior VTA → Hippocampus Tract, One Participant



Note. Streamline density over the mean $b = 0$ image. Left: axial view. Right: coronal view. The same layout applies to Figures S13 and S14.

Figure S13

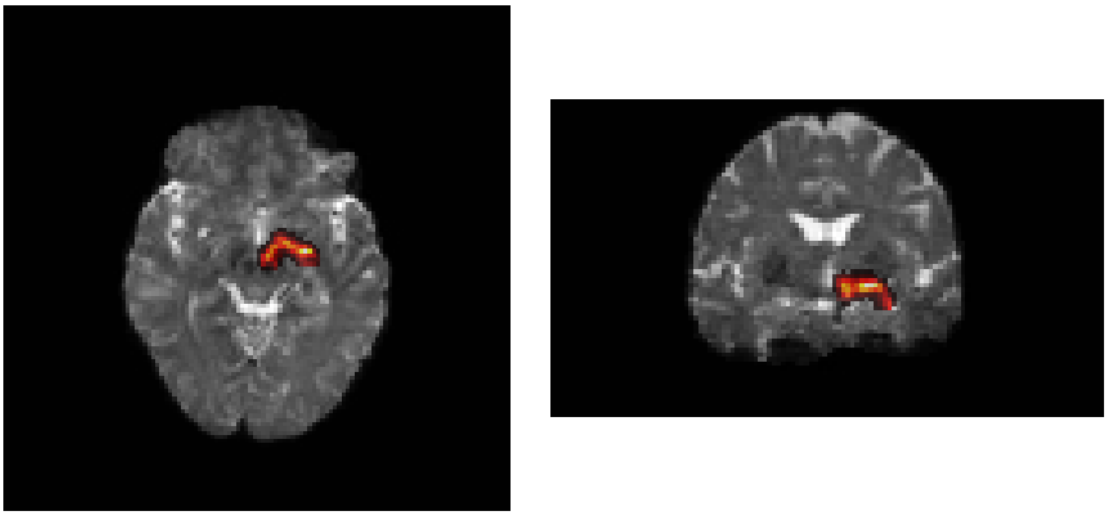
Cleaned Tract at the Low End of Retention



Note. About a quarter of the streamlines survived cleaning; the bundle is still well formed.

Figure S14

Cleaned Anterior VTA → Hippocampus Tract



Note. Same participant as Figure S12.

Interactive review

The overlay images support a batch review. Flagged participants, and a sample of unflagged participants, should additionally be examined in a viewer with slice navigation, magnification and opacity control.

```
fsleyes "$PROJECT/dwi/$s/mean_b0.nii.gz" \
  "$OUT/$s/tckgen/$TRACT/${TRACT}_${CUTOFF}_cleaned.tck" -cm hot -a 70 &

mrview "$PROJECT/dwi/$s/mean_b0.nii.gz" \
  -tractography.load "$OUT/$s/tckgen/$TRACT/${TRACT}_${CUTOFF}_cleaned.tck" &
```

Comparison with the atlas

For a quantitative check, each participant's cleaned TDI can be warped to Montreal Neurological Institute (MNI) space with the inverse warp from Step 1 and compared with the atlas by Dice overlap against the 50% map, or by the fraction of the participant tract lying within the 25% probability map. This overlap should not be reported as validation of the atlas, because the reconstruction was constrained by it.

```
tckmap "$tck" "$tdi" -template "$PROJECT/dwi/$s/nodif_brain_mask.nii.gz" -force
fslmaths "$tdi" -thr 1 -bin "${tdi%.nii.gz}_bin.nii.gz"
```

Step 8. Along-tract profiles

Each cleaned bundle is converted to a 100-point profile of a scalar map: fractional anisotropy (FA) from the tensor fit; the neurite density index (NDI), orientation dispersion index (ODI) and free-water fraction (FWF) from neurite orientation dispersion and density imaging (NODDI; Zhang et al., 2012); or any other map on the diffusion grid. Profiling follows the tract-profile approach of Yeatman et al. (2012) as implemented in DIPY (Garyfallidis et al., 2014).

Two operations make profiles comparable across participants. First, because streamlines in a tractogram file may run in either direction, each bundle is oriented against its QuickBundles centroid (Garyfallidis et al., 2012; `threshold=np.inf`, yielding one cluster), and the centroid is itself oriented so that it begins at the end nearer the warped seed region. Node 0 therefore corresponds to the seed end and node 99 to the target end in every participant. Second, `afq_profile` with `gaussian_weights` weights each streamline's contribution at a node by its Mahalanobis distance from the bundle core, reducing the influence of outlying streamlines.

Procedure

```
metric = AveragePointwiseEuclideanMetric(ResampleFeature(nb_points=100))
centroid = QuickBundles(threshold=np.inf, metric=metric).cluster(streamlines).centroids[0]
if distance(centroid[0], seed_centre) > distance(centroid[-1], seed_centre):
    centroid = centroid[::-1]                       # node 0 at the seed end
oriented = orient_by_streamline(streamlines, centroid)
weights = gaussian_weights(oriented, n_points=100)
profile = afq_profile(data, oriented, affine, n_points=100, weights=weights)
```

The `METRICS` dictionary in the script specifies the scalar maps; the script writes one long-format CSV per tract with one column per metric. Processing takes roughly 10 s per participant and tract.

The script `08_node_profiles.py` is reproduced in the Supplementary Scripts appendix.

Node range

Nodes near the ends of the profile (0 to 4 and 95 to 99) lie in or adjacent to gray matter and are affected by partial-volume contamination from the seed and target regions. Deep white matter spans approximately nodes 25 to 75. Whether end nodes are trimmed should be decided before modelling.

NODDI inputs

NODDI is fitted with AMICO (Daducci et al., 2015) on the eddy-corrected data; the script `08a_noddi_fit.py`, shown below, performs the fit. The modulated NDI and ODI maps (`fit_NDI_modulated.nii.gz`, `fit_ODI_modulated.nii.gz`), which incorporate the tissue-

weighted partial-volume correction of Parker et al. (2021), are used for profiling; FWF has no modulated form and `fit_FWF.nii.gz` is used. Settings used in the example dataset were `bStep=200` during scheme conversion (rounding jittered *b*-values), `b0_thr=100`, `doSaveModulatedMaps=True`, `doComputeRMSE=True` and `BLAS_nthreads=1`. By default AMICO writes its response kernels to a directory shared by all participants, and concurrent jobs then delete one another's files; the script instead generates the kernels in a working directory for each participant, so several instances can be run at once on different participants (`python 08a_noddi_fit.py sub-01 sub-02`). AMICO also empties its output directory when saving, so nothing else should be stored there.

When NODDI is sampled within a gray-matter region (for example the hippocampus), the model should be refitted with the gray-matter intrinsic parallel diffusivity (approximately 1.1×10^{-3} mm²/s rather than the white-matter default of 1.7×10^{-3} mm²/s) by setting `NODDI_DPAR=1.1e-3`; the script then writes to `$PROJECT/noddi_gm` so that the white-matter maps are retained. The white-matter fit is not appropriate in gray matter and the two fits can yield different results.

The script `08a_noddi_fit.py` is reproduced in the Supplementary Scripts appendix.

Analysis file

Inference in Step 9 reads a wide file with one row per participant: the covariates and outcomes, the streamline count and mean length of the cleaned bundle from Step 6, and the 100 node values as columns `<METRIC>_0` to `<METRIC>_99`. The script `08b_build_analysis_csv.py`, shown below, builds one such file per tract and metric from the long profile CSV, the tract statistics file and a participant-level covariates file (`COVARIATES_CSV` in the configuration; columns `Subject` plus covariates and outcomes).

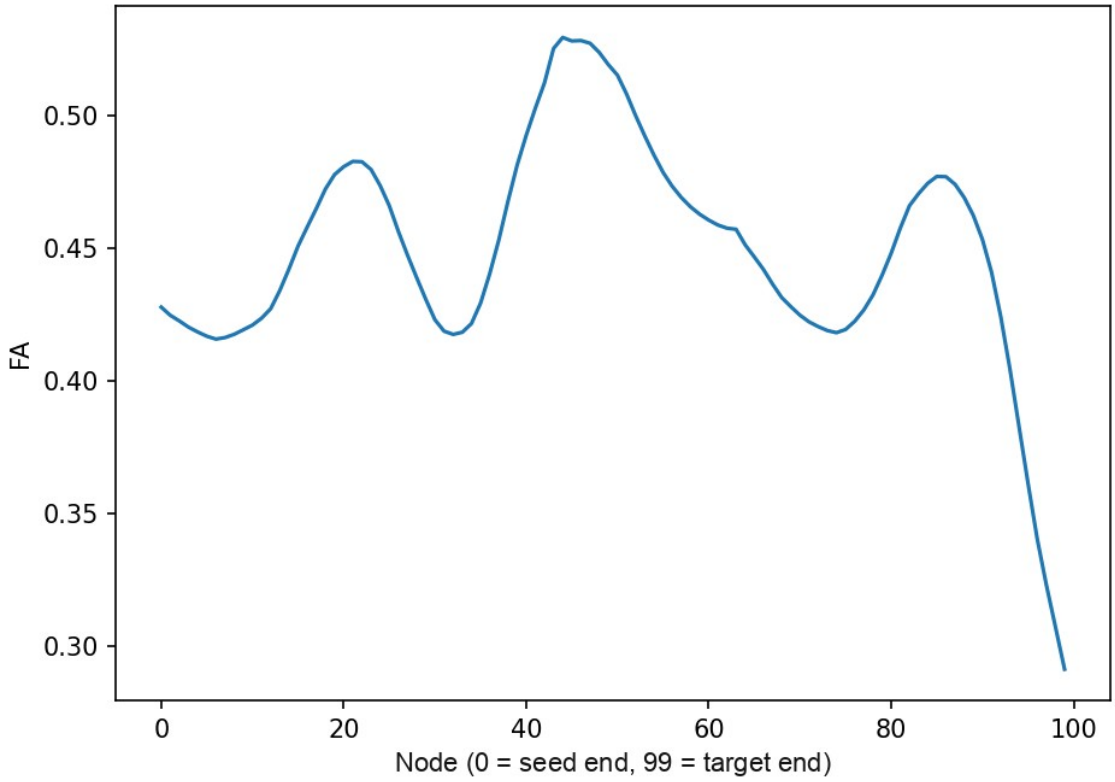
The script `08b_build_analysis_csv.py` is reproduced in the Supplementary Scripts appendix.

Example profiles

Figures S15 to S17 show profiles from one participant in the example dataset.

Figure S15

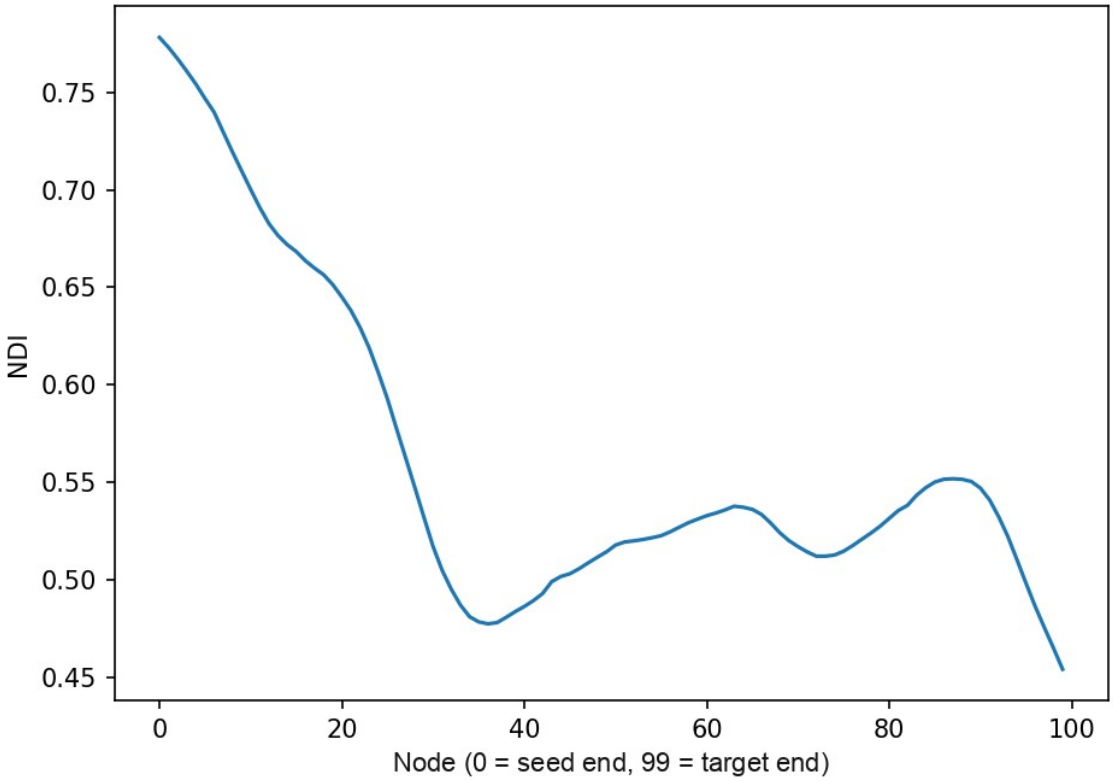
Fractional Anisotropy Along the Left Posterior VTA → Hippocampus Tract

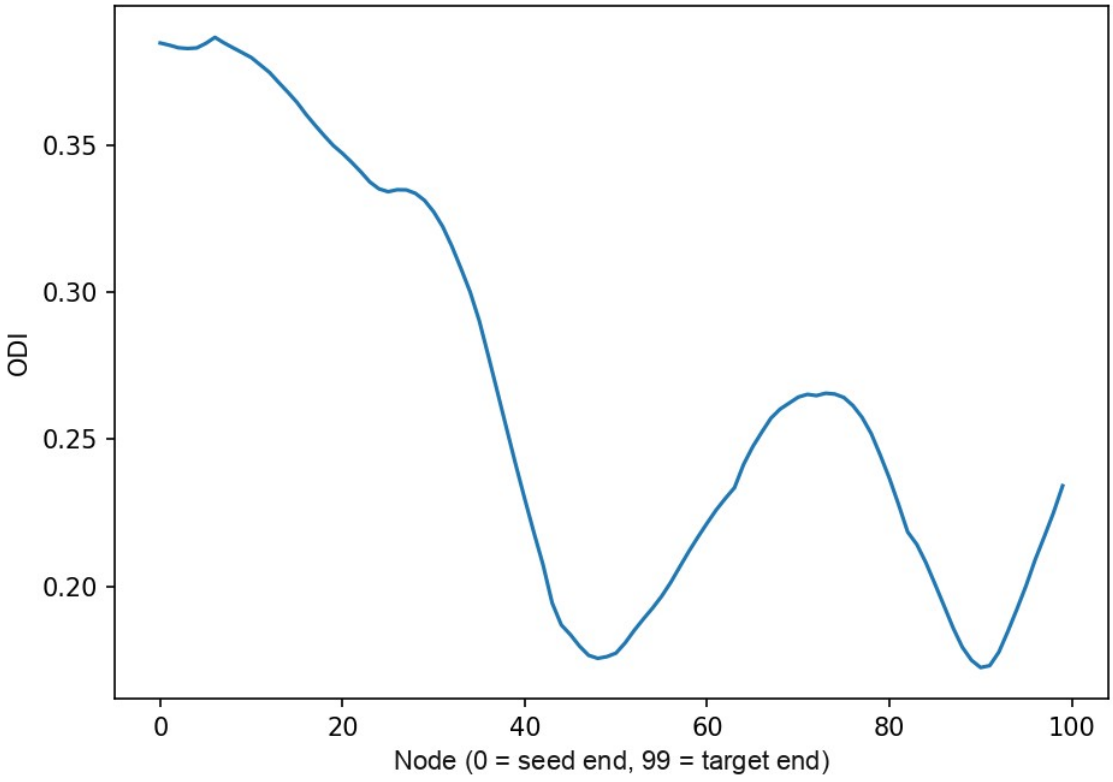


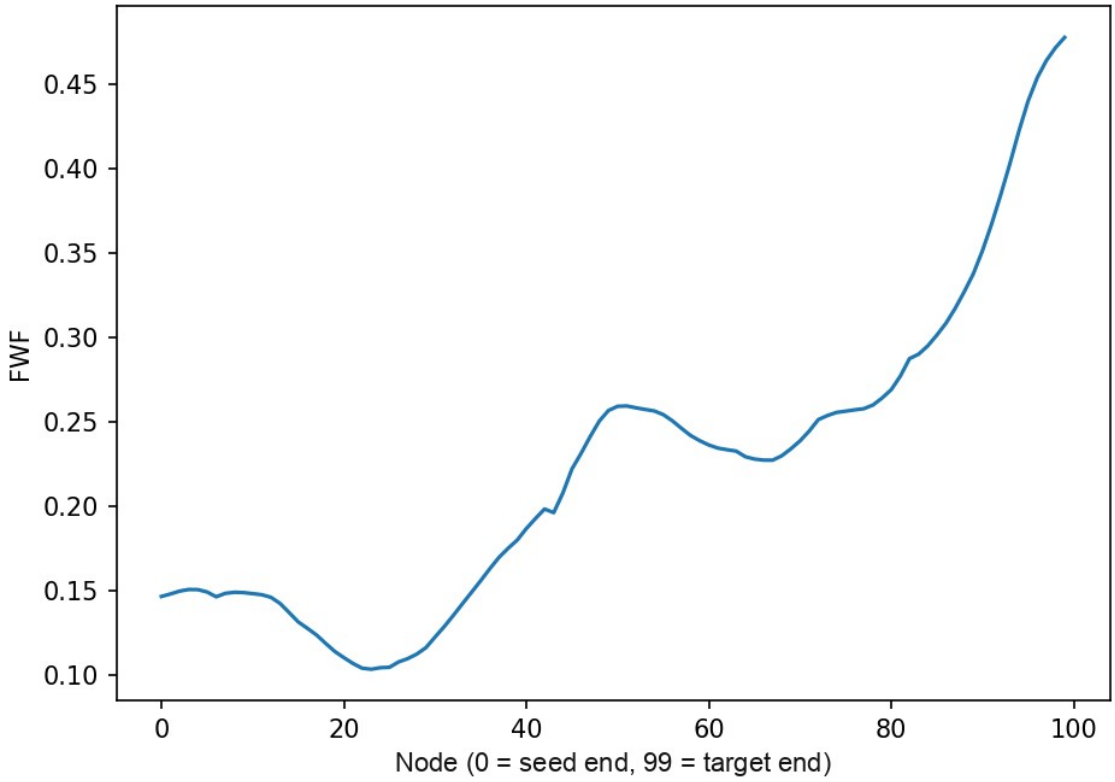
Note. FA peaks in deep white matter and declines toward the hippocampal end. VTA = ventral tegmental area.

Figure S16

NODDI Indices Along the Same Bundle


NDI
0.75
0.70
0.65
0.60
0.55
0.50
0.45
0
20
40
60
80
100
Node (0 = seed end, 99 = target end)

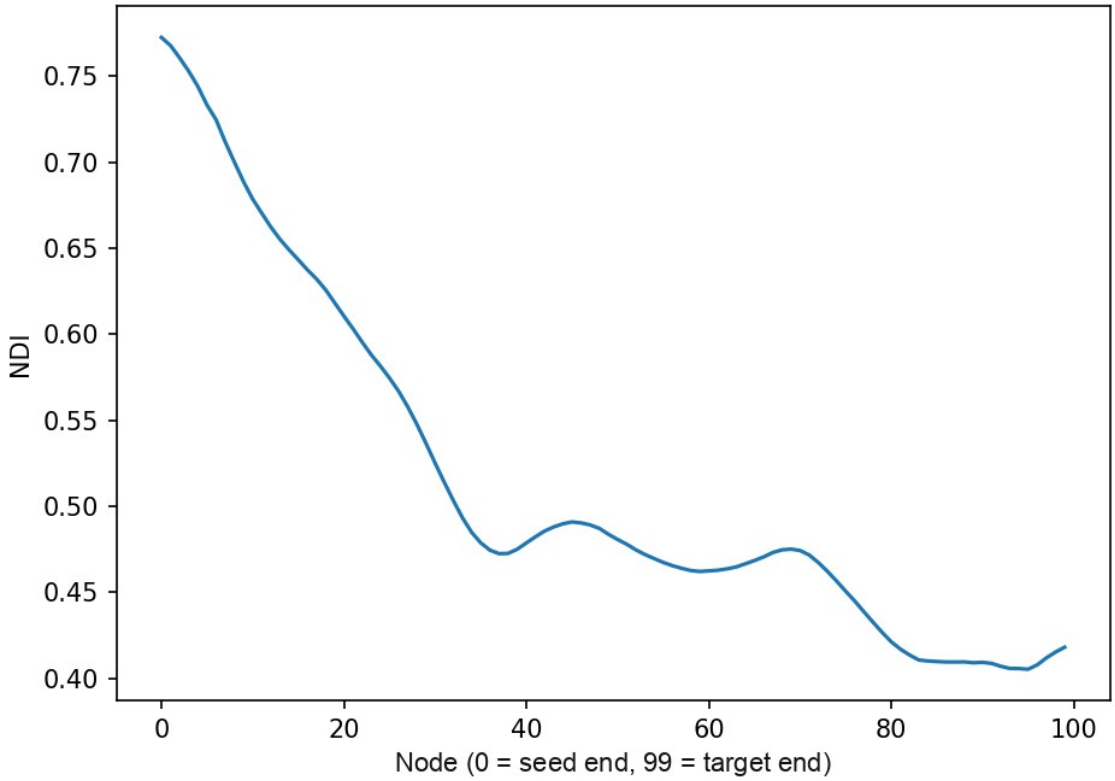

ODI
0.35
0.30
0.25
0.20
0
20
40
60
80
100
Node (0 = seed end, 99 = target end)



Note. NDI (top), ODI (middle) and FWF (bottom). NDI is high near the VTA and increases again toward the hippocampus; ODI is higher at both ends where fibres fan; FWF rises sharply at the hippocampal end adjacent to cerebrospinal fluid.

Figure S17

NDI Along the Anterior Tract for the Same Participant



Note. The anterior tract shares the early trajectory and diverges late.

Verification

The row count of each CSV should equal the number of participants × 100.

When both hemispheres are processed, alignment should be confirmed before averaging. The across-participant correlation between node *i* on the left and node *i* on the right is strongly positive when the profiles are aligned and strongly negative when one side is reversed.

Step 9. Group-level inference

Along-tract data can be analysed at three spatial resolutions: the whole tract, quartiles of the tract, and individual nodes (Table S16). The three address different questions, and no single resolution is appropriate for every study. The resolution, or combination of resolutions, should be specified before results are examined, and the same covariates should be used throughout. Where the neurite density index (NDI) is mentioned below, it refers to the neurite orientation dispersion and density imaging metric profiled in Step 8; VTA denotes the ventral tegmental area.

Table S16

Three Resolutions for Along-Tract Inference

Resolution	Unit of analysis	Question	Correction	Script
Whole tract	Metric averaged across nodes; one value per participant per tract	Is the metric associated with the outcome; does the association differ between subregional tracts	None within tract; across outcomes as pre-specified	`09a_tract_models.py`
Quartiles	Metric averaged within nodes 0–24, 25–49, 50–74, 75–99	Does the association vary along the tract; is it confined to a segment	False discovery rate (FDR) or maximum-statistic permutation across the four quartiles	`09a_tract_models.py`
Nodes	Metric at each of 100 nodes	Where along the tract is the association located	Cluster-extent family-wise error (FWE) by permutation	`09b_nodewise_permutation.R`

Whole-tract model

The metric is averaged across all 100 nodes, or across the retained range if end nodes are trimmed (`--trim`), and the outcome is regressed on the averaged metric and the covariates. All

variables are standardized, so coefficients are standardized betas. When two subregional tracts share most of their course (posterior and anterior VTA → hippocampus), both are entered in one mixed model with a subregion term, two rows per participant, a random intercept for participant, and each row carrying its own streamline length and count. The outcome × subregion interaction is tested by likelihood ratio; when it does not improve fit, the main effect is reported. In the mixed model the metric is the dependent variable, because each participant contributes one metric value per tract and a single outcome value. This resolution has the greatest power when the effect is distributed along the tract, and it does not localize the effect.

Quartile model

The same model is fitted with the metric averaged within each quartile. Because adjacent quartiles are correlated, correction across the four should either use the FDR or a maximum-statistic permutation that preserves their dependence. Uniformity along the tract is tested directly by contrasting the seed-end quartile against the target-end quartile with the mean and the difference entered together: when the mean carries the effect and the difference does not differ from zero, the effect is diffuse rather than localized.

Both resolutions are fitted by one script, which reads the analysis files from Step 8b and the covariate list (`COVARIATES`) from the configuration. With one tract it reports the whole-tract and quartile regressions, the FDR-corrected quartile *p* values and the uniformity contrast; with two tracts it adds the mixed model and the likelihood-ratio test of the interaction for every segment.

```
source 00_config.sh
python 09a_tract_models.py --metric NDI --outcomes memory_accuracy,memory_bias
python 09a_tract_models.py --metric NDI --outcomes memory_accuracy \
    --tracts l_vta_l_hipp,anterior_l_vta_l_hipp
```

Results are printed and written to `$OUT/analysis/<tract>__<METRIC>__tract_models.csv`, one row per model term, with the columns `model` (`ols`, `uniformity` or `mixed`), `tract`, `segment`, `outcome`, `term`, `n`, `beta`, `statistic`, `p`, `p_fdr` and `interaction_p`.

Node-wise model

At each node the outcome is regressed on the node's metric and the covariates, and the fit is compared with a reduced model omitting the metric.

```
full    ~ metric_node + ICV + tract_length + streamline_count + motion + age
reduced ~               ICV + tract_length + streamline_count + motion + age
```

Each node yields a *t* statistic and a parametric *p* value, and adjacent nodes with $p < .05$ form clusters. Family-wise error is controlled at the cluster level with a Freedman–Lane permutation procedure (Freedman & Lane, 1983; Winkler et al., 2014): the residuals of the reduced model are permuted, all 100 nodes are refitted, the largest cluster of each permutation is recorded, and the procedure is repeated 5,000 times. A cluster's *p* value is the proportion of permutations whose

largest cluster is at least as long, and a cluster is retained when that value does not exceed .05; the smallest extent that meets this criterion is reported as the extent threshold. The permutations are drawn once from a fixed seed, so results do not depend on the number of cores. Variables are not standardized in this script; the node-wise estimates are in the units of the outcome per unit of the metric.

The script reads the wide analysis file produced in Step 8b and is called once per outcome, tract and metric: `Rscript 09b_nodewise_permutation.R <analysis.csv> <outcome> <METRIC>_ <out_dir> <label>`, with the label written as `<tract>__<metric>__<outcome>`. Covariate column names and the number of permutations are taken from the `COVARIATES` and `N_PERMUTATIONS` settings in the configuration. Outputs per analysis are `_nodewise.csv` (node, estimate, *t*, *p*), `_clusters.csv` and `_summary.csv`; each call requires a few minutes on one core.

```
source 00_config.sh
mkdir -p "$OUT/permutation"
for outcome in memory_accuracy memory_bias; do
  Rscript 09b_nodewise_permutation.R "$OUT/analysis/${TRACT}__NDI__analysis.csv" \
    "$outcome" NDI_ "$OUT/permutation" "${TRACT}__NDI__${outcome}"
done
```

This resolution localizes an effect along the tract. Its limitation is that cluster-extent correction detects contiguous runs of supra-threshold nodes; when an effect is uniform along the tract, per-node *t* statistics remain near threshold at every node and the outcome becomes sensitive to small changes in model specification. In that situation node-wise results are descriptive and the whole-tract or quartile model carries the inference.

Covariates

Covariates comprise intracranial volume (ICV), the mean streamline length and streamline count of the cleaned bundle (written by Step 6), absolute head motion and age. Length and count are tract-specific and index reconstruction quality. Streamline count can be strongly correlated with the metric under test, which reduces the residual variance available to the test. Its inclusion should be decided in advance and both specifications reported.

Choosing a resolution

A hypothesis about a particular segment of a pathway calls for the quartile or node-wise model. A hypothesis about the pathway as a whole calls for the whole-tract model. Exploratory work is often best served by the whole-tract model for inference with the node-wise profile reported as description. Tests across several metrics, tracts and outcomes accumulate rapidly at any resolution; fixing the analytic hierarchy in advance (primary metric, primary outcome family) is more readily justified than correcting across all tests afterwards.

Preparing node-wise results for the Explorer

Node-wise outputs are stacked into a single long-format file for the Node-wise Tract Explorer, the browser-based viewer described in the Explorer section.

```
python 09c_stack_for_explorer.py "$OUT/permutation"   # writes results_long.csv in that directory
```

The file contains one row per node with the columns `outcome, tract, metric, node, t, p`, together with `hemisphere, N, covariates, extent_threshold, cluster_p` and `passed`.

Scripts

The script `09a_tract_models.py` is reproduced in the Supplementary Scripts appendix.

The script `09b_nodewise_permutation.R` is reproduced in the Supplementary Scripts appendix.

The script `09c_stack_for_explorer.py` is reproduced in the Supplementary Scripts appendix.

Node-wise Tract Explorer

The Node-wise Tract Explorer is a browser-based viewer for node-wise statistical results distributed with this site. It reads a results file in the browser, without uploading it, and presents each analysis as a table row that expands to show the node-wise *t*-value profile, the clusters identified, and, where both hemispheres are present, a left–right comparison. Whole-tract and quartile results are tabular and do not require it.

Open the Explorer · sample file (synthetic data in the expected format)

Input format

The input is a long-format CSV with one row per node. Six columns are required: `outcome`, `tract`, `metric`, `node`, `t` and `p`. Optional columns are `hemisphere` (L or R), `N`, `covariates`, `extent_threshold`, `cluster_p` and `passed`. Any additional column is treated as a grouping variable and becomes a filter. The column names produced by `09b_nodewise_permutation.R` (`Node`, `t_value`, `p_value`) are accepted without renaming. `09c_stack_for_explorer.py` (Step 9) produces this file from the permutation outputs.

Derived quantities

From the node rows the viewer derives the number of significant nodes, clusters (contiguous runs with $p < .05$), the largest cluster, which clusters exceed the extent threshold, and, when a hemisphere column is present, the counts of significant nodes per hemisphere and their overlap.

Display

Selecting a row shows the *t*-value profile with a labelled axis and reference lines at ±2, significant nodes highlighted, retained clusters shaded, and a cluster table. For lateralized data the left and right profiles are drawn on a common axis so that both hemispheres' full *t*-value profiles are visible. The hemisphere panel is descriptive: it reports the number of significant nodes per side and their overlap and does not constitute a test of lateralization.

Page labels may be supplied in the URL, for example `?title=…&method=Freedman-Lane&node0=VTA&node1=hippocampus&n_perms=5000` for a ventral tegmental area (VTA) → hippocampus analysis, and `?data=URL` loads a hosted file directly.

Warping the atlas streamlines

Alongside the volumetric maps, the atlas package distributes the group-average streamlines of each pathway as MRtrix tractograms (`.tck`), two per pathway, one per hemisphere. In this approach those streamlines are carried into a participant with a registration and microstructure is sampled along them, so along-tract profiles are obtained without reconstructing the tract in the participant's own data.

The trajectory represents the group rather than the participant, so this approach suits studies whose outcome is microstructure measured along a pathway. For questions about tract geometry, an individual's trajectory or connectivity, tractography in the participant's own data remains the recommended method and is documented in this site's workflow section. Accuracy depends on the registration: where registration is poor, or anatomy varies between participants, the streamlines sample tissue displaced from the intended location, and this displacement is spatially systematic rather than random noise. Warped streamlines nevertheless recover node-wise profiles similar to those obtained by tracking in the participant's own data.

Procedure

The registration is the one produced in Step 1, with the Montreal Neurological Institute (MNI) template as the moving image and the participant's T1 as the fixed image. No separate registration is needed.

Streamlines are transformed by moving their vertices, not by resampling an image, and vertices move in the opposite sense to images. Step 2 carries the atlas *image* into the participant with the forward warp and the affine; the same registration carries *points* with the inverted affine and the inverse warp field, `mni2t1_1InverseWarp.nii.gz`, which `antsRegistrationSyNQuick.sh` writes in Step 1. Every vertex of every streamline is transformed in one call, and the per-streamline vertex counts are the only record of where one streamline ends and the next begins, so they are kept and used to reassemble the bundle.

```
antsApplyTransformsToPoints -d 3 -i mni_lps.csv -o t1_lps.csv \
  -t "[mni2t1_0GenericAffine.mat,1]" -t mni2t1_1InverseWarp.nii.gz
```

The coordinate files are in left-posterior-superior (LPS) order, while tractogram vertices are read and written in right-anterior-superior (RAS) order, so the x and y signs are flipped before the call and again afterwards. The script below performs the whole operation for every participant and audits the result.

The script `02s_warp_atlas_streamlines.py` is reproduced in the Supplementary Scripts appendix.

Sampling requires the scalar maps and the warped streamlines to be in the same space. Where

the maps are already in anatomical space, as in output from containerized pipelines that reconstruct in a resampled anatomical space, they can be sampled directly. Where the maps are in diffusion space, as in Step 8a, the diffusion-to-T1 direction of the Step 2 transform brings them onto the T1 grid; when the T1 and the diffusion images are header-aligned and differ only in grid, the two spaces already coincide and nothing is applied.

```
convert_xfm -omat diff2str.mat -inverse "$PROJECT/xfm/$s/str2diff.mat"
flirt -in "$NDI" -ref "$T1" -applyxfm -init diff2str.mat -interp trilinear -out ndi_in_T1.nii.gz
```

Each scalar map is then sampled at 100 nodes along the bundle, with the profiler of Step 8, which reads the bundle from `$OUT/<participant>/tckgen/<tract>/`; the warped file is either copied there or that path is changed to the `atlas_streamlines` folder written above. Two of that step's operations are unnecessary here: every streamline in the distributed files carries exactly 100 vertices, so no resampling is required, and the streamlines of a given file share a common orientation, so no reorientation against the seed region is required. Group-level models follow Step 9 unchanged, since the result is a 100-node profile per participant in the same form.

Verification

The script transforms every vertex of every streamline in a single call and compares the returned array's shape with the input's. A different vertex count or any non-finite coordinate stops processing for that participant without writing a file. This failure indicates a wrong or missing transform rather than an inaccurate one.

The script reports, per participant, the proportion of warped vertices inside the non-zero part of that participant's T1-weighted brain image. This proportion is interpreted relative to the sample: a value well below those of other participants indicates a failed registration.

The warped streamlines are overlaid on the participant's own image to confirm that their endpoints occupy the intended anatomical regions. No numeric summary substitutes for this visual inspection of the trajectory.

The warp preserves vertex order within each streamline, carrying the source atlas file's orientation unchanged into every participant. When hemisphere profiles are combined, the two source files are checked for the same anatomical direction. If their directions differ, node 1 in one hemisphere corresponds to the opposite tract end from node 1 in the other, so averaging combines anatomically opposite positions.

The first check automatically reports failure and prevents file creation. Poor but unbroken registration, misplaced endpoints, and reversed hemispheric correspondence all pass that check: files are written, profiles are computed, and numbers are returned without an error, leaving these failures undetected unless the remaining checks are performed.

Other routes

Streamline registration is not specific to ANTs; MRtrix `tcktransform` applies a warp field to a tractogram and serves the same purpose.

For data that already has a whole-brain tractogram, RecoBundles, implemented in DIPY (Garyfallidis et al., 2014), can extract bundles matching an atlas template. Standard whole-brain tracking settings often fail to reconstruct these pathways because their small size makes them sensitive to fibre orientation distribution amplitude thresholds and termination criteria; relaxing the amplitude threshold enables reconstruction. RecoBundles is therefore a possible route for existing tractograms rather than a reliable substitute for tractography targeted at the pathway, and the extracted bundles require inspection against the intended pathway.

Reporting

Reports should state that profiles were sampled along group-average atlas streamlines warped into each participant, name the registration and the transform direction used, and give the quality-control criteria applied to the warped bundles. The profiles should not be described as participant-level tractography, and the limits of a group trajectory should be stated where the outcome concerns anatomy rather than microstructure.

Whole-tract extraction

In this approach an atlas map is warped into a participant and a scalar map is averaged within it. It is suitable for descriptive summaries, replication checks and visualization. It is not recommended as a primary endpoint because spatial variation along the tract is not retained.

Procedure

The 50% binary map (`_thr50`) provides a conservative core; the probabilistic map (`_overlap_prop`, both in the atlas package) supports probability-weighted extraction. The map is warped to T1 space with the Step 1 transforms, using nearest-neighbour interpolation for binary maps and linear interpolation followed by thresholding for probabilistic maps.

```
ATLAS_PROB="$ATLAS_DIR/vta_posterior_hpc/left_vta_posterior_hpc_mni152_1mm_overlap_prop.nii.gz"
antsApplyTransforms -d 3 -i "$ATLAS_PROB" -r "$T1" -o prob_in_T1.nii.gz \
  -t mni2t1_1Warp.nii.gz -t mni2t1_0GenericAffine.mat -n Linear
fslmaths prob_in_T1.nii.gz -thr 0.50 -bin prob50_in_T1_bin.nii.gz
```

The mask is then moved to the scalar map's grid. When the T1 and the scalar map are header-aligned and differ only in grid, the mask is resampled (`flirt -in mask -ref scalar -applyxfm -usesqform -interp nearestneighbour`); otherwise the T1 → diffusion transform is applied. Voxel count, volume and the mean and standard deviation of the scalar within the mask are then extracted.

```
fslstats mask_on_scalar_grid.nii.gz -V                   # voxel count, mm3
fslstats "$NDI" -k mask_on_scalar_grid.nii.gz -M -S       # mean, SD
```

Reporting

For multi-shell data with a fitted neurite orientation dispersion and density imaging (NODDI) model, the neurite density index (NDI) is generally preferred to fractional anisotropy (FA) as a whole-tract summary. It remains a model-derived quantity sensitive to partial volume, registration and mask boundaries, and should be reported with covariates for age, sex, motion, intracranial volume and whole-white-matter NDI and described as a summary measure.

Atlas-guided synthetic streamlines

This approach is intended for studies that require along-tract profiles but cannot reconstruct the tract in every participant. Where the pathway's group-average streamlines are used directly, warping the atlas streamlines is the more direct route to the same kind of profile; the approach described here builds a scaffold from the atlas mask instead, and applies when a mask is the only atlas input available. The warped atlas serves as a geometric scaffold: an ordered centerline is estimated from one endpoint to the other within the mask, 100 nodes are placed along it, jittered streamlines may be generated within the mask, and scalar maps are sampled at each node. The resulting streamlines constitute a reproducible sampling coordinate system rather than a tractography reconstruction, and the approach is less established than corridor-constrained tractography.

Procedure

The probabilistic map is warped onto the scalar grid with linear interpolation and thresholded, at 50% for a conservative core or 25% when the core is short or discontinuous; lower thresholds require justification. The scaffold is built from the mask, the probability map, and native-space start and end regions, with node 0 assigned to the seed side and node 99 to the target side for every participant; an image of the node ordering is saved. Each scalar map is sampled at every node using pyAFQ, DIPY or a custom function, producing one row per participant, tract, metric and node in the long format read by the Explorer. Modelling proceeds as in Step 9 of the main workflow, with cluster or permutation correction.

Reporting

Recommended language: "Atlas-guided streamline scaffolds were used to define a consistent along-tract coordinate system for node-wise sampling." The scaffold should not be described as participant-specific tractography.

Parameters

Table S17 consolidates the parameters used in atlas construction at 7 T and in the example dataset at 3 T.

Table S17

Consolidated Parameters

Step	Parameter	Atlas construction (7 T)	Example dataset (3 T)	Notes
Atlas	threshold	50%	50%	25% when a wa core is short or discontinuous
Registration	tool	`antsRegistrationSyNQuick.sh`	same	Skull-stripped participant imag against `MNI152_T1_1mm n`
Warping	interpolation	nearest-neighbour	same	Linear for proba maps, then thres
Corridor	dilation	1–2 voxels	2	4 when registrat uncertain
Tractography	algorithm	iFOD2 (default)	same	
Tractography	`-cutoff`	0.03 to 0.08 by pathway; 0.06 for VTA → hippocampus (Atlas construction)	0.01	0.01 is the value routine use with corridor mask a default in `00_config.sh`; confirm by pilot 4). MRtrix3 def 0.05
Tractography	`-select`	2500	2500	
Tractography	`-seeds`	25,000,000	25,000,000	
Tractography	`-minlength`, `-maxlength`	35, 65 mm	35, 65 mm	Per tract family
Tractography	`-seed_unidirectional`	on	on	

Step	Parameter	Atlas construction (7 T)	Example dataset (3 T)	Notes
	, `-stop`			
Tractography	ACT, backtrack, angle, step size	off / default	off / default	ACT did not reconstruct with masks
Cleaning	`n_points`, `clean_rounds`	100, 5	same	
Cleaning	`distance_threshold`, `length_threshold`	3, 2	same	Mahalanobis *SD* length *SD*
Cleaning	QuickBundles threshold	5 mm	not used	For two-bundle
Quality control	flags		voxels < 50 or > 5,000; maximum density < 5	
Profiling	nodes	100	100	End nodes 0–4 a 99 may be trimm
Profiling	weighting	Gaussian (Mahalanobis)	same	
NODDI	intrinsic parallel diffusivity, white / gray matter	1.7 / 1.1 × 10^{-3} mm²/s	same	Refit for gray-m regions
NODDI	maps	modulated NDI and ODI; FWF	same	
Inference, whole tract	model		mixed model with subregion term; likelihood-ratio test for interaction	one value per participant per t
Inference, quartiles	segments and correction		nodes 0–24, 25–49, 50–74, 75–99; FDR or maximum-statistic permutation	seed-end versus end contrast for uniformity
Inference, nodes	permutations	5,000	5,000	Freedman–Lane
Inference,	cluster criterion	extent ≥ 95th percentile of null	same	α = .05 per node

Step	Parameter	Atlas construction (7 T)	Example dataset (3 T)	Notes
nodes		maximum		
Inference, nodes	covariates		ICV, length, count, motion, age	Count is colline NDI; decide in advance

Note. VTA = ventral tegmental area; ACT = anatomically constrained tractography; NODDI = neurite orientation dispersion and density imaging; NDI = neurite density index; ODI = orientation dispersion index; FWF = free-water fraction; FDR = false discovery rate; ICV = intracranial volume.

Troubleshooting

Table S18 lists common problems, their likely causes and remedies. A quality-assurance checklist follows.

Table S18

Common Problems, Causes and Remedies

Symptom	Likely cause	Remedy
Warped atlas displaced anteriorly or posteriorly	Poor T1 → MNI registration, or mismatched coverage (skull-stripped participant image with a whole-head template)	Check brain extraction; use the brain template; overlay the warped template on the T1 before warping labels
Left and right appear reversed	Incorrect transform, header orientation error, mislabelled hemisphere	Check with `fslorient` and `fslhd`; overlay known asymmetric anatomy
Atlas mask empty after warping	Threshold too strict after linear interpolation, or grid mismatch	Inspect the probability map before thresholding; derive a 25% map from the `_overlap_prop` file (downloads page); warp the binary map with nearest-neighbour interpolation
Mask includes ventricle or gray matter	Registration error or excessive dilation	Reduce dilation; inspect in native space
VTA in the cerebral peduncle or pons	Local registration failure	Rerun registration for the participant; consider a midbrain-weighted mask via `-x`
`tckgen` reaches far fewer streamlines than the target	Cutoff too high for the field strength; corridor too narrow; seed or target disconnected from the corridor	Run the pilot sweep; confirm the seed and target lie within the inclusion zone; try a 25% map derived from the `_overlap_prop` file, or one additional voxel of dilation
Tract appears thick at a permissive cutoff	Expected before cleaning	Compare cleaned bundles; if still thick, reduce dilation
Cleaning removes nearly all streamlines	Two bundles cleaned as one, or a sparse raw tract	Apply the QuickBundles division; check the raw streamline count

Symptom	Likely cause	Remedy
Profile endpoints reversed in some participants	Orientation failed	Assign node 0 by the seed region; check centroid orientation per participant
Whole-tract NDI implausibly high or low	Grid mismatch, partial volume, global tissue differences	Confirm a common grid; add a whole-white-matter NDI covariate; inspect the overlay
AMICO `FileNotFoundError` during parallel runs	Kernel directory regenerated by every job	Generate kernels once on one participant before starting the pool
`pip install pyAFQ` fails on `zipp`	Outdated system package	`pip install --user --upgrade zipp`
Coverage audit flags every corridor as too small	Corridors are below 1% of brain volume by design	Raise the threshold or disregard the flag

Note. MNI = Montreal Neurological Institute; VTA = ventral tegmental area; NDI = neurite density index.

Symptoms that appear at Step 1 or Step 2 often originate upstream, in the brain mask, the skull strip or the T1-to-diffusion transform; those steps are documented in the Diffusion MRI Preprocessing tutorial.

Quality-assurance checklist

- Record atlas version, threshold, dilation, interpolation, transforms, cutoff, scalar maps, covariates and software versions.
- Inspect the atlas in MNI space before warping.
- Inspect every warped region over the T1 and diffusion images.
- Check voxel counts and volumes after warping.
- Save at least one overlay image per participant and step.
- Inspect streamlines in native space and compare with the atlas in MNI space.
- Verify node ordering and coverage.
- Check missingness by tract and participant before group models.

Software

Table S19 lists the software used in the workflow. The workflow depends on FSL (Jenkinson et al., 2012), ANTs (Avants et al., 2008), MRtrix3 (Tournier et al., 2019) and Python with DIPY (Garyfallidis et al., 2014) and pyAFQ (Kruper et al., 2021); AMICO (Daducci et al., 2015) is required only for neurite orientation dispersion and density imaging (NODDI) fitting and R only for the node-wise permutation test.

Table S19

Software Used in the Workflow

Software	Use
FSL	Templates; `fslmaths`, `fslstats`, `flirt`; `probtrackx2` for the FSL route
ANTs	`antsRegistrationSyNQuick.sh`, `antsApplyTransforms`
MRtrix3	`mrconvert`, `dwi2response`, `responsemean`, `dwi2fod`, `mtnormalise`, `tckgen`, `tckinfo`, `tckstats`, `tckmap`, `mrview`
Python 3 with DIPY, pyAFQ, nibabel, numpy, scipy, pandas, statsmodels, matplotlib	Cleaning, profiling, quality control, whole-tract and quartile models
AMICO	NODDI fitting
R with readr, dplyr, stringr, tibble, foreach, doParallel	`09b_nodewise_permutation.R`
FSLeyes, mrview or ITK-SNAP	Inspection

Fibre orientation distribution estimation

The Diffusion MRI Preprocessing tutorial covers this as its Steps 11 and 12 (response functions, FOD estimation) and is the fuller description. The commands are repeated here for readers whose preprocessing ended at the tensor fit and who arrive without a `wm_fod_norm.mif`.

Response functions are estimated per participant with the unsupervised method of Dhollander et al. (2016), averaged across participants, and used for multi-shell multi-tissue constrained spherical deconvolution (Jeurissen et al., 2014) followed by intensity normalisation (Raffelt et al., 2017). The group average ties the cohort together: adding participants later changes it, so their fibre orientation distributions are comparable with the earlier ones only if every participant is refitted. The script `00b_fod_estimation.sh`, shown below, runs these commands across

participants.

```
# 1. conversion with gradients embedded
mrconvert data.nii.gz dwi.mif -fslgrad bvecs bvals
mrconvert nodif_brain_mask.nii.gz mask.mif

# 2. tissue response functions (multi-shell)
dwi2response dhollander dwi.mif wm_response.txt gm_response.txt csf_response.txt -mask mask.mif

# 3. group-average response functions (once, across participants)
responsemean */wm_response.txt group_wm_response.txt
responsemean */gm_response.txt group_gm_response.txt
responsemean */csf_response.txt group_csf_response.txt

# 4. multi-shell multi-tissue CSD
dwi2fod msmt_csd dwi.mif group_wm_response.txt wm_fod.mif \
        group_gm_response.txt gm_fod.mif group_csf_response.txt csf_fod.mif -mask mask.mif

# 5. intensity normalisation
mtnormalise wm_fod.mif wm_fod_norm.mif gm_fod.mif gm_fod_norm.mif csf_fod.mif csf_fod_norm.mif
-mask mask.mif
```

Single-shell data require single-shell constrained spherical deconvolution (`dwi2response tournier`, `dwi2fod csd`) rather than the multi-shell multi-tissue model used for the atlas and the example dataset. The corridor workflow is independent of how the FOD image was estimated.

The script `00b_fod_estimation.sh` is reproduced in the Supplementary Scripts appendix.

FSL route

`probtrackx2` accepts the dilated corridor as a `--waypoints` mask and the exclusions as `--avoid`, with a BEDPOSTX model as input. Its output is voxel-wise connectivity rather than streamlines, so along-tract profiling is not available from it; it is appropriate when a whole-tract summary is sufficient.

Scripts

Each script appears in full on the workflow page for its step. Table S20 lists the set in run order; `02s_warp_atlas_streamlines.py` belongs to an alternative route rather than the nine steps. All scripts read a single configuration file, `00_config.sh`, which is edited once per project. The shell scripts source it themselves; the Python and R scripts read the exported settings, so `source 00_config.sh` is run in the shell before they are called, and again after every edit to the file. Each shell script writes a log to `$OUT/logs/`. Steps 0b, 1, 2, 2s, 3, 5, 6, 7 and 8a skip participants whose output already exists (`FORCE=1` recomputes); Steps 4, 8, 8b and 9 recompute on every run. Missing inputs are reported by participant.

Table S20

Scripts, in Run Order

File	Step	Requires
`00_config.sh`	Project paths, tract definition, parameters, covariate list	bash
`00b_fod_estimation.sh`	0b. Fibre orientation distributions, when preprocessing ended at the tensor	MRtrix3
`01_register_mni_to_t1.sh`	1. Registration	ANTs
`02_warp_rois.sh`	2. Warping of seed, target and atlas	ANTs, FSL
`02s_warp_atlas_streamlines.py`	2s. Warping of the atlas streamlines, for the appendix route in place of Steps 2 to 6	Python, ANTs
`03_build_corridor_mask.sh`	3. Corridor and exclusion mask	FSL
`04_tune_cutoff.sh`	4. Pilot sweep of the cutoff	MRtrix3
`04b_compare_cutoffs.py`	4. Side-by-side images, Dice overlap, summary table	Python, MRtrix3
`05_tractography.sh`	5. Tractography	MRtrix3
`06_clean_bundles.py`	6. Cleaning; streamline count and length of the cleaned bundle	Python, pyAFQ, DIPY
`07_visual_qc.py`	7. Overlay images and flags	Python, MRtrix3
`08a_noddi_fit.py`	8. NODDI fit	Python, AMICO

File	Step	Requires
`08_node_profiles.py`	8. Along-tract profiles	Python, DIPY
`08b_build_analysis_csv.py`	8. Analysis files for Step 9	Python
`09a_tract_models.py`	9. Whole-tract and quartile models	Python, statsmodels
`09b_nodewise_permutation.R`	9. Node-wise permutation test	R
`09c_stack_for_explorer.py`	9. Explorer input	Python

Note. NODDI = neurite orientation dispersion and density imaging.

The complete set can be retrieved with the following command.

```
mkdir -p mesoconnect_scripts && cd mesoconnect_scripts
base="https://diffusiontensorimaging-repos.github.io/MesoConnect-Tutorial/scripts"
for f in 00_config.sh 00b_fod_estimation.sh 01_register_mni_to_t1.sh 02_warp_rois.sh \
         02s_warp_atlas_streamlines.py 03_build_corridor_mask.sh 04_tune_cutoff.sh
04b_compare_cutoffs.py \
         05_tractography.sh 06_clean_bundles.py 07_visual_qc.py 08a_noddi_fit.py \
         08_node_profiles.py 08b_build_analysis_csv.py 09a_tract_models.py \
         09b_nodewise_permutation.R 09c_stack_for_explorer.py; do
  curl -sSLO "$base/$f"
done
```

Validation status

The scripts generalize those used to process the example dataset. Before release they were run from Step 0b to Step 8b on three participants of that dataset and compared with the outputs the dataset's own scripts had produced: warped regions and corridors overlapped at Dice ≥ .93, cleaned streamline counts and lengths agreed within the run-to-run variability of tractography, NODDI maps were identical, and along-tract profiles correlated at $r \geq .94$ across nodes. The group-level scripts reproduce that dataset's whole-tract, quartile and node-wise results. The run used MRtrix3 3.0.7, FSL 6.0.5.1, ANTs 2.3.5, Python 3.8, DIPY 1.8.0, pyAFQ 1.3.5, AMICO 2.1.1 and R 4 on Linux.

The atlas-streamline script was checked separately on three participants of the same dataset, against the atlas map that the dataset's own pipeline had warped into each participant independently as an image: 98.7% to 99.2% of the warped streamline vertices fell inside that map, and the streamline ends lay a median of 0.5 mm from the seed region and 0.7 to 0.9 mm from the target region.

The scripts from which these were generalized are in the SDN-IMPACT-DTI repository.

Suggested methods text

The paragraphs below describe the corridor workflow in the form required for a Method section and may be adapted. Values in square brackets are study-specific and should be replaced with the values used; values stated without brackets are those of the workflow as documented here.

The whole tutorial is also available as a single document, with tables and figures numbered S1, S2 and so on and the scripts in an appendix: PDF and Word.

Tractography

Mesolimbic pathways were reconstructed in each participant using the MesoConnect Atlas as an anatomical constraint on participant-level tractography. For each pathway, the seed region, the target region and the atlas's 50% group-overlap map were transformed from Montreal Neurological Institute (MNI) space to each participant's T1-weighted image using symmetric diffeomorphic registration (ANTs; Avants et al., 2008) and then to diffusion space using the affine transform estimated during preprocessing, with nearest-neighbour interpolation. The warped atlas was dilated by [two] voxels and combined with the seed and target regions to define an inclusion zone; its inverse served as the sole exclusion mask. Fibre orientation distributions were estimated with multi-shell multi-tissue constrained spherical deconvolution (Jeurissen et al., 2014) using group-averaged response functions (Dhollander et al., 2016) and intensity normalisation in MRtrix3 (Tournier et al., 2019). Probabilistic tractography (iFOD2; Tournier et al., 2010) was seeded unidirectionally from the seed region, required to enter the target region, and terminated on entry, with [2,500] streamlines selected from a maximum of [25 million] seeds, a fibre orientation distribution amplitude cutoff of [0.01], and length bounds of [35] to [65] mm. The cutoff was selected from a pilot sweep in [five] participants as the most permissive value that reached the streamline target in every pilot participant and reproduced the trajectory obtained at the most conservative cutoff.

Bundle cleaning and quality control

Streamlines were resampled to 100 points and cleaned with pyAFQ (Kruper et al., 2021; Yeatman et al., 2012) by iteratively removing streamlines more than 3 *SD* (Mahalanobis distance) from the bundle centroid or longer than the mean length by more than 2 *SD*, for up to five iterations. [Where a tract reconstructed as two distinct bundles, streamlines were first clustered with QuickBundles (Garyfallidis et al., 2012; 5 mm threshold), each cluster was cleaned separately, and the anatomically correct cluster was retained after inspection.] Every cleaned bundle was inspected as a tract-density overlay on the mean $b = 0$ image; bundles with fewer than 50 or more than 5,000 tract-density voxels, or a maximum density below 5 streamlines per voxel, were flagged for review. [*n*] participants were excluded at this stage.

Along-tract profiles

Each cleaned bundle was oriented from seed to target against its QuickBundles centroid and sampled at 100 equidistant nodes using Gaussian-weighted tract profiling (Yeatman et al., 2012) as implemented in DIPY (Garyfallidis et al., 2014). [Fractional anisotropy was taken from the diffusion tensor fit. Neurite density index, orientation dispersion index and free-water fraction were obtained from NODDI (Zhang et al., 2012) fitted with AMICO (Daducci et al., 2015), using tissue-weighted modulated maps for neurite density and orientation dispersion (Parker et al., 2021).] Streamline count and mean streamline length were recorded for each bundle as indices of reconstruction quality.

Statistical analysis

[Whole tract: The metric was averaged across nodes and related to the outcome in a linear model (or, for subregional tracts sharing most of their course, a mixed model with a subregion term and a random intercept for participant, in which the outcome × subregion interaction was tested by likelihood ratio).] [Quartiles: The metric was averaged within nodes 0–24, 25–49, 50–74 and 75–99 and the same model was fitted per quartile, with correction across quartiles by false discovery rate (or maximum-statistic permutation).] [Nodes: At each node the outcome was regressed on the metric and covariates. Adjacent nodes with parametric $p < .05$ formed clusters, and family-wise error was controlled at the cluster level: a null distribution of maximum cluster extent was obtained from 5,000 Freedman–Lane permutations (Freedman & Lane, 1983; Winkler et al., 2014), and clusters were retained when the proportion of permutations with a maximum cluster at least as long did not exceed .05.] Covariates were [intracranial volume, the mean streamline length and streamline count of the cleaned bundle, absolute head motion and age]. [Variables in the whole-tract and quartile models were standardized.] The resolution of analysis and the covariate set were specified before results were examined.

Abbreviations

Abbreviation	Term
ACT	anatomically constrained tractography
AMICO	Accelerated Microstructure Imaging via Convex Optimization
CSD	constrained spherical deconvolution
DIPY	Diffusion Imaging in Python
FA	fractional anisotropy
FDR	false discovery rate
FOD	fibre orientation distribution
FWE	family-wise error
FWF	free-water fraction
HCP	Human Connectome Project
ICV	intracranial volume
iFOD2	second-order integration over fibre orientation distributions
LPS	left-posterior-superior (coordinate order)
MNI	Montreal Neurological Institute
NDI	neurite density index
NODDI	neurite orientation dispersion and density imaging
ODI	orientation dispersion index
pyAFQ	Python Automated Fiber Quantification
RAS	right-anterior-superior (coordinate order)
ROI	region of interest
SN	substantia nigra
TDI	tract-density image
VTA	ventral tegmental area

Citation

The atlas is described in the manuscript *Mapping Mesolimbic Dopamine Projections to Subcortical Circuits Underlying Adaptive Behavior: A Human 7T Diffusion Tractography Atlas* (Elliott, Mopuru, Hoffman, Leong, Volkow, Olson, & Murty), currently under review. Work that uses the atlas or this workflow should cite that paper; the full reference and DOI will be added here when it is published. The atlas package's terms are scoped by material (Human Connectome Project (HCP)-derived maps under the WU-Minn HCP Consortium Open Access Data Use Terms; documentation and figures under CC BY 4.0; code under the MIT License; the Montreal Neurological Institute MNI152 template under the FSL licence); see the `LICENSE` file in the package and the atlas repository.

Source atlases and software used in the workflow are cited on the pages where they are used and listed on the references page.

References

Avants, B. B., Epstein, C. L., Grossman, M., & Gee, J. C. (2008). Symmetric diffeomorphic image registration with cross-correlation: Evaluating automated labeling of elderly and neurodegenerative brain. *Medical Image Analysis, 12*(1), 26–41. https://doi.org/10.1016/j.media.2007.06.004

Daducci, A., Canales-Rodríguez, E. J., Zhang, H., Dyrby, T. B., Alexander, D. C., & Thiran, J.-P. (2015). Accelerated microstructure imaging via convex optimization (AMICO) from diffusion MRI data. *NeuroImage, 105*, 32–44. https://doi.org/10.1016/j.neuroimage.2014.10.026

Dhollander, T., Raffelt, D., & Connelly, A. (2016). Unsupervised 3-tissue response function estimation from single-shell or multi-shell diffusion MR data without a co-registered T1 image. *ISMRM Workshop on Breaking the Barriers of Diffusion MRI*, 5.

Frazier, J. A., Chiu, S., Breeze, J. L., Makris, N., Lange, N., Kennedy, D. N., Herbert, M. R., Bent, E. K., Koneru, V. K., Dieterich, M. E., Hodge, S. M., Rauch, S. L., Grant, P. E., Cohen, B. M., Seidman, L. J., Caviness, V. S., & Biederman, J. (2005). Structural brain magnetic resonance imaging of limbic and thalamic volumes in pediatric bipolar disorder. *American Journal of Psychiatry, 162*(7), 1256–1265. https://doi.org/10.1176/appi.ajp.162.7.1256

Freedman, D., & Lane, D. (1983). A nonstochastic interpretation of reported significance levels. *Journal of Business & Economic Statistics, 1*(4), 292–298. https://doi.org/10.1080/07350015.1983.10509354

Garyfallidis, E., Brett, M., Correia, M. M., Williams, G. B., & Nimmo-Smith, I. (2012). QuickBundles, a method for tractography simplification. *Frontiers in Neuroscience, 6*, 175. https://doi.org/10.3389/fnins.2012.00175

Garyfallidis, E., Brett, M., Amirbekian, B., Rokem, A., van der Walt, S., Descoteaux, M., Nimmo-Smith, I., & DIPY Contributors. (2014). Dipy, a library for the analysis of diffusion MRI data. *Frontiers in Neuroinformatics, 8*, 8. https://doi.org/10.3389/fninf.2014.00008

Hoopes, A., Mora, J. S., Dalca, A. V., Fischl, B., & Hoffmann, M. (2022). SynthStrip: Skull-stripping for any brain image. *NeuroImage, 260*, 119474. https://doi.org/10.1016/j.neuroimage.2022.119474

Jenkinson, M., Beckmann, C. F., Behrens, T. E. J., Woolrich, M. W., & Smith, S. M. (2012). FSL. *NeuroImage, 62*(2), 782–790. https://doi.org/10.1016/j.neuroimage.2011.09.015

Jeurissen, B., Tournier, J.-D., Dhollander, T., Connelly, A., & Sijbers, J. (2014). Multi-tissue constrained spherical deconvolution for improved analysis of multi-shell diffusion MRI data. *NeuroImage, 103*, 411–426. https://doi.org/10.1016/j.neuroimage.2014.07.061

Kruper, J., Yeatman, J. D., Richie-Halford, A., Bloom, D., Grotheer, M., Caffarra, S., Kiar, G.,

Karipidis, I. I., Roy, E., Chandio, B. Q., Garyfallidis, E., & Rokem, A. (2021). Evaluating the reliability of human brain white matter tractometry. *Aperture Neuro, 2021*(1). https://doi.org/10.52294/e6198273-b8e3-4b63-babb-6e6b0da10669

Makris, N., Goldstein, J. M., Kennedy, D., Hodge, S. M., Caviness, V. S., Faraone, S. V., Tsuang, M. T., & Seidman, L. J. (2006). Decreased volume of left and total anterior insular lobule in schizophrenia. *Schizophrenia Research, 83*(2–3), 155–171. https://doi.org/10.1016/j.schres.2005.11.020

Murty, V. P., Shermohammed, M., Smith, D. V., Carter, R. M., Huettel, S. A., & Adcock, R. A. (2014). Resting state networks distinguish human ventral tegmental area from substantia nigra. *NeuroImage, 100*, 580–589. https://doi.org/10.1016/j.neuroimage.2014.06.047

Parker, C. S., Veale, T., Bocchetta, M., Slattery, C. F., Malone, I. B., Thomas, D. L., Schott, J. M., Cash, D. M., & Zhang, H. (2021). Not all voxels are created equal: Reducing estimation bias in regional NODDI metrics using tissue-weighted means. *NeuroImage, 245*, 118749. https://doi.org/10.1016/j.neuroimage.2021.118749

Pauli, W. M., Nili, A. N., & Tyszka, J. M. (2018). A high-resolution probabilistic in vivo atlas of human subcortical brain nuclei. *Scientific Data, 5*, 180063. https://doi.org/10.1038/sdata.2018.63

Raffelt, D., Dhollander, T., Tournier, J.-D., Tabbara, R., Smith, R. E., Pierre, E., & Connelly, A. (2017). Bias field correction and intensity normalisation for quantitative analysis of apparent fibre density. *Proceedings of the International Society for Magnetic Resonance in Medicine, 25*, 3541.

Tournier, J.-D., Calamante, F., & Connelly, A. (2010). Improved probabilistic streamlines tractography by 2nd order integration over fibre orientation distributions. *Proceedings of the International Society for Magnetic Resonance in Medicine, 18*, 1670.

Tournier, J.-D., Smith, R., Raffelt, D., Tabbara, R., Dhollander, T., Pietsch, M., Christiaens, D., Jeurissen, B., Yeh, C.-H., & Connelly, A. (2019). MRtrix3: A fast, flexible and open software framework for medical image processing and visualisation. *NeuroImage, 202*, 116137. https://doi.org/10.1016/j.neuroimage.2019.116137

Trutti, A. C., Fontanesi, L., Mulder, M. J., Bazin, P.-L., Hommel, B., & Forstmann, B. U. (2021). A probabilistic atlas of the human ventral tegmental area (VTA) based on 7 Tesla MRI data. *Brain Structure and Function, 226*, 1155–1167. https://doi.org/10.1007/s00429-021-02231-w

Tziortzi, A. C., Haber, S. N., Searle, G. E., Tsoumpas, C., Long, C. J., Shotbolt, P., Douaud, G., Jbabdi, S., Behrens, T. E. J., Rabiner, E. A., Jenkinson, M., & Gunn, R. N. (2014). Connectivity-based functional analysis of dopamine release in the striatum using diffusion-weighted MRI and positron emission tomography. *Cerebral Cortex, 24*(5), 1165–1177. https://doi.org/10.1093/cercor/bhs397

Vu, A. T., Auerbach, E., Lenglet, C., Moeller, S., Sotiropoulos, S. N., Jbabdi, S., Andersson, J., Yacoub, E., & Ugurbil, K. (2015). High resolution whole brain diffusion imaging at 7T for the Human Connectome Project. *NeuroImage, 122*, 318–331. https://doi.org/10.1016/j.neuroimage.2015.08.004

Winkler, A. M., Ridgway, G. R., Webster, M. A., Smith, S. M., & Nichols, T. E. (2014). Permutation inference for the general linear model. *NeuroImage, 92*, 381–397. https://doi.org/10.1016/j.neuroimage.2014.01.060

Yeatman, J. D., Dougherty, R. F., Myall, N. J., Wandell, B. A., & Feldman, H. M. (2012). Tract profiles of white matter properties: Automating fiber-tract quantification. *PLoS ONE, 7*(11), e49790. https://doi.org/10.1371/journal.pone.0049790

Zhang, H., Schneider, T., Wheeler-Kingshott, C. A., & Alexander, D. C. (2012). NODDI: Practical in vivo neurite orientation dispersion and density imaging of the human brain. *NeuroImage, 61*(4), 1000–1016. https://doi.org/10.1016/j.neuroimage.2012.03.072

Supplementary scripts

The scripts below are those distributed with the tutorial, in run order.

00_config.sh

```
#!/bin/bash
# ============================================================================
# Shared configuration for the MesoConnect corridor workflow; every step script sources it.
# Inputs:  $PROJECT/{anat,dwi,noddi,xfm}/<subj>/, subjects.txt, covariates.csv, $ATLAS_DIR
# Outputs: none of its own; every step writes under $OUT
# Run:     source 00_config.sh   (FSL set up first; re-source after every edit)
# ============================================================================

# --- project ----------------------------------------------------------------
export PROJECT="/path/to/project"
export SUBJECTS_FILE="$PROJECT/subjects.txt"     # one participant ID per line
export ATLAS_DIR="/path/to/MesoConnect_Atlas"    # unzipped atlas package, plus roi_maps/
export OUT="$PROJECT/derivatives/mesoconnect"    # everything this workflow writes
export FORCE="${FORCE:-0}"                        # 1 = recompute existing outputs

# --- tract definition (one tract per run; change these lines and rerun Steps 2-9) ---
export TRACT="l_vta_l_hipp"                      # name used for all outputs
export SEED_MNI="$ATLAS_DIR/roi_maps/left_VTA_0.25_bin.nii.gz"
export TARGET_MNI="$ATLAS_DIR/roi_maps/HPC_L_0.5_bin.nii.gz"
export ATLAS_MNI="$ATLAS_DIR/vta_posterior_hpc/left_vta_posterior_hpc_mni152_1mm_thr50.nii.gz"
# Streamline file of the same pathway; read only by the atlas-streamline route (appendix).
export ATLAS_STREAMLINES_MNI="$ATLAS_DIR/vta_posterior_hpc/left_vta_posterior_hpc_mni152_1mm.tck"

# --- T1 -> DWI transform (Step 2): "matrix" = xfm/<subj>/str2diff.mat, "header" = same grid
export T1_TO_DWI="matrix"

# --- corridor and tractography parameters (see Reference: Parameters) --------
export DILATE_VOX=2          # corridor dilation, voxels (1-2; 4 if registration is uncertain)
export CUTOFF=0.01           # FOD amplitude cutoff selected in Step 4 (MRtrix default 0.05)
export SELECT=2500           # streamlines to retain
export SEEDS=25000000        # maximum seeding attempts
export MINLEN=35             # minimum streamline length, mm
export MAXLEN=65             # maximum streamline length, mm

# --- group-level models (Steps 8b and 9) -------------------------------------
export COVARIATES_CSV="$PROJECT/covariates.csv"  # Subject, covariates, outcomes
# Columns of covariates.csv plus Mean_length_mm and Streamline_count (Step 6, cleaned bundle)
export COVARIATES="ICV,Mean_length_mm,Streamline_count,absolute_motion,age"
export N_PERMUTATIONS=5000

# --- software and concurrency (peak load is about jobs x threads; tuned for 48 cores) ---
export MNI_TEMPLATE="$FSLDIR/data/standard/MNI152_T1_1mm_brain.nii.gz"
export ANTSPATH="${ANTSPATH:-/usr/local/ants/bin}"
export PATH="$ANTSPATH:$PATH"
export NTHREADS=8            # threads per MRtrix command
export MAXJOBS=8             # participants processed concurrently in light steps (2, 3)
export ANTS_THREADS=4        # threads per registration (Step 1)
export ANTS_JOBS=4           # registrations run concurrently (Step 1)
export FOD_JOBS=2            # FOD estimations run concurrently (Step 0b; memory-intensive)

# --- helper functions used by the step scripts -------------------------------

start_log() {
```

```
  mkdir -p "$OUT/logs"
  exec > >(tee -a "$OUT/logs/$(basename "$1").log") 2>&1
  echo "== $(date '+%F %T')  $(basename "$1")  TRACT=$TRACT"
}

read_subjects() {
  SUBJECTS=($(tr -d '\r' < "$SUBJECTS_FILE"))
  echo "== ${#SUBJECTS[@]} participants in $SUBJECTS_FILE"
}

# Block until fewer than N background jobs are running (usage: throttle N)
throttle() {
  while [ "$(jobs -r | wc -l)" -ge "$1" ]; do
    sleep 1
  done
}

# A bare "wait" hangs on bash 5.0-5.2 (it also waits for the tee started by start_log).
wait_for_jobs() {
  throttle 1
}

nvox() {
  fslstats "$1" -V | awk '{print $1}'
}

present() {
  if [ -f "$1" ]; then echo ok; else echo MISSING; fi
}
```

01_register_mni_to_t1.sh

```
#!/bin/bash
# ================================================================================
# Step 1. ANTs SyN registration of the MNI152 template (moving) to each T1 (fixed)
# ================================================================================
# Step 2 applies the forward affine + warp to carry MNI-space seed, target and atlas into T1.
# In:  $PROJECT/anat/<subj>/<subj>_T1w_brain.nii.gz (skull-stripped), $MNI_TEMPLATE
# Out: $OUT/<subj>/reg/mni2t1_{0GenericAffine.mat,1Warp,1InverseWarp,Warped}; QC = Warped
# Run: bash 01_register_mni_to_t1.sh  (FORCE=1 redoes everyone). Hours; log in $OUT/logs/
# ================================================================================

source "$(dirname "$0")/00_config.sh"
start_log "$0"
read_subjects

# Register one participant; returns early if the T1 is missing or the warp already exists.
register_one() {
  local s=$1
  local t1="$PROJECT/anat/$s/${s}_T1w_brain.nii.gz"
  local d="$OUT/$s/reg"
  if [ ! -f "$t1" ]; then
    echo "!! $s missing $t1"
    return
  fi
  if [ "$FORCE" != 1 ] && [ -f "$d/mni2t1_1Warp.nii.gz" ]; then
    echo "== $s already registered (FORCE=1 to recompute)"
    return
  fi
  mkdir -p "$d"
  antsRegistrationSyNQuick.sh -d 3 \
```

```
    -f "$t1" \
    -m "$MNI_TEMPLATE" \
    -o "$d/mni2t1_" \
    -n "$ANTS_THREADS"
  echo ">> $s registered"
}

for s in "${SUBJECTS[@]}"; do
  register_one "$s" &
  throttle "$ANTS_JOBS"
done
# Not a bare "wait": that would also hang on the tee process start_log opened.
wait_for_jobs

# Audit. Step 2 skips any participant with a MISSING affine or warp, so fix those first.
printf "\nSubject\tAffine\tWarp\tInverseWarp\n"
for s in "${SUBJECTS[@]}"; do
  d="$OUT/$s/reg"
  printf "%s\t%s\t%s\t%s\n" "$s" \
    "$(present "$d/mni2t1_0GenericAffine.mat")" \
    "$(present "$d/mni2t1_1Warp.nii.gz")" \
    "$(present "$d/mni2t1_1InverseWarp.nii.gz")"
done
```

02_warp_rois.sh

```
#!/bin/bash
# ================================================================================
# Step 2. Warp the seed, target and tract atlas into diffusion space
# ================================================================================
# MNI -> T1 with the Step 1 ANTs transforms, then T1 -> diffusion with FLIRT by matrix or
# by image header (T1_TO_DWI in 00_config.sh). Nearest neighbour throughout, re-binarized.
# In:  $OUT/<subj>/reg/mni2t1_*, $PROJECT/{anat,dwi,xfm}/<subj>/ (T1 brain, nodif mask, xfm)
# Out: $OUT/<subj>/rois/${TRACT}_{seed,target,atlas}_{t1,diff}.nii.gz (_t1 kept for checks)
# Run: bash 02_warp_rois.sh   (FORCE=1 bash 02_warp_rois.sh redoes finished participants)
# ================================================================================
source "$(dirname "$0")/00_config.sh"
start_log "$0"
read_subjects

# Warp one participant's seed, target and atlas; a missing input prints "!!" and returns.
warp_one() {
  local s=$1
  local d="$OUT/$s/rois"
  local t1="$PROJECT/anat/$s/${s}_T1w_brain.nii.gz"
  local ref="$PROJECT/dwi/$s/nodif_brain_mask.nii.gz"
  local warp="$OUT/$s/reg/mni2t1_1Warp.nii.gz"
  local aff="$OUT/$s/reg/mni2t1_0GenericAffine.mat"
  local mat="$PROJECT/xfm/$s/str2diff.mat"
  local name src in_t1 in_diff

  # Resume: the atlas is written last, so if it exists this participant already finished.
  if [ "$FORCE" != 1 ] && [ -f "$d/${TRACT}_atlas_diff.nii.gz" ]; then
    echo "== $s already warped"
    return
  fi
  if [ ! -f "$warp" ] || [ ! -f "$aff" ] || [ ! -f "$ref" ]; then
    echo "!! $s missing registration outputs or diffusion reference"
    return
  fi
  if [ "$T1_TO_DWI" = matrix ] && [ ! -f "$mat" ]; then
```

```
    echo "!! $s missing $mat"
    return
  fi
  mkdir -p "$d"

  for name in seed target atlas; do
    case $name in
      seed)   src=$SEED_MNI ;;
      target) src=$TARGET_MNI ;;
      atlas)  src=$ATLAS_MNI ;;
    esac
    in_t1="$d/${TRACT}_${name}_t1.nii.gz"
    in_diff="$d/${TRACT}_${name}_diff.nii.gz"

    # MNI -> T1. ANTs applies the -t stack last-listed first: affine, then warp. Do not swap.
    antsApplyTransforms -d 3 -i "$src" -r "$t1" -o "$in_t1" \
      -t "$warp" -t "$aff" -n NearestNeighbor

    # T1 -> diffusion
    if [ "$T1_TO_DWI" = matrix ]; then
      flirt -in "$in_t1" -ref "$ref" -applyxfm -init "$mat" \
        -interp nearestneighbour -out "$in_diff"
    else
      # -usesqform: alignment from the two images' sform/qform headers, no matrix file
      flirt -in "$in_t1" -ref "$ref" -applyxfm -usesqform \
        -interp nearestneighbour -out "$in_diff"
    fi
    # Guarantee a clean 0/1 mask: Step 3 adds and inverts these, Step 5 seeds from them.
    fslmaths "$in_diff" -thr 0.5 -bin "$in_diff"
  done
  echo ">> $s warped"
}

# Catch a bad T1_TO_DWI now rather than after warping everyone the wrong way.
case "$T1_TO_DWI" in
  matrix) echo "== T1 -> diffusion: applying xfm/<subj>/str2diff.mat" ;;
  header) echo "== T1 -> diffusion: resampling by image header (no matrix)" ;;
  *)      echo "!! T1_TO_DWI must be matrix or header"; exit 1 ;;
esac

# --- Warp everyone, MAXJOBS at a time ---
for s in "${SUBJECTS[@]}"; do
  warp_one "$s" &
  throttle "$MAXJOBS"
done
# Not a bare "wait": some bash versions would also wait on the start_log logger (never exits).
wait_for_jobs

# --- Audit: voxel counts and seed-target overlap (overlap must be 0) ---
tmp=$(mktemp -d)
printf "\nSubject\tseed_vox\ttarget_vox\tatlas_vox\tseed_target_overlap\n"
for s in "${SUBJECTS[@]}"; do
  d="$OUT/$s/rois"
  if [ ! -f "$d/${TRACT}_atlas_diff.nii.gz" ]; then
    printf "%s\tMISSING\n" "$s"
    continue
  fi
  # A voxel in both masks would count as "reached" in Step 5 before the streamline has moved.
  fslmaths "$d/${TRACT}_seed_diff.nii.gz" -mul "$d/${TRACT}_target_diff.nii.gz" "$tmp/overlap"
  printf "%s\t%s\t%s\t%s\t%s\n" "$s" \
    "$(nvox "$d/${TRACT}_seed_diff.nii.gz")" \
    "$(nvox "$d/${TRACT}_target_diff.nii.gz")" \
    "$(nvox "$d/${TRACT}_atlas_diff.nii.gz")" \
```

```
    "$(nvox "$tmp/overlap")"
done
rm -rf "$tmp"
```

03_build_corridor_mask.sh

```
#!/bin/bash
# ================================================================================
# Step 3. Build the corridor and its exclusion mask
# ================================================================================
# Dilate the warped atlas, add seed and target, binarize, invert: tckgen's -exclude mask.
# In:  $OUT/<participant>/rois/<TRACT>_{atlas,seed,target}_diff.nii.gz (Step 2)
# Out: same dir, <TRACT>_{atlas_dilated,inclusion_zone,exclusion_mask}.nii.gz
# Run: bash 03_build_corridor_mask.sh   (after Step 2; FORCE=1 redoes all)
# ================================================================================
source "$(dirname "$0")/00_config.sh"
start_log "$0"
read_subjects

# One -dilM pass (3x3x3) per voxel; DILATE_VOX=2 (4 mm) is slack for registration error.
DILATE_ARGS=""
for ((i = 0; i < DILATE_VOX; i++)); do
  DILATE_ARGS="$DILATE_ARGS -dilM"
done

# build_one <ID>: writes the three corridor images; skips if the mask exists unless FORCE=1
build_one() {
  local s=$1
  local d="$OUT/$s/rois"
  if [ "$FORCE" != 1 ] && [ -f "$d/${TRACT}_exclusion_mask.nii.gz" ]; then
    echo "== $s corridor exists"
    return
  fi
  if [ ! -f "$d/${TRACT}_atlas_diff.nii.gz" ]; then
    echo "!! $s missing warped atlas (run Step 2)"
    return
  fi
  # $DILATE_ARGS is unquoted on purpose: it must expand to separate -dilM options
  fslmaths "$d/${TRACT}_atlas_diff.nii.gz" $DILATE_ARGS "$d/${TRACT}_atlas_dilated.nii.gz"
  # Seed and target go in before the inversion, or the corridor ends short of them
  fslmaths "$d/${TRACT}_atlas_dilated.nii.gz" \
    -add "$d/${TRACT}_seed_diff.nii.gz" \
    -add "$d/${TRACT}_target_diff.nii.gz" \
    -bin "$d/${TRACT}_inclusion_zone.nii.gz"
  fslmaths "$d/${TRACT}_inclusion_zone.nii.gz" -binv "$d/${TRACT}_exclusion_mask.nii.gz"
  echo ">> $s corridor: $(nvox "$d/${TRACT}_inclusion_zone.nii.gz") voxels"
}

for s in "${SUBJECTS[@]}"; do
  build_one "$s" &
  throttle "$MAXJOBS"
done
# Not a bare wait: see wait_for_jobs in 00_config.sh
wait_for_jobs

# Audit: seed_excluded and target_excluded must be 0 for every participant before Step 4
tmp=$(mktemp -d)
printf "\nSubject\tseed_excluded\ttarget_excluded\tcorridor_vox\n"
for s in "${SUBJECTS[@]}"; do
  d="$OUT/$s/rois"
  excl="$d/${TRACT}_exclusion_mask.nii.gz"
```

```
  if [ ! -f "$excl" ]; then
    printf "%s\tMISSING\n" "$s"
    continue
  fi
  fslmaths "$excl" -mul "$d/${TRACT}_seed_diff.nii.gz" "$tmp/seed"
  fslmaths "$excl" -mul "$d/${TRACT}_target_diff.nii.gz" "$tmp/target"
  printf "%s\t%s\t%s\t%s\n" "$s" \
    "$(nvox "$tmp/seed")" \
    "$(nvox "$tmp/target")" \
    "$(nvox "$d/${TRACT}_inclusion_zone.nii.gz")"
done
rm -rf "$tmp"
```

04_tune_cutoff.sh

```
#!/bin/bash
# ================================================================================
# Step 4. Pilot sweep of the FOD amplitude cutoff
# ================================================================================
# Runs reduced-budget tractography at several cutoffs in a few pilot participants.
# Needs $PROJECT/dwi/<s>/wm_fod_norm.mif (Step 0b) and, from Steps 2 and 3, $OUT/<s>/rois/
#   <TRACT>_seed_diff.nii.gz, <TRACT>_target_diff.nii.gz, <TRACT>_exclusion_mask.nii.gz
# Writes $OUT/<s>/tckgen/<TRACT>/<TRACT>_pilot_<cutoff>.tck and $OUT/logs/04_tune_cutoff.sh.log
# Usage: bash 04_tune_cutoff.sh [participants] [cutoffs]; defaults: first 5, 0.1 0.08 0.06 0.01
source "$(dirname "$0")/00_config.sh"
start_log "$0"
read_subjects

PILOT=${1:-${SUBJECTS[*]:0:5}}
# Highest cutoff first: 04b_compare_cutoffs.py uses the first one as its Dice reference.
CUTOFFS=${2:-"0.1 0.08 0.06 0.01"}
# Reduced budgets so the sweep is cheap; the seed cap stops a failing cutoff in bounded time.
PILOT_SELECT=1000
PILOT_SEEDS=5000000

printf "%-12s %-8s %-12s %-12s %-12s\n" Subject Cutoff Selected Generated MeanLen_mm
for s in $PILOT; do
  rois="$OUT/$s/rois"
  tdir="$OUT/$s/tckgen/$TRACT"
  if [ ! -f "$rois/${TRACT}_exclusion_mask.nii.gz" ]; then
    echo "!! $s missing corridor (run Steps 2 and 3)"
    continue
  fi
  mkdir -p "$tdir"
  for c in $CUTOFFS; do
    tck="$tdir/${TRACT}_pilot_${c}.tck"
    # -seed_unidirectional: track one way from the seed.  -stop: end the streamline once it
    # enters the target.  -exclude: the inverted corridor, so a streamline dies on leaving it.
    tckgen "$PROJECT/dwi/$s/wm_fod_norm.mif" "$tck" \
      -algorithm iFOD2 \
      -seed_image "$rois/${TRACT}_seed_diff.nii.gz" -seed_unidirectional \
      -include "$rois/${TRACT}_target_diff.nii.gz" \
      -exclude "$rois/${TRACT}_exclusion_mask.nii.gz" \
      -select "$PILOT_SELECT" -seeds "$PILOT_SEEDS" -cutoff "$c" \
      -minlength "$MINLEN" -maxlength "$MAXLEN" -stop \
      -nthreads "$NTHREADS" -force -quiet
    count=$(tckinfo "$tck" | awk '$1 == "count:" {print $2}')
    generated=$(tckinfo "$tck" | awk '$1 == "total_count:" {print $2}')
    if [ "${count:-0}" -gt 0 ]; then
      meanlen=$(tckstats "$tck" -output mean -quiet | awk '{print $1}')
    else
```

```
      meanlen=NA
    fi
    printf "%-12s %-8s %-12s %-12s %-12s\n" "$s" "$c" "$count" "$generated" "$meanlen"
  done
done
echo "Reaching the streamline target in every pilot participant is necessary but not"
echo "sufficient: compare the reconstructions with 04b_compare_cutoffs.py before choosing."
```

04b_compare_cutoffs.py

```
#!/usr/bin/env python3
"""Step 4b. Compare pilot tractograms from 04_tune_cutoff.sh to pick CUTOFF for 00_config.sh.

Inputs:  $OUT/<subj>/tckgen/<TRACT>/<TRACT>_pilot_<cutoff>.tck
         $PROJECT/dwi/<subj>/nodif_brain_mask.nii.gz and mean_b0.nii.gz (optional)
Outputs: $OUT/qc/<TRACT>_cutoff_pilot/<subj>_cutoff_compare.png
         $OUT/qc/<TRACT>_cutoff_pilot/cutoff_summary.csv and cutoff_summary.png
Run:     source 00_config.sh
         python 04b_compare_cutoffs.py "sub-01 sub-02 sub-03" "0.1 0.08 0.06 0.01"
         Both arguments optional; list the most conservative cutoff first (Dice reference).
"""
import os
import subprocess
import sys
from pathlib import Path

# File-only backend: this usually runs on a cluster node with no display.
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import nibabel as nib
import numpy as np
import pandas as pd


# Read one variable exported by 00_config.sh; fail clearly if it was not sourced.
def env(name):
    if name not in os.environ:
        sys.exit(f"{name} is not set: run `source 00_config.sh` first")
    return os.environ[name]


PROJECT = Path(env("PROJECT"))
OUT = Path(env("OUT"))
TRACT = env("TRACT")

if len(sys.argv) > 1:
    PILOT = sys.argv[1].split()
else:
    PILOT = Path(env("SUBJECTS_FILE")).read_text().split()[:5]
# Cutoffs stay strings: "0.1" has to match the file name 04_tune_cutoff.sh wrote.
CUTOFFS = sys.argv[2].split() if len(sys.argv) > 2 else ["0.1", "0.08", "0.06", "0.01"]
REFERENCE = CUTOFFS[0]          # most conservative cutoff: reference for Dice overlap

QC = OUT / "qc" / f"{TRACT}_cutoff_pilot"
QC.mkdir(parents=True, exist_ok=True)


# Run a command and return its stdout; check=True so a damaged .tck stops the script.
def run(cmd):
    return subprocess.run(cmd, capture_output=True, text=True, check=True).stdout
```

```
def tck_summary(tck):
    """Streamlines selected, generated (selected plus rejected), mean and SD of length (mm)."""
    info = {}
    for line in run(["tckinfo", str(tck)]).splitlines():
        key, _, value = line.strip().partition(":")
        info[key] = value.strip()
    # "count" is what tckgen kept; "total_count" is everything it generated (one per seed).
    count = int(info.get("count", 0))
    generated = int(info.get("total_count", 0))
    if count == 0:
        return count, generated, np.nan, np.nan
    # -output mean/std: print that one number only, so the whole stdout casts to float.
    mean = float(run(["tckstats", str(tck), "-output", "mean", "-quiet"]))
    sd = float(run(["tckstats", str(tck), "-output", "std", "-quiet"]))
    return count, generated, mean, sd


def density_mask(tck, template, scratch):
    """Binary mask of the voxels visited by at least one streamline."""
    # tckmap: tract-density image; -template puts it on the brain mask's voxel grid.
    run(["tckmap", str(tck), str(scratch), "-template", str(template), "-force", "-quiet"])
    # Binarise at zero so a sparse stray branch counts as much as the core.
    mask = nib.load(str(scratch)).get_fdata() > 0
    scratch.unlink()
    return mask


# Dice overlap of two boolean masks; NaN when both are empty.
def dice(a, b):
    total = a.sum() + b.sum()
    return 2 * np.logical_and(a, b).sum() / total if total else np.nan


# One slice: grey background with the tract mask drawn on top.
def overlay(ax, background, mask, title):
    ax.imshow(np.rot90(background), cmap="gray")
    ax.imshow(np.rot90(np.ma.masked_where(~mask, mask.astype(float))),
              cmap="autumn", alpha=0.8, vmin=0, vmax=1)
    ax.set_title(title, fontsize=10)
    ax.axis("off")


rows = []
for s in PILOT:
    # The mean b0 is optional in the project layout, so fall back to the brain mask.
    template = PROJECT / "dwi" / s / "nodif_brain_mask.nii.gz"
    b0 = PROJECT / "dwi" / s / "mean_b0.nii.gz"
    background = nib.load(str(b0 if b0.exists() else template)).get_fdata()

    masks = {}
    for c in CUTOFFS:
        # A missing tractogram is skipped, not fatal, so a partial sweep still plots.
        tck = OUT / s / "tckgen" / TRACT / f"{TRACT}_pilot_{c}.tck"
        if not tck.exists():
            print(f"[{s}] cutoff {c}: no tractogram (run 04_tune_cutoff.sh)")
            continue
        count, generated, mean, sd = tck_summary(tck)
        masks[c] = density_mask(tck, template, QC / f"_{s}_{c}_tdi.nii.gz")
        rows.append({
            "subject": s, "cutoff": c, "selected": count, "generated": generated,
            "voxels": int(masks[c].sum()), "mean_len_mm": mean, "sd_len_mm": sd,
```

```
        })
    if not masks:
        continue

    # Dice against the reference; rows holds every participant so far, hence the ID filter.
    for row in rows:
        if row["subject"] == s and REFERENCE in masks:
            row["dice_vs_reference"] = round(dice(masks[REFERENCE], masks[row["cutoff"]]), 3)

    # One slice for every column: the fullest axial and coronal slice of the union.
    union = np.logical_or.reduce(list(masks.values()))
    k_axial = int(np.argmax(union.sum(axis=(0, 1))))
    k_coronal = int(np.argmax(union.sum(axis=(0, 2))))

    fig, axes = plt.subplots(2, len(masks), figsize=(3.2 * len(masks), 6.4), squeeze=False)
    for j, (c, mask) in enumerate(masks.items()):
        overlay(axes[0][j], background[:, :, k_axial], mask[:, :, k_axial],
                f"Axial, cutoff {c}")
        overlay(axes[1][j], background[:, k_coronal, :], mask[:, k_coronal, :],
                f"Coronal, cutoff {c}")
    # "uncleaned" is deliberate: judge the path, not the width; Step 6 trims the excess.
    fig.suptitle(f"{s}  {TRACT}  pilot cutoffs (uncleaned)", fontsize=11)
    fig.tight_layout()
    fig.savefig(QC / f"{s}_cutoff_compare.png", dpi=110, bbox_inches="tight")
    plt.close(fig)
    print(f"[{s}] wrote {s}_cutoff_compare.png")

if not rows:
    sys.exit("no pilot tractograms found")

summary = pd.DataFrame(rows)
summary.to_csv(QC / "cutoff_summary.csv", index=False)

# Mean across pilot participants, one bar per cutoff; Dice is in the CSV only.
panels = [("selected", "Streamlines selected"), ("generated", "Streamlines generated"),
          ("voxels", "Occupied voxels"), ("sd_len_mm", "SD of length (mm)")]
# sort=False keeps command-line order; sorted as strings, "0.1" would land after "0.08".
means = summary.groupby("cutoff", sort=False)[[k for k, _ in panels]].mean()
fig, axes = plt.subplots(1, len(panels), figsize=(15, 3.6))
for ax, (key, title) in zip(axes, panels):
    ax.bar(means.index.astype(str), means[key], color="#3A6B8C")
    ax.set_title(title)
    ax.set_xlabel("FOD amplitude cutoff")
fig.tight_layout()
fig.savefig(QC / "cutoff_summary.png", dpi=110)
plt.close(fig)

print(summary.to_string(index=False))
print(f"\nsummary -> {QC / 'cutoff_summary.csv'} and cutoff_summary.png")
```

05_tractography.sh

```
#!/bin/bash
# ==============================================================================
# Step 5. Corridor-constrained tractography for every participant
# ==============================================================================
# Tracks from seed to target and drops any streamline that leaves the Step 3 corridor.
# In:  $PROJECT/dwi/<s>/wm_fod_norm.mif; $OUT/<s>/rois/<TRACT>_{seed,target}_diff.nii.gz,
#      $OUT/<s>/rois/<TRACT>_exclusion_mask.nii.gz
# Out: $OUT/<s>/tckgen/<TRACT>/<TRACT>_<CUTOFF>.tck; $OUT/qc/<TRACT>_tckgen_summary.csv
# Run: bash 05_tractography.sh  (FORCE=1 redoes everyone; about 2 hr for 57 participants)
```

```
# ============================================================================
source "$(dirname "$0")/00_config.sh"
start_log "$0"
read_subjects

# --- Tractography, one participant at a time; tckgen itself uses NTHREADS cores ---
for s in "${SUBJECTS[@]}"; do
  rois="$OUT/$s/rois"
  tdir="$OUT/$s/tckgen/$TRACT"
  # The cutoff is in the file name, so a new cutoff is never skipped as "exists".
  tck="$tdir/${TRACT}_${CUTOFF}.tck"
  if [ ! -f "$PROJECT/dwi/$s/wm_fod_norm.mif" ]; then
    echo "!! $s missing FOD image (run 00b)"
    continue
  fi
  # The exclusion mask is the last thing Step 3 writes, so it stands in for seed and target.
  if [ ! -f "$rois/${TRACT}_exclusion_mask.nii.gz" ]; then
    echo "!! $s missing corridor (run Steps 2 and 3)"
    continue
  fi
  if [ "$FORCE" != 1 ] && [ -f "$tck" ]; then
    echo "== $s tractogram exists"
    continue
  fi
  mkdir -p "$tdir"
  # tckgen creates the output at start-up; write to a temp name so an interrupted run is redone.
  partial="$tdir/${TRACT}_${CUTOFF}.partial.tck"
  # -seed_unidirectional: track one way from the seed.  -stop: end on entering the target.
  # -exclude is the inverted corridor, which is what makes a cutoff this low safe.
  if tckgen "$PROJECT/dwi/$s/wm_fod_norm.mif" "$partial" \
    -algorithm iFOD2 \
    -seed_image "$rois/${TRACT}_seed_diff.nii.gz" -seed_unidirectional \
    -include "$rois/${TRACT}_target_diff.nii.gz" \
    -exclude "$rois/${TRACT}_exclusion_mask.nii.gz" \
    -select "$SELECT" -seeds "$SEEDS" -cutoff "$CUTOFF" \
    -minlength "$MINLEN" -maxlength "$MAXLEN" -stop \
    -nthreads "$NTHREADS" -force; then
    mv -f "$partial" "$tck"
    echo ">> $s finished"
  else
    rm -f "$partial"
    echo "!! $s tckgen failed or was interrupted"
  fi
done

# --- Summary table, rebuilt from everything on disk, not just this run ---
mkdir -p "$OUT/qc"
stats="$OUT/qc/${TRACT}_tckgen_summary.csv"
echo "Subject,Selected,MeanLength_mm,Generated" > "$stats"
reached=0
total=0
for s in "${SUBJECTS[@]}"; do
  tck="$OUT/$s/tckgen/$TRACT/${TRACT}_${CUTOFF}.tck"
  total=$((total + 1))
  if [ ! -f "$tck" ]; then
    echo "MISSING: $s"
    continue
  fi
  # count = streamlines kept; total_count = streamlines generated, including rejected ones.
  count=$(tckinfo "$tck" | awk '$1 == "count:" {print $2}')
  generated=$(tckinfo "$tck" | awk '$1 == "total_count:" {print $2}')
  if [ "${count:-0}" -gt 0 ]; then
    meanlen=$(tckstats "$tck" -output mean -quiet | awk '{print $1}')
```

```
  else
    meanlen=NA
  fi
  echo "$s,$count,$meanlen,$generated" >> "$stats"
  # Anyone short is still kept (count is not connectivity); SHORT flags whom to check in Step 7.
  if [ "${count:-0}" -ge "$SELECT" ]; then
    reached=$((reached + 1))
  else
    echo "SHORT: $s ($count selected of $generated generated)"
  fi
done
echo "reached $SELECT streamlines: $reached / $total"
echo "summary -> $stats"
```

06_clean_bundles.py

```
#!/usr/bin/env python3
"""Step 6. Clean each participant's raw corridor tractogram with pyAFQ, then rebuild the
covariate table (streamline count and mean length of every cleaned bundle).
Inputs:  $OUT/<subj>/tckgen/<TRACT>/<TRACT>_<CUTOFF>.tck      raw tractogram from Step 5
         $PROJECT/dwi/<subj>/nodif_brain_mask.nii.gz           reference grid for the .tck
         $SUBJECTS_FILE                                        one participant ID per line
Outputs: $OUT/<subj>/tckgen/<TRACT>/<TRACT>_<CUTOFF>_cleaned.tck
         $OUT/nodewise/<TRACT>_tract_stats.csv
Run:     source 00_config.sh && python 06_clean_bundles.py
         FORCE=1 in the shell redoes participants whose cleaned bundle already exists.
"""
import inspect   # used once, to ask the installed clean_bundle whether it takes core_only
import os
import sys
from pathlib import Path

import pandas as pd
# clean_bundle moved between pyAFQ releases; try the current location, then the old one.
try:
    from AFQ.recognition.cleaning import clean_bundle
except ImportError:                       # pyAFQ 1.3.2 and earlier
    from AFQ.segmentation import clean_bundle
from dipy.io.stateful_tractogram import StatefulTractogram
from dipy.io.streamline import load_tractogram, save_tractogram
from dipy.segment.clustering import QuickBundles
from dipy.segment.featurespeed import ResampleFeature
from dipy.segment.metricspeed import AveragePointwiseEuclideanMetric
from dipy.tracking.streamline import length   # per-streamline length, for the table


# One variable exported by 00_config.sh, or stop with a pointer to source it.
def env(name):
    if name not in os.environ:
        sys.exit(f"{name} is not set: run `source 00_config.sh` first")
    return os.environ[name]


# --- Settings from 00_config.sh ---
PROJECT = Path(env("PROJECT"))
OUT = Path(env("OUT"))
TRACT = env("TRACT")
CUTOFF = env("CUTOFF")   # stays a string; it is only ever used in file names
FORCE = os.environ.get("FORCE", "0") == "1"
SUBJECTS = Path(env("SUBJECTS_FILE")).read_text().split()
```

```
# --- Cleaning thresholds (Table 1 on the Step 6 page; not in 00_config.sh) ---
N_POINTS = 100          # resampling for the Mahalanobis distance
CLEAN_ROUNDS = 5        # maximum cleaning iterations
DISTANCE_SD = 3         # Mahalanobis distance threshold, SD
LENGTH_SD = 2           # length threshold, SD above the mean
QB_SPLIT = False        # True: split into two bundles before cleaning
QB_THRESHOLD_MM = 5.0   # QuickBundles distance threshold


CLEAN_ARGS = dict(n_points=N_POINTS, clean_rounds=CLEAN_ROUNDS, stat="mean",
                  distance_threshold=DISTANCE_SD, length_threshold=LENGTH_SD)
# pyAFQ 3+ measures the distance over the middle 60% of each streamline by default;
# core_only=0 restores the full-length computation; pass it only if this pyAFQ knows it.
if "core_only" in inspect.signature(clean_bundle).parameters:
    CLEAN_ARGS["core_only"] = 0


# Clean one tractogram.  We rebuild the result from the kept indices rather than use the
# copy pyAFQ returns, so the output stays in the input's space under every pyAFQ version.
def clean(sft):
    _, keep = clean_bundle(sft, return_idx=True, **CLEAN_ARGS)
    return StatefulTractogram.from_sft(sft.streamlines[keep], sft)


# Load a .tck on the participant's brain-mask grid (a .tck carries no image grid of its own).
# bbox_valid_check=False: do not refuse a streamline that grazes the volume edge.
def load(tck, reference):
    return load_tractogram(str(tck), str(reference), bbox_valid_check=False)


# --- Pass 1: clean every participant that still needs it ---
for s in SUBJECTS:
    tdir = OUT / s / "tckgen" / TRACT
    raw = tdir / f"{TRACT}_{CUTOFF}.tck"
    out = tdir / f"{TRACT}_{CUTOFF}_cleaned.tck"
    reference = PROJECT / "dwi" / s / "nodif_brain_mask.nii.gz"

    # Step 5 renames its output at the end, so a raw file at this path is always complete.
    if not raw.exists():
        print(f"[{s}] SKIP: no tractogram from Step 5")
        continue
    if out.exists() and not FORCE:
        print(f"[{s}] cleaned bundle exists")
        continue
    sft = load(raw, reference)
    n_raw = len(sft.streamlines)
    if n_raw == 0:
        print(f"[{s}] SKIP: 0 streamlines")
        continue

    if not QB_SPLIT:
        cleaned = clean(sft)
        save_tractogram(cleaned, str(out), bbox_valid_check=False)
        n_clean = len(cleaned.streamlines)
        print(f"[{s}] {n_raw} -> {n_clean} streamlines ({100 * n_clean / n_raw:.0f}% retained)")
        continue

    # Two-bundle path: cluster with QuickBundles and clean the two largest clusters separately.
    metric = AveragePointwiseEuclideanMetric(ResampleFeature(nb_points=N_POINTS))
    clusters = QuickBundles(threshold=QB_THRESHOLD_MM, metric=metric).cluster(sft.streamlines)
    largest = sorted(clusters, key=len, reverse=True)[:2]
    for i, cluster in enumerate(largest, start=1):
        part = StatefulTractogram.from_sft(sft.streamlines[cluster.indices], sft)
```

```
        cleaned = clean(part)
        name = f"{TRACT}_{CUTOFF}_cluster{i}_cleaned.tck"
        save_tractogram(cleaned, str(tdir / name), bbox_valid_check=False)
        print(f"[{s}] cluster {i}: {len(cluster)} -> {len(cleaned.streamlines)} streamlines")
    # Nothing goes to _cleaned.tck here: copy the right cluster to it by hand, then rerun.
    print(f"[{s}] inspect both clusters; copy the correct one to {out.name}")

# --- Pass 2: covariate table from the cleaned bundles (rebuilt every run) ---
# A separate loop so the table also covers participants skipped above or copied in by hand.
rows = []
for s in SUBJECTS:
    tdir = OUT / s / "tckgen" / TRACT
    raw = tdir / f"{TRACT}_{CUTOFF}.tck"
    out = tdir / f"{TRACT}_{CUTOFF}_cleaned.tck"
    if not out.exists():
        continue
    reference = PROJECT / "dwi" / s / "nodif_brain_mask.nii.gz"
    lengths = length(load(out, reference).streamlines)
    n_raw = len(load(raw, reference).streamlines) if raw.exists() else float("nan")
    # Streamline_count and Mean_length_mm feed the Step 9 models; the other two are QC only.
    rows.append({
        "Subject": s,
        "Streamline_count": len(lengths),
        "Mean_length_mm": round(float(lengths.mean()), 3) if len(lengths) else float("nan"),
        "Count_uncleaned": n_raw,
        "Retention_pct": round(100 * len(lengths) / n_raw, 1) if n_raw else float("nan"),
    })

table = OUT / "nodewise" / f"{TRACT}_tract_stats.csv"
table.parent.mkdir(parents=True, exist_ok=True)
pd.DataFrame(rows).to_csv(table, index=False)
print(f"\n{len(rows)} of {len(SUBJECTS)} participants have a cleaned bundle")
print(f"covariate table -> {table}")
```

07_visual_qc.py

```
#!/usr/bin/env python3
"""Step 7. Tract-density overlay of each cleaned bundle on the b0, with gross-failure flags.

Inputs:  $OUT/<subj>/tckgen/<TRACT>/<TRACT>_<CUTOFF>_cleaned.tck  (Step 6)
         $PROJECT/dwi/<subj>/nodif_brain_mask.nii.gz and mean_b0.nii.gz (optional)
Outputs: $OUT/qc/<TRACT>/<subj>_<TRACT>_qc.png and $OUT/qc/<TRACT>/qc_flags.csv
Run:     source 00_config.sh && python 07_visual_qc.py   (tckmap must be on PATH)
"""
import os
import subprocess
import sys
from pathlib import Path

# Agg backend, so the script also runs on a cluster node or over ssh with no display.
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import nibabel as nib
import numpy as np
import pandas as pd


# Read one variable exported by 00_config.sh, or stop with a message saying what to do.
def env(name):
    if name not in os.environ:
```

```
        sys.exit(f"{name} is not set: run `source 00_config.sh` first")
    return os.environ[name]


# CUTOFF stays a string because it only builds the file name Steps 5 and 6 used.
PROJECT = Path(env("PROJECT"))
OUT = Path(env("OUT"))
TRACT = env("TRACT")
CUTOFF = env("CUTOFF")
FORCE = os.environ.get("FORCE", "0") == "1"
SUBJECTS = Path(env("SUBJECTS_FILE")).read_text().split()

# Flag thresholds (Table 1 on the Step 7 page). They catch gross failures only and depend
# on voxel size and DILATE_VOX, so revisit them if your corridor differs from the example.
MIN_VOXELS = 50         # fewer occupied voxels: LOW_COVERAGE
MAX_VOXELS = 5000       # more occupied voxels: HIGH_COVERAGE (bundle leaves the pathway)
MIN_PEAK_DENSITY = 5    # peak streamlines per voxel below this: LOW_DENSITY

QC = OUT / "qc" / TRACT
QC.mkdir(parents=True, exist_ok=True)


# Turn the two numbers measured from the TDI into one flag; first hit wins, "" means pass.
def flag_for(n_voxels, peak):
    if n_voxels < MIN_VOXELS:
        return "LOW_COVERAGE"
    if n_voxels > MAX_VOXELS:
        return "HIGH_COVERAGE"
    if peak < MIN_PEAK_DENSITY:
        return "LOW_DENSITY"
    return ""


# Draw one panel, the b0 slice in grey with the density slice on top. np.rot90 puts
# anterior (axial) or superior (coronal) at the top; left-right is storage order, not L/R.
def overlay(ax, background, density, title):
    ax.imshow(np.rot90(background), cmap="gray")
    ax.imshow(np.rot90(np.ma.masked_where(density <= 0, density)), cmap="hot", alpha=0.7)
    ax.set_title(title)
    ax.axis("off")


# Every participant gets a row, so qc_flags.csv always lists the whole sample.
rows = []
for s in SUBJECTS:
    # The brain mask doubles as the tckmap template so the TDI lands on the diffusion grid.
    tck = OUT / s / "tckgen" / TRACT / f"{TRACT}_{CUTOFF}_cleaned.tck"
    template = PROJECT / "dwi" / s / "nodif_brain_mask.nii.gz"
    b0 = PROJECT / "dwi" / s / "mean_b0.nii.gz"
    png = QC / f"{s}_{TRACT}_qc.png"

    if not tck.exists():
        rows.append({"Subject": s, "Voxels": 0, "Peak_density": 0, "Flag": "MISSING"})
        print(f"[{s}] MISSING cleaned bundle")
        continue

    # tckmap's default output is the tract-density image (streamlines per voxel); the file
    # is scratch and is deleted once the array is in memory.
    tdi = QC / f"_{s}_tdi.nii.gz"
    subprocess.run(["tckmap", str(tck), str(tdi), "-template", str(template),
                    "-force", "-quiet"], check=True)
    density = nib.load(str(tdi)).get_fdata()
    tdi.unlink()
```

```
    n_voxels = int((density > 0).sum())
    peak = float(density.max())
    flag = flag_for(n_voxels, peak)
    rows.append({"Subject": s, "Voxels": n_voxels, "Peak_density": peak, "Flag": flag})
    print(f"[{s}] {n_voxels} voxels, peak {peak:.0f}  {flag}")

    # Flags were recomputed above; the picture is only redrawn when missing or FORCE=1.
    if png.exists() and not FORCE:
        continue
    background = nib.load(str(b0 if b0.exists() else template)).get_fdata()
    # Busiest axial and coronal slices; assumes x left-right, y front-back, z bottom-top.
    k_axial = int(np.argmax(density.sum(axis=(0, 1))))
    k_coronal = int(np.argmax(density.sum(axis=(0, 2))))
    fig, axes = plt.subplots(1, 2, figsize=(10, 5))
    overlay(axes[0], background[:, :, k_axial], density[:, :, k_axial], "Axial")
    overlay(axes[1], background[:, k_coronal, :], density[:, k_coronal, :], "Coronal")
    fig.suptitle(f"{s}  {TRACT}  cleaned" + (f"  [{flag}]" if flag else ""), fontsize=11)
    fig.tight_layout()
    fig.savefig(png, dpi=100, bbox_inches="tight")
    plt.close(fig)

# Anything with a non-empty Flag counts as flagged, so MISSING participants show up too.
flags = pd.DataFrame(rows)
flags.to_csv(QC / "qc_flags.csv", index=False)
flagged = flags[flags["Flag"] != ""]
print(f"\n{len(flagged)} of {len(flags)} participants flagged")
if len(flagged):
    print(flagged.to_string(index=False))
print(f"flags -> {QC / 'qc_flags.csv'}")
```

08_node_profiles.py

```
#!/usr/bin/env python3
"""Step 8. Sample each scalar map at 100 nodes along the cleaned bundle (AFQ tract profile).

Inputs, per participant:
    $OUT/<s>/tckgen/<TRACT>/<TRACT>_<CUTOFF>_cleaned.tck   cleaned bundle (Step 6)
    $OUT/<s>/rois/<TRACT>_seed_diff.nii.gz                 seed region (Step 2); node 0 end
    $PROJECT/dwi/<s>/fa.nii.gz                             FA map
    $PROJECT/noddi/<s>/fit_*.nii.gz                        NODDI maps (Step 8a, optional)
Output: $OUT/nodewise/<TRACT>_nodewise_all_subjects.csv (long format; Step 8b pivots it)
Run:    source 00_config.sh && python 08_node_profiles.py
"""
import os
import sys
from pathlib import Path

import nibabel as nib
import numpy as np
import pandas as pd
from dipy.io.image import load_nifti
from dipy.io.streamline import load_tractogram
from dipy.segment.clustering import QuickBundles
from dipy.segment.featurespeed import ResampleFeature
from dipy.segment.metricspeed import AveragePointwiseEuclideanMetric
from dipy.stats.analysis import afq_profile, gaussian_weights
from dipy.tracking.streamline import Streamlines, orient_by_streamline


# Read one variable exported by 00_config.sh, or stop with a hint.
```

```
def env(name):
    if name not in os.environ:
        sys.exit(f"{name} is not set: run `source 00_config.sh` first")
    return os.environ[name]


# TRACT and CUTOFF go into file names, so they must match what Steps 5 and 6 ran with.
PROJECT = Path(env("PROJECT"))
OUT = Path(env("OUT"))
TRACT = env("TRACT")
CUTOFF = env("CUTOFF")
# bash cannot export an array, so the ID list is re-read from the file.
SUBJECTS = Path(env("SUBJECTS_FILE")).read_text().split()

# 100 nodes is the AFQ convention; Step 9a's quartiles and Step 9c's blocks of 100 assume it.
N_NODES = 100
# A real bundle has hundreds of streamlines; a handful means tracking or cleaning failed.
MIN_STREAMLINES = 5
METRICS = {
    "FA": "dwi/{s}/fa.nii.gz",
    "NDI": "noddi/{s}/fit_NDI_modulated.nii.gz",
    "ODI": "noddi/{s}/fit_ODI_modulated.nii.gz",
    "FWF": "noddi/{s}/fit_FWF.nii.gz",
}


unavailable = [m for m, template in METRICS.items()
               if not any((PROJECT / template.format(s=s)).exists() for s in SUBJECTS)]
for m in unavailable:
    print(f"metric {m}: no image found for any participant; not profiled")
    del METRICS[m]
if not METRICS:
    sys.exit("no scalar images found")


def seed_centre_mm(s):
    """Centre of mass of the warped seed region, in scanner (mm) coordinates."""
    img = nib.load(str(OUT / s / "rois" / f"{TRACT}_seed_diff.nii.gz"))
    voxels = np.argwhere(img.get_fdata() > 0)
    return nib.affines.apply_affine(img.affine, voxels.mean(axis=0))


def orient_seed_to_target(streamlines, seed_mm):
    """Orient all streamlines alike, with node 0 at the seed end."""
    # threshold=np.inf: one cluster, so centroids[0] is the mean streamline of the whole bundle.
    metric = AveragePointwiseEuclideanMetric(ResampleFeature(nb_points=N_NODES))
    centroid = QuickBundles(threshold=np.inf, metric=metric).cluster(streamlines).centroids[0]
    # The centroid may run either way; flip it so node 0 is the seed end in every participant.
    starts_at_target = (np.linalg.norm(centroid[0] - seed_mm)
                        > np.linalg.norm(centroid[-1] - seed_mm))
    if starts_at_target:
        centroid = centroid[::-1]
    return Streamlines(orient_by_streamline(streamlines, centroid))


# Sample one scalar map along the oriented bundle; one value per node.
def profile(image, streamlines, weights):
    data, affine = load_nifti(str(image))
    return afq_profile(data, streamlines, affine, n_points=N_NODES, weights=weights)


# --- Main loop, one participant at a time ---
rows = []
```

```
for s in SUBJECTS:
    tck = OUT / s / "tckgen" / TRACT / f"{TRACT}_{CUTOFF}_cleaned.tck"
    maps = {m: PROJECT / template.format(s=s) for m, template in METRICS.items()}
    missing = [str(p) for p in [tck, *maps.values()] if not p.exists()]
    if missing:
        print(f"[{s}] SKIP: missing {missing[0]}")
        continue

    # Any map can be the grid; bbox_valid_check=False accepts points outside it, as in Step 6.
    reference = next(iter(maps.values()))
    streamlines = load_tractogram(str(tck), str(reference), bbox_valid_check=False).streamlines
    if len(streamlines) < MIN_STREAMLINES:
        print(f"[{s}] SKIP: {len(streamlines)} streamlines")
        continue

    # Orient first; the weights are per node, so node i must line up across streamlines.
    # Weights depend on geometry only, so one matrix serves every metric.
    streamlines = orient_seed_to_target(streamlines, seed_centre_mm(s))
    weights = gaussian_weights(streamlines, n_points=N_NODES)
    profiles = {m: profile(p, streamlines, weights) for m, p in maps.items()}
    for node in range(N_NODES):
        rows.append({"Subject": s, "Tract": TRACT, "Node": node,
                     **{m: float(profiles[m][node]) for m in METRICS}})
    print(f"[{s}] {len(streamlines)} streamlines, {N_NODES} nodes, {len(METRICS)} metrics")

out = OUT / "nodewise" / f"{TRACT}_nodewise_all_subjects.csv"
out.parent.mkdir(parents=True, exist_ok=True)
pd.DataFrame(rows).to_csv(out, index=False)
print(f"DONE -> {out}  ({len(rows) // N_NODES} participants)")
```

08a_noddi_fit.py

```
#!/usr/bin/env python3
"""Step 8a. Fit NODDI with AMICO so Step 8 can profile NDI, ODI and FWF along the bundle.

Inputs:  $PROJECT/dwi/<subj>/{data.nii.gz, bvals, bvecs, nodif_brain_mask.nii.gz}
Outputs: $PROJECT/noddi/<subj>/fit_{NDI,ODI,FWF,NDI_modulated,ODI_modulated,RMSE}.nii.gz
         (noddi_gm/ instead when NODDI_DPAR is set); work/<subj>/ is scratch, safe to delete
Run:     source 00_config.sh && python 08a_noddi_fit.py [sub-01 sub-02 ...]
"""
import os
import sys
from pathlib import Path

N_THREADS = os.environ.get("NODDI_NTHREADS", "4")
# numpy's BLAS reads these at import time, and importing amico imports numpy.
os.environ["OPENBLAS_NUM_THREADS"] = N_THREADS     # must be set before amico is imported
os.environ["OMP_NUM_THREADS"] = N_THREADS

import amico


# Read a variable exported by 00_config.sh, or stop with a message naming it.
def env(name):
    if name not in os.environ:
        sys.exit(f"{name} is not set: run `source 00_config.sh` first")
    return os.environ[name]


PROJECT = Path(env("PROJECT"))
FORCE = os.environ.get("FORCE", "0") == "1"
```

```
SUBJECTS = sys.argv[1:] or Path(env("SUBJECTS_FILE")).read_text().split()

# dPar is fixed, not fitted, so it must suit the tissue: 1.7e-3 white matter, 1.1e-3 gray.
D_PAR = os.environ.get("NODDI_DPAR")     # mm2/s; unset = AMICO default for white matter
D_ISO = 3.0e-3                           # isotropic (free water) diffusivity, mm2/s
B0_THRESHOLD = 100                       # volumes with b below this are treated as b = 0
B_STEP = 200                             # b-values are rounded to the nearest multiple

STUDY = PROJECT / ("noddi_gm" if D_PAR else "noddi")
STUDY.mkdir(parents=True, exist_ok=True)
# One-off: caches AMICO's rotation matrices in ~/.dipy, so only the first call is slow.
amico.setup()

for s in SUBJECTS:
    dwi_dir = PROJECT / "dwi" / s
    dwi = dwi_dir / "data.nii.gz"
    bvals = dwi_dir / "bvals"
    bvecs = dwi_dir / "bvecs"
    mask = dwi_dir / "nodif_brain_mask.nii.gz"
    out = STUDY / s

    if (out / "fit_NDI_modulated.nii.gz").exists() and not FORCE:
        print(f"[{s}] NODDI maps exist")
        continue
    missing = [p.name for p in (dwi, bvals, bvecs, mask) if not p.exists()]
    if missing:
        print(f"[{s}] SKIP: missing {', '.join(missing)}")
        continue
    # AMICO clears the output directory on save, so the scheme and kernels live in work/.
    # One work directory per participant: with AMICO's shared default, concurrent runs collide.
    work = STUDY / "work" / s
    work.mkdir(parents=True, exist_ok=True)
    out.mkdir(parents=True, exist_ok=True)

    scheme = work / f"{s}.scheme"
    amico.util.fsl2scheme(str(bvals), str(bvecs), str(scheme), bStep=B_STEP)

    # Evaluation(study_dir, participant, output_path): the maps go straight to noddi/<subj>/.
    ae = amico.Evaluation(str(STUDY), s, str(out))
    # BLAS_nthreads=1 so the two thread counts do not multiply and oversubscribe the machine.
    ae.set_config("nthreads", int(N_THREADS))
    ae.set_config("BLAS_nthreads", 1)
    ae.set_config("doSaveModulatedMaps", True)
    ae.set_config("doComputeRMSE", True)
    ae.load_data(str(dwi), str(scheme), str(mask), b0_thr=B0_THRESHOLD)

    ae.set_model("NODDI")
    ae.set_config("ATOMS_path", str(work / "kernels"))     # after set_model, which resets it
    # model.set(dPar, dIso, IC_VFs, IC_ODs, isExvivo); only dPar changes here.
    if D_PAR:
        ae.model.set(float(D_PAR), D_ISO, ae.model.IC_VFs, ae.model.IC_ODs, False)
    # regenerate=True so a half-written kernel set from a killed run is never picked up.
    ae.generate_kernels(regenerate=True)
    ae.load_kernels()
    ae.fit()
    ae.save_results()
    print(f"[{s}] NODDI maps written (dPar = {ae.model.dPar})")

print(f"DONE -> {STUDY}")
```

08b_build_analysis_csv.py

```
#!/usr/bin/env python3
"""Step 8b. Reshape the Step 8 node profiles to one row per participant and join covariates.
Inputs:  $OUT/nodewise/<TRACT>_{nodewise_all_subjects,tract_stats}.csv (Steps 8 and 6)
         $COVARIATES_CSV (Subject, covariates, outcomes)
Output:  $OUT/analysis/<TRACT>__<METRIC>__analysis.csv, one per metric, named as Step 9 expects
Run:     source 00_config.sh && python 08b_build_analysis_csv.py
"""
import os
import sys
from pathlib import Path

import pandas as pd


# Read one exported config variable or stop; exports need `source 00_config.sh`, not `bash`.
def env(name):
    if name not in os.environ:
        sys.exit(f"{name} is not set: run `source 00_config.sh` first")
    return os.environ[name]


OUT = Path(env("OUT"))
TRACT = env("TRACT")

# Subject as str in all three files: all-digit IDs keep leading zeros and the merges match.
profiles = pd.read_csv(OUT / "nodewise" / f"{TRACT}_nodewise_all_subjects.csv",
                       dtype={"Subject": str})
tract_stats = pd.read_csv(OUT / "nodewise" / f"{TRACT}_tract_stats.csv", dtype={"Subject": str})
covariates = pd.read_csv(env("COVARIATES_CSV"), dtype={"Subject": str})
metrics = [c for c in profiles.columns if c not in ("Subject", "Tract", "Node")]

analysis_dir = OUT / "analysis"
analysis_dir.mkdir(parents=True, exist_ok=True)

for metric in metrics:
    wide = profiles.pivot(index="Subject", columns="Node", values=metric)
    wide.columns = [f"{metric}_{int(node)}" for node in wide.columns]
    bundle = tract_stats[["Subject", "Streamline_count", "Mean_length_mm"]]
    # Inner joins: anyone missing from an input is dropped, so N can be below subjects.txt.
    table = covariates.merge(bundle, on="Subject").merge(wide.reset_index(), on="Subject")
    path = analysis_dir / f"{TRACT}__{metric}__analysis.csv"
    table.to_csv(path, index=False)
    print(f"{path.name}: {len(table)} participants, {wide.shape[1]} nodes")

profiled = set(profiles["Subject"])
absent = sorted(profiled - set(covariates["Subject"]))
if absent:
    print(f"profiled but absent from the covariate file: {', '.join(absent)}")
```

09a_tract_models.py

```
#!/usr/bin/env python3
"""Step 9a. Collapse the 100 nodes into whole-tract and quartile means, regress each outcome
on each mean with the covariates, and with two tracts fit a mixed model of outcome x tract.

Input:  $OUT/analysis/<TRACT>__<METRIC>__analysis.csv   (Step 8b; --analysis-dir overrides)
Output: $OUT/analysis/<label>__<METRIC>__tract_models.csv   (<label> is the tract, or a+b)
Run:    source 00_config.sh
```

```
        python 09a_tract_models.py --metric NDI --outcomes memory_accuracy,memory_bias
        python 09a_tract_models.py --metric NDI --outcomes memory_accuracy --tracts a,b
"""
import argparse
import os
import sys
import warnings
from pathlib import Path

import pandas as pd
import statsmodels.formula.api as smf
from scipy.stats import chi2
from statsmodels.stats.multitest import multipletests


# Read a variable exported by 00_config.sh, or stop and say so.
def env(name):
    if name not in os.environ:
        sys.exit(f"{name} is not set: run `source 00_config.sh` first")
    return os.environ[name]


parser = argparse.ArgumentParser(description="Whole-tract and quartile models")
parser.add_argument("--metric", required=True, help="metric prefix, e.g. NDI")
parser.add_argument("--outcomes", required=True, help="comma-separated outcome columns")
parser.add_argument("--tracts", default=None, help="one tract, or two separated by a comma")
parser.add_argument("--trim", type=int, default=0, help="nodes excluded at each end")
parser.add_argument("--analysis-dir", default=None)
args = parser.parse_args()

METRIC = args.metric
OUTCOMES = args.outcomes.split(",")
TRACTS = (args.tracts or env("TRACT")).split(",")
COVARIATES = env("COVARIATES").split(",")
ANALYSIS = Path(args.analysis_dir) if args.analysis_dir else Path(env("OUT")) / "analysis"
# The mixed model is written for exactly two tracts; trimming 25+ nodes would empty Q1 or Q4.
if len(TRACTS) > 2:
    sys.exit("--tracts takes one tract or two")
if not 0 <= args.trim < 25:
    sys.exit("--trim must be between 0 and 24 (nodes excluded at each end)")

# Half-open node ranges. Node 0 is the seed end and 99 the target end (Step 8 orients every
# bundle that way), so Q1 sits at the seed and Q4 at the target. --trim shortens Whole, Q1, Q4.
SEGMENTS = {"Whole": (args.trim, 100 - args.trim), "Q1": (args.trim, 25), "Q2": (25, 50),
            "Q3": (50, 75), "Q4": (75, 100 - args.trim)}


def q(column):
    """Quote a column name for patsy, so names like memory-bias or 2back work in a formula."""
    return f'Q("{column}")'


# Covariate part of every formula, so all fits are adjusted identically.
RHS = " + ".join(q(c) for c in COVARIATES)


# Standardize each column (sample SD, as R's scale() does), so the betas are standardized.
def zscore(frame):
    return (frame - frame.mean()) / frame.std(ddof=1)


# Read one tract's Step 8b table (Subject as str keeps leading zeros); stop on missing columns.
def load(tract):
```

```
    table = pd.read_csv(ANALYSIS / f"{tract}__{METRIC}__analysis.csv", dtype={"Subject": str})
    absent = [c for c in COVARIATES + OUTCOMES if c not in table.columns]
    if absent:
        sys.exit(f"{tract}: columns not found: {', '.join(absent)}")
    return table


# Per-participant mean of the metric over one segment's nodes; NaN nodes are skipped.
def segment_mean(table, segment):
    lo, hi = SEGMENTS[segment]
    return table[[f"{METRIC}_{node}" for node in range(lo, hi)]].mean(axis=1)


def single_tract(tract):
    """OLS per segment, FDR across the quartiles, and the uniformity contrast."""
    table = load(tract)
    rows = []
    for outcome in OUTCOMES:
        quartile_rows = []
        for segment in SEGMENTS:
            d = table[COVARIATES + [outcome]].assign(metric=segment_mean(table, segment))
            d = d.dropna()
            fit = smf.ols(f"{q(outcome)} ~ metric + {RHS}", zscore(d)).fit()
            row = {"model": "ols", "tract": tract, "segment": segment, "outcome": outcome,
                   "term": "metric", "n": int(fit.nobs), "beta": fit.params["metric"],
                   "statistic": fit.tvalues["metric"], "p": fit.pvalues["metric"]}
            rows.append(row)
            if segment != "Whole":
                quartile_rows.append(row)
        # BH across this outcome's four quartiles; quartile_rows shares its dicts with rows.
        adjusted = multipletests([r["p"] for r in quartile_rows], method="fdr_bh")[1]
        for row, p_fdr in zip(quartile_rows, adjusted):
            row["p_fdr"] = p_fdr

        # mean(Q1, Q4) catches an effect shared by both ends, Q4 - Q1 one stronger at one end.
        q1, q4 = segment_mean(table, "Q1"), segment_mean(table, "Q4")
        d = table[COVARIATES + [outcome]].assign(mean_q1q4=(q1 + q4) / 2, diff_q4q1=q4 - q1)
        d = d.dropna()
        fit = smf.ols(f"{q(outcome)} ~ mean_q1q4 + diff_q4q1 + {RHS}", zscore(d)).fit()
        for term in ("mean_q1q4", "diff_q4q1"):
            rows.append({"model": "uniformity", "tract": tract, "segment": "Q1,Q4",
                         "outcome": outcome, "term": term, "n": int(fit.nobs),
                         "beta": fit.params[term], "statistic": fit.tvalues[term],
                         "p": fit.pvalues[term]})
    return rows


def two_tracts(tract_a, tract_b):
    """Mixed model across two stacked tracts; likelihood-ratio test of outcome x tract."""
    tables = {tract: load(tract) for tract in (tract_a, tract_b)}
    rows = []
    for outcome in OUTCOMES:
        for segment in SEGMENTS:
            # Two rows per participant, each with its own tract's segment mean, streamline
            # count and length (Step 6 measured those per tract).
            stacked = pd.concat([
                t[["Subject", outcome] + COVARIATES].assign(metric=segment_mean(t, segment),
                                                            tract=name)
                for name, t in tables.items()], ignore_index=True).dropna()
            # Standardize pooled, so an offset between tracts goes to the tract term.
            numeric = ["metric", outcome] + COVARIATES
            stacked[numeric] = zscore(stacked[numeric])
            # metric is the DV since each participant has two of them and one outcome.
```

```
            # reml=False: the LR test needs ML fits. mixedlm's boundary warnings are muted.
            with warnings.catch_warnings():
                warnings.simplefilter("ignore")
                main = smf.mixedlm(f"metric ~ {q(outcome)} + tract + {RHS}", stacked,
                                   groups=stacked["Subject"]).fit(reml=False)
                full = smf.mixedlm(f"metric ~ {q(outcome)} * tract + {RHS}", stacked,
                                   groups=stacked["Subject"]).fit(reml=False)
            # Likelihood ratio on 1 df; max(..., 0) clamps a tiny negative from the optimiser.
            lr = max(2 * (full.llf - main.llf), 0)
            # Main effect from the additive model (term Q("<outcome>")); n counts participants.
            rows.append({"model": "mixed", "tract": f"{tract_a}+{tract_b}", "segment": segment,
                         "outcome": outcome, "term": outcome,
                         "n": stacked["Subject"].nunique(), "beta": main.params[q(outcome)],
                         "statistic": main.tvalues[q(outcome)], "p": main.pvalues[q(outcome)],
                         "interaction_p": chi2.sf(lr, 1)})
    return rows


results = []
for tract in TRACTS:
    results += single_tract(tract)
if len(TRACTS) == 2:
    results += two_tracts(*TRACTS)

# "+" in the file name keeps a two-tract run from overwriting a single-tract file.
results = pd.DataFrame(results)
label = "+".join(TRACTS)
path = ANALYSIS / f"{label}__{METRIC}__tract_models.csv"
results.to_csv(path, index=False)
with pd.option_context("display.width", 200, "display.float_format", "{:.3f}".format):
    print(results.to_string(index=False))
print(f"\n-> {path}")
```

09b_nodewise_permutation.R

```
# ============================================================================
# Step 9b. Node-wise cluster-extent permutation test (Freedman-Lane)
# ============================================================================
# One lm per node (outcome ~ metric_node + covariates); runs of p < .05 nodes form
# clusters, tested against the longest cluster per permutation of reduced-model residuals.
# Input:  $OUT/analysis/<TRACT>__<METRIC>__analysis.csv from Step 8b
# Output: <out_dir>/<label>_{nodewise,clusters,summary}.csv (label = tract__metric__outcome)
# Run:    source 00_config.sh (COVARIATES, N_PERMUTATIONS, R_PERM_CORES), then
#   Rscript 09b_nodewise_permutation.R <analysis.csv> <outcome> <METRIC>_ <out_dir> <label>
# ============================================================================
suppressPackageStartupMessages({
  library(readr); library(dplyr); library(stringr)
  library(foreach); library(doParallel); library(parallel); library(tibble)
})

args <- commandArgs(trailingOnly = TRUE)
stopifnot(length(args) >= 5)
data_csv      <- args[1]
response_col  <- args[2]
metric_prefix <- args[3]    # "NDI_", underscore included; "NDI" alone matches nothing
out_dir       <- args[4]
base          <- args[5]    # <tract>__<metric>__<outcome>; every output name starts with it

# Covariates: comma-separated column names, exported by 00_config.sh
# Same list as Step 9a, so the two resolutions are comparable.
covariate_env <- Sys.getenv("COVARIATES", unset = "")
```

```
if (covariate_env == "") stop("COVARIATES is not set: run `source 00_config.sh` first")
covariate_cols <- strsplit(covariate_env, ",")[[1]]

# Settings.  alpha_node forms clusters; alpha_familywise is applied to the cluster p values.
alpha_node       <- 0.05
alpha_familywise <- 0.05
num_permutations <- as.integer(Sys.getenv("N_PERMUTATIONS", unset = "5000"))
rng_seed         <- 123
use_parallel     <- TRUE

dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)

dat <- readr::read_csv(data_csv, show_col_types = FALSE)

# Collect node columns
node_cols <- grep(paste0("^", metric_prefix, "\\d+$"), names(dat), value = TRUE)
if (length(node_cols) == 0) {
  stop("No columns matching ", metric_prefix, "0.. in: ", data_csv)
}
# Sort by node number; the cluster code relies on node_idx increasing along the vector.
node_idx  <- as.integer(sub(paste0("^", metric_prefix), "", node_cols))
ord       <- order(node_idx)
node_cols <- node_cols[ord]; node_idx <- node_idx[ord]
num_nodes <- length(node_cols)

# Numeric casts (a stray text value makes readr read the whole column as text)
dat[[response_col]] <- as.numeric(dat[[response_col]])
for (cc in covariate_cols) dat[[cc]] <- as.numeric(dat[[cc]])

# Drop incomplete.  A participant missing any node is dropped entirely, so all nodes are
# fit on the same people and one shuffled residual vector serves every node.
all_model_cols <- c(response_col, covariate_cols, node_cols)
mask <- complete.cases(dat[, all_model_cols])
n_dropped <- sum(!mask)
if (n_dropped) message("Dropping ", n_dropped, " incomplete participants")
dat <- dat[mask, , drop = FALSE]
n_subj <- nrow(dat)
# Exit 0 so a shell loop under set -e carries on; the missing outputs mark the skip.
min_subj <- length(covariate_cols) + 3   # outcome, node, covariates, plus 2 residual df
if (n_subj < max(5, min_subj)) {
  message("Only ", n_subj, " complete participants for ", length(covariate_cols),
          " covariates; at least ", max(5, min_subj), " are needed - skipping ", base)
  quit(status = 0)
}

y <- dat[[response_col]]

# Formulas
# The full model tests the node; the reduced model (covariates only) is what gets permuted.
full_formula <- as.formula(
  paste("y ~ node +", paste(covariate_cols, collapse = " + "))
)
red_formula  <- as.formula(
  paste("y ~", paste(covariate_cols, collapse = " + "))
)

# Nodewise fit
# OLS of the full model for one node; NA statistics when the node cannot be tested.
fit_node_full <- function(y, node, covariates_df) {
  df0 <- data.frame(y = y, node = node, covariates_df)
  df0 <- df0[complete.cases(df0), ]
  if (nrow(df0) < 3 || sd(df0$node) == 0)
    return(c(Estimate=NA, t=NA, p=NA, df=NA, n=nrow(df0)))
```

```
  fit <- lm(full_formula, data = df0)
  sm <- summary(fit)$coefficients
  if (!("node" %in% rownames(sm)))
    return(c(Estimate=NA, t=NA, p=NA, df=fit$df.residual, n=nrow(df0)))
  # p recomputed with the same formula the permutation loop uses, so the two agree.
  tval <- unname(sm["node","t value"])
  est  <- unname(sm["node","Estimate"])
  pval <- 2*pt(-abs(tval), df=fit$df.residual)
  c(Estimate=est, t=tval, p=pval, df=fit$df.residual, n=nrow(df0))
}

# Observed statistics
set.seed(rng_seed)
node_stats <- lapply(seq_along(node_cols), function(i) {
  fit_node_full(y, dat[[node_cols[i]]], dat[, covariate_cols, drop=FALSE])
})
node_stats <- do.call(rbind, node_stats)
node_stats_df <- tibble(
  Node=node_idx, Estimate=node_stats[,"Estimate"],
  t_value=node_stats[,"t"], p_value=node_stats[,"p"],
  df=node_stats[,"df"], n=node_stats[,"n"]
)
write_csv(node_stats_df, file.path(out_dir, paste0(base, "_nodewise.csv")))

# Cluster helpers.  Runs of consecutive node numbers among the significant nodes; one
# rule for the observed pass and every permutation, so they cannot drift apart.
clusters_from_sig <- function(sig, nodes_numeric) {
  sig[is.na(sig)] <- FALSE          # a node whose model could not be fitted is not significant
  if (!any(sig)) return(list())
  idx <- which(sig); cls <- list(); run <- c(nodes_numeric[idx[1]])
  if (length(idx) > 1) for (k in 2:length(idx)) {
    if (nodes_numeric[idx[k]] == nodes_numeric[idx[k-1]] + 1)
      run <- c(run, nodes_numeric[idx[k]])
    else { cls[[length(cls)+1]] <- run; run <- c(nodes_numeric[idx[k]]) }
  }
  cls[[length(cls)+1]] <- run; cls
}
# Longest cluster, or 0 when there is none; this one number per permutation is the null.
max_cluster_size_from_sig <- function(sig, nodes_numeric) {
  cls <- clusters_from_sig(sig, nodes_numeric)
  if (length(cls)==0) 0L else max(vapply(cls, length, 1L))
}

# Observed clusters; nodes with an NA p count as not significant and end a cluster.
sig_mask <- !is.na(node_stats_df$p_value) & node_stats_df$p_value < alpha_node
obs_clusters <- clusters_from_sig(sig_mask, node_stats_df$Node)
obs_sizes <- vapply(obs_clusters, length, 1L)
obs_max_size <- if (length(obs_sizes)) max(obs_sizes) else 0L
num_sig_nodes <- sum(sig_mask)
num_clusters  <- length(obs_clusters)

# Freedman-Lane: permuted y = reduced-model fit + shuffled residuals, so what the
# covariates explain stays with its owner (Freedman & Lane 1983; Winkler et al. 2014).
df_red_global <- data.frame(y=y, dat[, covariate_cols, drop=FALSE])
fit_red <- lm(red_formula, data=df_red_global)
yhat_red <- fitted(fit_red); resid_red <- resid(fit_red)

# Constant nodes have no slope to test; checked once here rather than inside the loop.
analysable <- vapply(seq_along(node_cols), function(i)
  sd(dat[[node_cols[i]]]) > 0, logical(1))

# All permutations are drawn once, up front, so the null is identical for any core count.
set.seed(rng_seed)
```

```
perm_mat <- vapply(seq_len(num_permutations), function(k) sample.int(n_subj),
                   integer(n_subj))

# One permutation: refit every analysable node, return the longest run of p < alpha_node.
perm_fun <- function(.perm) {
  perm_idx <- perm_mat[, .perm]
  y_perm <- yhat_red + resid_red[perm_idx]
  p_perm <- rep(1, num_nodes)
  for (i in which(analysable)) {
    df_perm <- data.frame(y=y_perm, node=dat[[node_cols[i]]],
                          dat[, covariate_cols, drop=FALSE])
    fit <- lm(full_formula, data=df_perm)
    sm <- summary(fit)$coefficients
    if ("node" %in% rownames(sm)) {
      tval <- sm["node","t value"]
      p_perm[i] <- 2*pt(-abs(tval), df=fit$df.residual)
    }
  }
  max_cluster_size_from_sig(p_perm < alpha_node, node_idx)
}

# Core count: R_PERM_CORES overrides; otherwise every core but one.
env_cores <- suppressWarnings(as.integer(Sys.getenv("R_PERM_CORES", unset = NA)))
if (!is.na(env_cores) && env_cores > 0L) {
  cores <- env_cores
} else {
  cores <- max(1L, parallel::detectCores() - 1L)
}

# Run the permutations (a failing task says nothing useful; rerun with R_PERM_CORES=1).
if (cores > 1L) {
  cl <- parallel::makeCluster(cores); registerDoParallel(cl)
  perm_max_sizes <- foreach(perm = 1:num_permutations, .combine = c,
                            .packages = "stats") %dopar% perm_fun(perm)
  parallel::stopCluster(cl)
} else {
  perm_max_sizes <- vapply(1:num_permutations, perm_fun, integer(1))
}

# Cluster p: proportion of permutations whose longest cluster is at least as long.
# Extent threshold: one node above the (1 - alpha) quantile; type = 1 keeps it whole.
cluster_p_from_size <- function(size) mean(perm_max_sizes >= size)
extent_threshold <- as.integer(quantile(perm_max_sizes, probs = 1 - alpha_familywise,
                                        type = 1)) + 1L

# Cluster table.  PassExtentThreshold comes from the p value, so the two cannot disagree.
if (num_clusters > 0) {
  all_clusters_df <- do.call(rbind, lapply(seq_along(obs_clusters), function(k) {
    nodes <- obs_clusters[[k]]; idx_in_table <- match(nodes, node_stats_df$Node)
    t_vals <- node_stats_df$t_value[idx_in_table]
    ests   <- node_stats_df$Estimate[idx_in_table]
    tibble(
      ClusterID=k, Size=length(nodes), StartNode=min(nodes), EndNode=max(nodes),
      Nodes=paste(nodes, collapse=","),
      MeanTValue=mean(t_vals, na.rm=TRUE),
      Direction=ifelse(mean(t_vals, na.rm=TRUE)>0,"Positive","Negative"),
      MaxAbsTValue=t_vals[which.max(abs(t_vals))],
      MaxAbsTNode=nodes[which.max(abs(t_vals))],
      MeanEstimate=mean(ests, na.rm=TRUE),
      ClusterPValue=cluster_p_from_size(length(nodes)),
      ExtentThresholdNodes=extent_threshold,
      PassExtentThreshold=cluster_p_from_size(length(nodes)) <= alpha_familywise
    )
```

```
  }))
} else {
  # Header-only file with the same columns, so Step 9c still finds what it expects.
  all_clusters_df <- tibble(
    ClusterID=integer(), Size=integer(), StartNode=integer(), EndNode=integer(),
    Nodes=character(), MeanTValue=double(), Direction=character(),
    MaxAbsTValue=double(), MaxAbsTNode=integer(),
    MeanEstimate=double(),
    ClusterPValue=double(), ExtentThresholdNodes=integer(), PassExtentThreshold=logical()
  )
}
write_csv(all_clusters_df, file.path(out_dir, paste0(base, "_clusters.csv")))

# Summary.  Step 9c reads N_subjects, Covariates and ExtentThresholdNodes by name.
summary_df <- tibble(
  Outcome=response_col,
  MetricPrefix=metric_prefix,
  Covariates=paste(covariate_cols, collapse=", "),
  N_subjects=n_subj, N_dropped=n_dropped,
  NodesTested=num_nodes,
  NodewiseAlpha=alpha_node, FamilywiseAlpha=alpha_familywise,
  NumPermutations=num_permutations,
  NumNodewiseSignificant=num_sig_nodes,
  NumClustersFormed=num_clusters,
  ObservedMaxClusterSize=obs_max_size,
  ExtentThresholdNodes=extent_threshold,
  NumClustersPassingExtent=sum(all_clusters_df$PassExtentThreshold, na.rm=TRUE)
)
write_csv(summary_df, file.path(out_dir, paste0(base, "_summary.csv")))
message("Done: ", base, " — clusters=", num_clusters,
        " sig nodes=", num_sig_nodes, " ext_thr=", extent_threshold)
```

09c_stack_for_explorer.py

```
#!/usr/bin/env python3
"""Step 9c. Stack the Step 9b node-wise results into the one CSV the Explorer reads.

Inputs:  $OUT/permutation/<tract>__<metric>__<outcome>_{summary,nodewise,clusters}.csv
Output:  $OUT/permutation/results_long.csv (one row per node; overwritten every run)
Run:     python 09c_stack_for_explorer.py "$OUT/permutation"
"""
import sys
from pathlib import Path

import pandas as pd

results_dir = Path(sys.argv[1] if len(sys.argv) > 1 else ".")
rows = []

for summary_file in sorted(results_dir.glob("*_summary.csv")):
    label = summary_file.name[:-len("_summary.csv")]
    # maxsplit 2 keeps an outcome that itself contains "__" in one piece
    parts = label.split("__", 2)
    if len(parts) != 3:
        print(f"skipped {summary_file.name}: label is not <tract>__<metric>__<outcome>")
        continue
    tract, metric, outcome = parts

    summary = pd.read_csv(summary_file).iloc[0]
    nodewise = pd.read_csv(results_dir / f"{label}_nodewise.csv")
    clusters = pd.read_csv(results_dir / f"{label}_clusters.csv")
```

```
    # an empty clusters file reads back as object dtype, not bool, so we compare as text
    passing = clusters[clusters["PassExtentThreshold"].astype(str).str.upper() == "TRUE"]

    tokens = tract.lower().split("_")
    hemisphere = next((t.upper() for t in tokens if t in ("l", "r")), "")

    for node in nodewise.itertuples():
        rows.append({
            "outcome": outcome, "tract": tract, "metric": metric,
            "node": int(node.Node), "t": round(node.t_value, 4), "p": round(node.p_value, 5),
            "hemisphere": hemisphere, "N": int(summary["N_subjects"]),
            "covariates": summary["Covariates"],
            "extent_threshold": int(summary["ExtentThresholdNodes"]),
            "cluster_p": passing["ClusterPValue"].min() if len(passing) else "",
            "passed": int(len(passing) > 0),
        })

if not rows:
    sys.exit(f"no *_summary.csv files found in {results_dir}")
out = results_dir / "results_long.csv"
# index=False because the Explorer treats any column it does not know as a grouping variable
pd.DataFrame(rows).to_csv(out, index=False)
# assumes 100 nodes per analysis
print(f"wrote {out} ({len(rows)} rows, {len(rows) // 100} analyses)")
```

02s_warp_atlas_streamlines.py

```
#!/usr/bin/env python3
"""Step 2s. Carry the atlas streamlines from MNI space into each participant's T1 space.

This is the streamline alternative to Steps 2 to 6. Nothing is tracked in the participant:
every vertex of the atlas .tck is pushed through the Step 1 registration, so the bundle that
comes out has the atlas's geometry placed on the participant's anatomy.

Inputs, per participant:
    $OUT/<s>/reg/mni2t1_0GenericAffine.mat     Step 1 affine
    $OUT/<s>/reg/mni2t1_1InverseWarp.nii.gz    Step 1 inverse warp field
    $PROJECT/anat/<s>/<s>_T1w_brain.nii.gz     fixed image of Step 1; also the audit mask
    $ATLAS_STREAMLINES_MNI                     atlas streamlines, MNI space (atlas package)
Output: $OUT/<s>/atlas_streamlines/<TRACT>_atlas_t1.tck
Run:    source 00_config.sh && python 02s_warp_atlas_streamlines.py
        (FORCE=1 redoes finished participants)
"""
import os
import subprocess
import sys
import tempfile
from pathlib import Path

import nibabel as nib
import numpy as np


# Read one variable exported by 00_config.sh, or stop with a hint.
def env(name):
    if name not in os.environ:
        sys.exit(f"{name} is not set: run `source 00_config.sh` first")
    return os.environ[name]


PROJECT = Path(env("PROJECT"))
```

```
OUT = Path(env("OUT"))
TRACT = env("TRACT")
ATLAS_TCK = Path(env("ATLAS_STREAMLINES_MNI"))
FORCE = os.environ.get("FORCE", "0") == "1"
# bash cannot export an array, so the ID list is re-read from the file.
SUBJECTS = Path(env("SUBJECTS_FILE")).read_text().split()

# antsApplyTransformsToPoints reads and writes LPS, nibabel gives and takes RAS.
# Flipping x and y converts between them, and the same multiplier undoes it afterwards.
RAS_TO_LPS = np.array([-1.0, -1.0, 1.0])

if not ATLAS_TCK.is_file():
    sys.exit(f"atlas streamlines not found: {ATLAS_TCK}")

atlas = nib.streamlines.load(str(ATLAS_TCK)).streamlines
if len(atlas) == 0:
    sys.exit(f"{ATLAS_TCK} holds no streamlines")
# Vertex counts are the only record of where one streamline ends and the next begins,
# because every vertex of every streamline is transformed in a single call.
lengths = [len(s) for s in atlas]
mni_points = np.vstack(atlas)
print(f"== atlas: {len(atlas)} streamlines, {len(mni_points)} vertices, "
      f"{min(lengths)}-{max(lengths)} points each")


def warp_points(points, affine, inverse_warp):
    """Push an (N, 3) array of RAS millimetre coordinates from MNI into T1 space."""
    with tempfile.TemporaryDirectory() as tmp:
        src = Path(tmp) / "mni_lps.csv"
        dst = Path(tmp) / "t1_lps.csv"
        # comments='' keeps numpy from prefixing the header with "# ", which ANTs cannot read.
        np.savetxt(src, points * RAS_TO_LPS, delimiter=",", fmt="%.7f",
                   header="x,y,z", comments="")
        # Points move in the opposite sense to images: Step 2 warps the atlas IMAGE with the
        # forward warp and affine, so the same registration moves POINTS with the inverted
        # affine ([...,1]) and the inverse warp field. Do not substitute mni2t1_1Warp here.
        subprocess.run(
            ["antsApplyTransformsToPoints", "-d", "3",
             "-i", str(src), "-o", str(dst),
             "-t", f"[{affine},1]", "-t", str(inverse_warp)],
            check=True, stdout=subprocess.DEVNULL)
        # ANTs writes one header row and may add columns beyond x, y, z.
        out = np.loadtxt(dst, delimiter=",", skiprows=1, usecols=(0, 1, 2), ndmin=2)
    return out * RAS_TO_LPS


def fraction_in_brain(points, brain):
    """Share of vertices landing inside the non-zero part of the T1 brain image."""
    img = nib.load(str(brain))
    data = np.asarray(img.dataobj)
    ijk = np.rint(nib.affines.apply_affine(np.linalg.inv(img.affine), points)).astype(int)
    inside = np.all((ijk >= 0) & (ijk < np.array(data.shape)), axis=1)
    hit = np.zeros(len(points), dtype=bool)
    hit[inside] = data[tuple(ijk[inside].T)] > 0
    return hit.mean()


def warp_one(s):
    d = OUT / s / "atlas_streamlines"
    out_tck = d / f"{TRACT}_atlas_t1.tck"
    reg = OUT / s / "reg"
    affine = reg / "mni2t1_0GenericAffine.mat"
    inverse_warp = reg / "mni2t1_1InverseWarp.nii.gz"
```

```
    if not FORCE and out_tck.is_file():
        print(f"== {s} already warped")
        return
    missing = [p for p in (affine, inverse_warp) if not p.is_file()]
    if missing:
        print(f"!! {s} missing {missing[0]}")
        return
    d.mkdir(parents=True, exist_ok=True)

    t1_points = warp_points(mni_points, affine, inverse_warp)
    # The only automatic check that the transform did what was asked: every vertex came back,
    # and none of them is NaN or infinite. Anatomy still has to be inspected by eye.
    if t1_points.shape != mni_points.shape or not np.isfinite(t1_points).all():
        print(f"!! {s} transform returned {t1_points.shape}, expected {mni_points.shape}")
        return

    # float32 is the .tck storage type; the identity affine declares the vertices as
    # world millimetres, which is what Step 8's reader expects.
    streamlines = np.split(t1_points, np.cumsum(lengths)[:-1])
    tractogram = nib.streamlines.Tractogram(
        [np.asarray(s_, dtype=np.float32) for s_ in streamlines],
        affine_to_rasmm=np.eye(4))
    partial = out_tck.with_suffix(".partial.tck")
    nib.streamlines.save(tractogram, str(partial))
    partial.rename(out_tck)
    print(f">> {s} {len(streamlines)} streamlines")


for s in SUBJECTS:
    warp_one(s)

# --- Audit: a bundle that lands largely outside the brain means the registration is wrong ---
print(f"\nSubject\tStreamlines\tIn_brain")
for s in SUBJECTS:
    tck = OUT / s / "atlas_streamlines" / f"{TRACT}_atlas_t1.tck"
    brain = PROJECT / "anat" / s / f"{s}_T1w_brain.nii.gz"
    if not tck.is_file():
        print(f"{s}\tMISSING")
        continue
    warped = nib.streamlines.load(str(tck)).streamlines
    if brain.is_file():
        share = fraction_in_brain(np.vstack(warped), brain)
        print(f"{s}\t{len(warped)}\t{share:.3f}")
    else:
        print(f"{s}\t{len(warped)}\tno T1")
```

00b_fod_estimation.sh

```
#!/bin/bash
# ================================================================================
# Step 0b. Fibre orientation distributions (multi-shell multi-tissue CSD)
# ================================================================================
# Estimates per-participant response functions, averages them across the sample and
# deconvolves everyone with that average, so FOD amplitudes and CUTOFF are comparable.
# Inputs:  $PROJECT/dwi/<ID>/{data.nii.gz,bvals,bvecs,nodif_brain_mask.nii.gz}
# Outputs: $PROJECT/dwi/<ID>/{wm,gm,csf}_fod_norm.mif and $PROJECT/dwi/group_*_response.txt
# Run:     bash 00b_fod_estimation.sh   (FORCE=1 redoes everything; hours, use tmux)
# ================================================================================
# Single-shell data cannot use this multi-tissue model: use dwi2response tournier and
# dwi2fod csd (single-shell CSD) instead, as described on the software page.
```

```
source "$(dirname "$0")/00_config.sh"
start_log "$0"
read_subjects

# Phase 1 worker: convert one participant to .mif and estimate WM, GM and CSF responses.
estimate_response() {
  local s=$1
  local d="$PROJECT/dwi/$s"
  local f
  for f in data.nii.gz bvals bvecs nodif_brain_mask.nii.gz; do
    if [ ! -f "$d/$f" ]; then
      echo "!! $s missing $d/$f"
      return
    fi
  done
  if [ "$FORCE" != 1 ] && [ -f "$d/wm_response.txt" ]; then
    echo "== $s response functions exist"
    return
  fi
  # -fslgrad bvecs bvals: put the gradient table in the header, so later steps need no -fslgrad.
  mrconvert "$d/data.nii.gz" "$d/dwi.mif" -fslgrad "$d/bvecs" "$d/bvals" -force -quiet
  mrconvert "$d/nodif_brain_mask.nii.gz" "$d/mask.mif" -force -quiet
  # dhollander: unsupervised WM, GM and CSF response estimation.
  dwi2response dhollander "$d/dwi.mif" \
    "$d/wm_response.txt" "$d/gm_response.txt" "$d/csf_response.txt" \
    -mask "$d/mask.mif" -force -quiet
  echo ">> $s response functions"
}

# Phase 3 worker: msmt-CSD with the group-average responses, then mtnormalise.
estimate_fod() {
  local s=$1
  local d="$PROJECT/dwi/$s"
  local g="$PROJECT/dwi"
  if [ "$FORCE" != 1 ] && [ -f "$d/wm_fod_norm.mif" ]; then
    echo "== $s FOD exists"
    return
  fi
  # Group responses, not the participant's own, so every FOD is deconvolved the same way.
  dwi2fod msmt_csd "$d/dwi.mif" \
    "$g/group_wm_response.txt"  "$d/wm_fod.mif" \
    "$g/group_gm_response.txt"  "$d/gm_fod.mif" \
    "$g/group_csf_response.txt" "$d/csf_fod.mif" \
    -mask "$d/mask.mif" -nthreads "$NTHREADS" -force -quiet
  # wm_fod_norm.mif means "done", so write a temp name and rename only if mtnormalise succeeds.
  if mtnormalise \
    "$d/wm_fod.mif"  "$d/wm_fod_norm.partial.mif" \
    "$d/gm_fod.mif"  "$d/gm_fod_norm.mif" \
    "$d/csf_fod.mif" "$d/csf_fod_norm.mif" \
    -mask "$d/mask.mif" -force -quiet; then
    mv -f "$d/wm_fod_norm.partial.mif" "$d/wm_fod_norm.mif"
    echo ">> $s FOD"
  else
    rm -f "$d/wm_fod_norm.partial.mif"
    echo "!! $s FOD estimation failed or was interrupted"
  fi
}

# --- Phase 1: response functions, $MAXJOBS participants at a time ---
for s in "${SUBJECTS[@]}"; do
  estimate_response "$s" &
  throttle "$MAXJOBS"
done
```

```
# Not a bare "wait": the start_log logger never exits and bash 5.0-5.2 would wait on it too.
wait_for_jobs

# --- Phase 2: group-average response functions ---
# No FORCE guard (cheap). The glob takes every folder in $PROJECT/dwi, not just $SUBJECTS_FILE.
for tissue in wm gm csf; do
  responsemean "$PROJECT"/dwi/*/${tissue}_response.txt \
    "$PROJECT/dwi/group_${tissue}_response.txt" -force -quiet
done
echo "== group response functions written"

# --- Phase 3: CSD + normalisation, $FOD_JOBS at a time (dwi2fod is memory-heavy) ---
for s in "${SUBJECTS[@]}"; do
  estimate_fod "$s" &
  throttle "$FOD_JOBS"
done
wait_for_jobs

# --- Audit: ok or MISSING for the file Steps 4 and 5 track on ---
printf "\nSubject\twm_fod_norm\n"
for s in "${SUBJECTS[@]}"; do
  printf "%s\t%s\n" "$s" "$(present "$PROJECT/dwi/$s/wm_fod_norm.mif")"
done
```